Audio Course

Audio Transformers Course

By Hugging Face

github.com/huggingface/audio-transformers-course

Pre-trained models and datasets for audio classification

The Hugging Face Hub is home to over 500 pre-trained models for audio classification. In this section, we'll go through some of the most common audio classification tasks and suggest appropriate pre-trained models for each. Using the `pipeline()` class, switching between models and tasks is straightforward - once you know how to use `pipeline()` for one model, you'll be able to use it for any model on the Hub no code changes! This makes experimenting with the `pipeline()` class extremely fast, allowing you to quickly select the best pre-trained model for your needs.

Before we jump into the various audio classification problems, let's quickly recap the transformer architectures typically used. The standard audio classification architecture is motivated by the nature of the task; we want to transform a sequence of audio inputs (i.e. our input audio array) into a single class label prediction. Encoder-only models first map the input audio sequence into a sequence of hidden-state representations by passing the inputs through a transformer block. The sequence of hidden-state representations is then mapped to a class label output by taking the mean over the hidden-states, and passing the resulting vector through a linear classification layer. Hence, there is a preference for *encoder-only* models for audio classification.

Decoder-only models introduce unnecessary complexity to the task, since they assume that the outputs can also be a *sequence* of predictions (rather than a single class label prediction), and so generate multiple outputs. Therefore, they have slower inference speed and tend not to be used. Encoder-decoder models are largely omitted for the same reason. These architecture choices are analogous to those in NLP, where encoder-only models such as BERT are favoured for sequence classification tasks, and decoder-only models such as GPT reserved for sequence generation tasks.

Now that we've recapped the standard transformer architecture for audio classification, let's jump into the different subsets of audio classification and cover the most popular models!

🤗 Transformers Installation

At the time of writing, the latest updates required for audio classification pipeline are only on the `main` version of the 🤗 Transformers repository, rather than the latest PyPi version. To make sure we have these updates locally, we'll install Transformers from the `main` branch with the following command:

```
pip install git+https://github.com/huggingface/transformers
```

Keyword Spotting

Keyword spotting (KWS) is the task of identifying a keyword in a spoken utterance. The set of possible keywords forms the set of predicted class labels. Hence, to use a pre-trained keyword spotting model, you should ensure that your keywords match those that the model was pre-trained on. Below, we'll introduce two datasets and models for keyword spotting.

Minds-14

Let's go ahead and use the same MINDS-14 dataset that you have explored in the previous unit. If you recall, MINDS-14 contains recordings of people asking an e-banking system questions in several languages and dialects, and has the `intent_class` for each recording. We can classify the recordings by intent of the call.

```
from datasets import load_dataset

minds = load_dataset("PolyAI/minds14", name="en-AU", split="train")
```

We'll load the checkpoint `"anton-l/xtreme_s_xlsr_300m_minds14"`, which is an XLS-R model fine-tuned on MINDS-14 for approximately 50 epochs. It achieves 90% accuracy over all languages from MINDS-14 on the evaluation set.

```
from transformers import pipeline

classifier = pipeline(
    "audio-classification",
    model="anton-l/xtreme_s_xlsr_300m_minds14",
)
```

Finally, we can pass a sample to the classification pipeline to make a prediction:

```
classifier(minds[0]["audio"])
```

Output:

```
[
    ,
    ,
    ,
    ,
    ,
]
```

Great! We've identified that the intent of the call was paying a bill, with probability 96%. You can imagine this kind of keyword spotting system being used as the first stage of an automated call centre, where we want to categorise incoming customer calls based on their query and offer them contextualised support accordingly.

Speech Commands

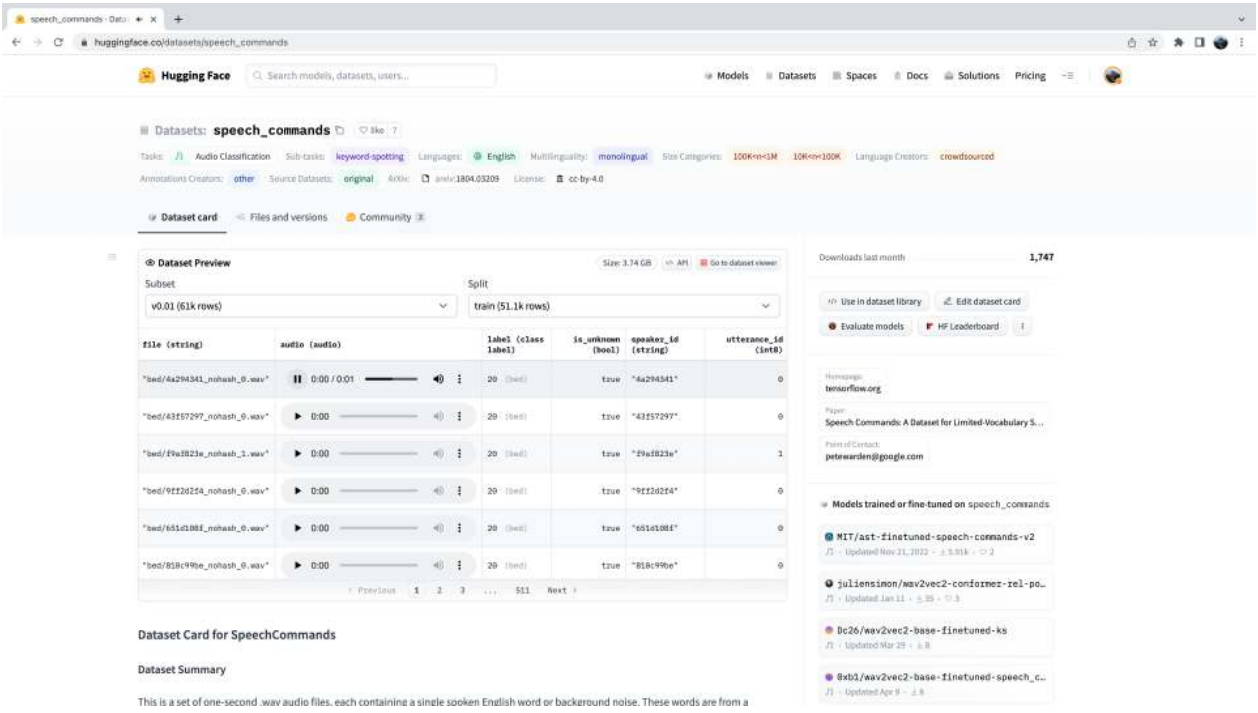
Speech Commands is a dataset of spoken words designed to evaluate audio classification models on simple command words. The dataset consists of 15 classes of keywords, a class for silence, and an unknown class to include the false positive. The 15 keywords are single words that would typically be used in on-device settings to control basic tasks or launch other processes.

A similar model is running continuously on your mobile phone. Here, instead of having single command words, we have 'wake words' specific to your device, such as "Hey Google" or "Hey Siri". When the audio classification model detects these wake words, it triggers your phone to start listening to the microphone and transcribe your speech using a speech recognition model.

The audio classification model is much smaller and lighter than the speech recognition model, often only several millions of parameters compared to several hundred millions for speech recognition. Thus, it can be run continuously on your device without draining your battery! Only when the wake word is detected is the larger speech recognition model launched, and afterwards it is shut down again. We'll cover transformer models for speech recognition in the next Unit, so by the end of the course you should have the tools you need to build your own voice activated assistant!

As with any dataset on the Hugging Face Hub, we can get a feel for the kind of audio data it has present without downloading or committing it memory. After heading to the Speech Commands' dataset card on the Hub, we can use the Dataset Viewer to

scroll through the first 100 samples of the dataset, listening to the audio files and checking any other metadata information:



The Dataset Preview is a brilliant way of experiencing audio datasets before committing to using them. You can pick any dataset on the Hub, scroll through the samples and listen to the audio for the different subsets and splits, gauging whether it's the right dataset for your needs. Once you've selected a dataset, it's trivial to load the data so that you can start using it.

Let's do exactly that and load a sample of the Speech Commands dataset using streaming mode:

```
speech_commands = load_dataset(
    "speech_commands", "v0.02", split="validation", streaming=True
)
sample = next(iter(speech_commands))
```

We'll load an official Audio Spectrogram Transformer checkpoint fine-tuned on the Speech Commands dataset, under the namespace `"MIT/ast-finetuned-speech-commands-v2"`:

```
classifier = pipeline(
    "audio-classification", model="MIT/ast-finetuned-speech-commands-v2"
)
classifier(sample["audio"].copy())
```

Output:

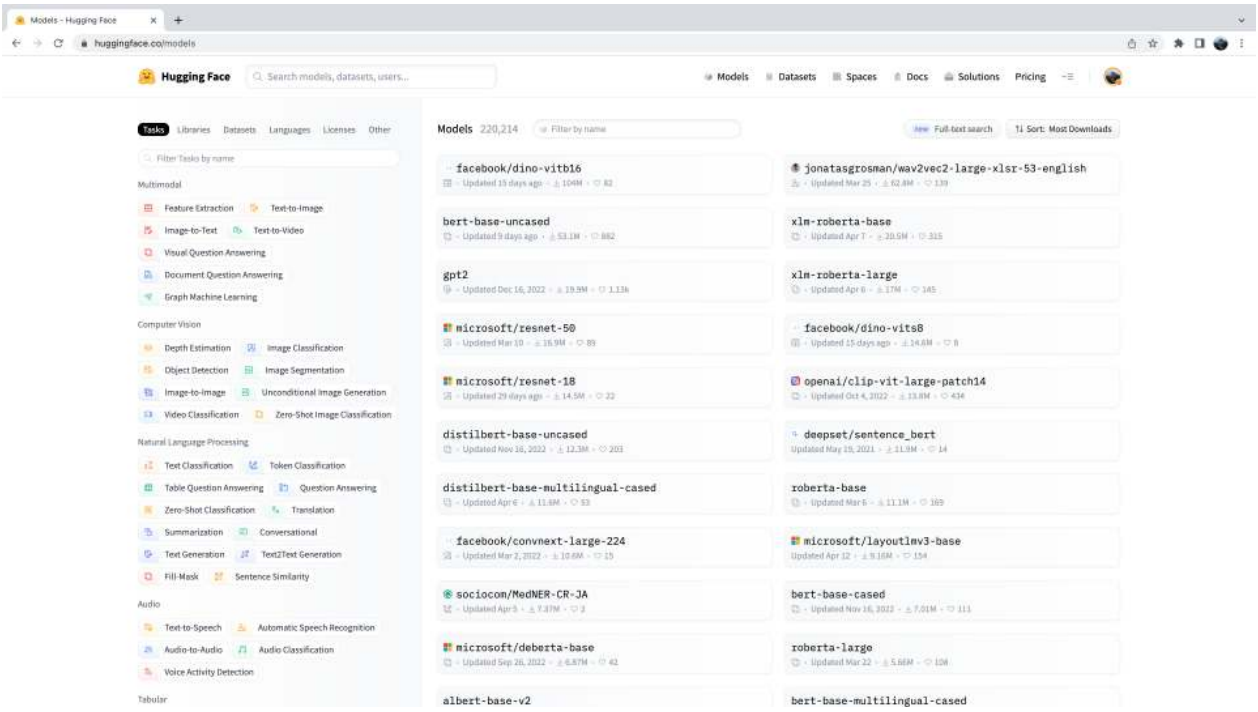
```
[,
 ,
 ,
 ,
 ]
```

Cool! Looks like the example contains the word "backward" with high probability. We can take a listen to the sample and verify this is correct:

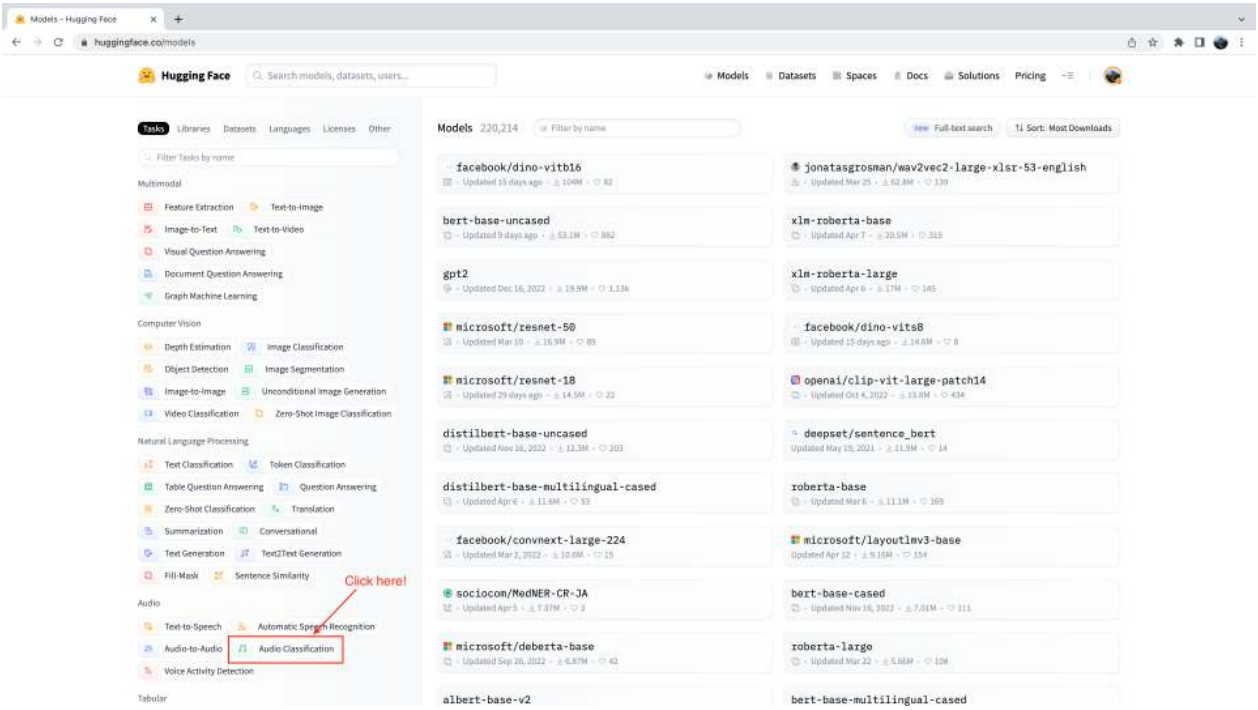
```
from IPython.display import Audio

Audio(sample["audio"]["array"], rate=sample["audio"]["sampling_rate"])
```

Now, you might be wondering how we've selected these pre-trained models to show you in these audio classification examples. The truth is, finding pre-trained models for your dataset and task is very straightforward! The first thing we need to do is head to the Hugging Face Hub and click on the "Models" tab: https://huggingface.co/models

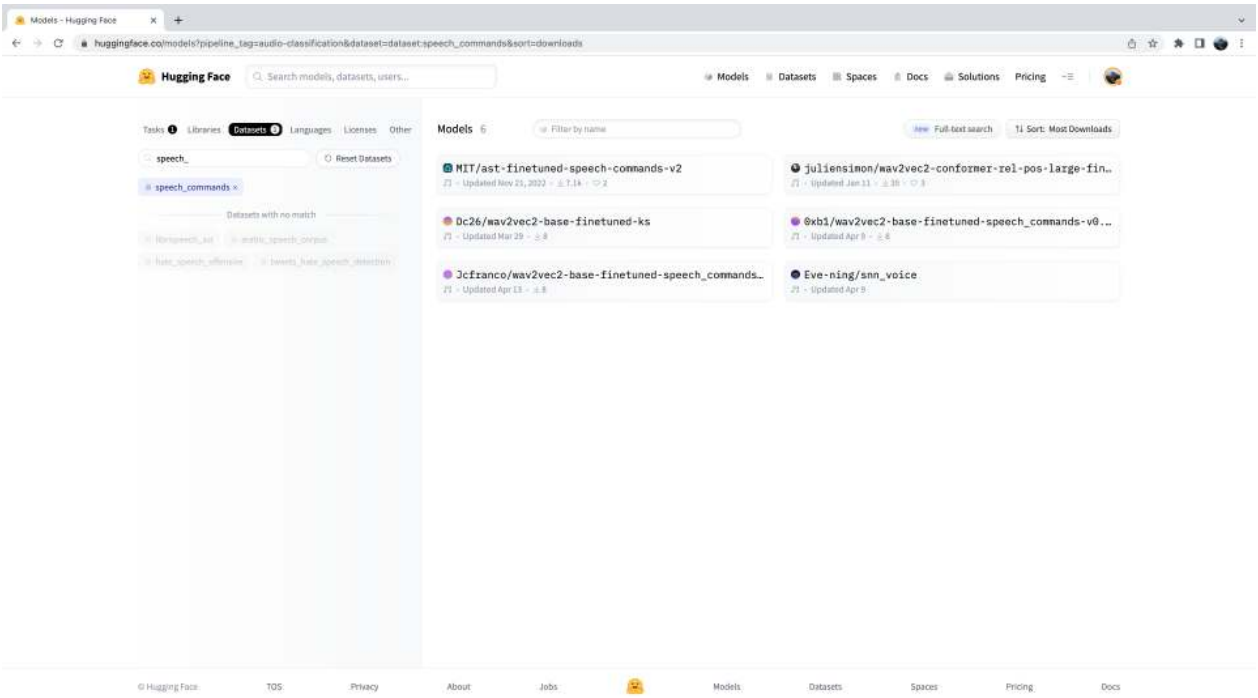
This is going to bring up all the models on the Hugging Face Hub, sorted by downloads in the past 30 days:



You'll notice on the left-hand side that we have a selection of tabs that we can select to filter models by task, library, dataset, etc. Scroll down and select the task "Audio Classification" from the list of audio tasks:



We're now presented with the sub-set of 500+ audio classification models on the Hub. To further refine this selection, we can filter models by dataset. Click on the tab "Datasets", and in the search box type "speech_commands". As you begin typing, you'll see the selection for `speech_commands` appear underneath the search tab. You can click this button to filter all audio classification models to those fine-tuned on the Speech Commands dataset:



Great! We see that we have 6 pre-trained models available to us for this specific dataset and task. You'll recognise the first of these models as the Audio Spectrogram Transformer checkpoint that we used in the previous example. This process of filtering models on the Hub is exactly how we went about selecting the checkpoint to show you!

Language Identification

Language identification (LID) is the task of identifying the language spoken in an audio sample from a list of candidate languages. LID can form an important part in many speech pipelines. For example, given an audio sample in an unknown language, an LID model can be used to categorise the language(s) spoken in the audio sample, and then select an appropriate speech recognition model trained on that language to transcribe the audio.

FLEURS

FLEURS (Few-shot Learning Evaluation of Universal Representations of Speech) is a dataset for evaluating speech recognition systems in 102 languages, including many that are classified as 'low-resource'. Take a look at the FLEURS dataset card on the Hub and explore the different languages that are present: google/fleurs. Can you find your native tongue here? If not, what's the most closely related language?

Let's load up a sample from the validation split of the FLEURS dataset using streaming mode:

```
fleurs = load_dataset("google/fleurs", "all", split="validation", streaming=True)
sample = next(iter(fleurs))
```

Great! Now we can load our audio classification model. For this, we'll use a version of Whisper fine-tuned on the FLEURS dataset, which is currently the most performant LID model on the Hub:

```
classifier = pipeline(
    "audio-classification", model="sanchit-gandhi/whisper-medium-fleurs-lang-id"
)
```

We can then pass the audio through our classifier and generate a prediction:

```
classifier(sample["audio"])
```

Output:

```
[,
 ,
 ,
 ,
 ]
```

We can see that the model predicted the audio was in Afrikaans with extremely high probability (near 1). The FLEURS dataset contains audio data from a wide range of languages - we can see that possible class labels include Northern-Sotho, Icelandic, Danish and Cantonese Chinese amongst others. You can find the full list of languages on the dataset card here: google/fleurs.

Over to you! What other checkpoints can you find for FLEURS LID on the Hub? What transformer models are they using under-the-hood?

Zero-Shot Audio Classification

In the traditional paradigm for audio classification, the model predicts a class label from a *pre-defined* set of possible classes. This poses a barrier to using pre-trained models for audio classification, since the label set of the pre-trained model must match that of the downstream task. For the previous example of LID, the model must predict one of the 102 langauge classes on which it was trained. If the downstream task actually requires 110 languages, the model would not be able to predict 8 of the 110 languages, and so would require re-training to achieve full coverage. This limits the effectiveness of transfer learning for audio classification tasks.

Zero-shot audio classification is a method for taking a pre-trained audio classification model trained on a set of labelled examples and enabling it to be able to classify new examples from previously unseen classes. Let's take a look at how we can achieve this!

Currently, 🤗 Transformers supports one kind of model for zero-shot audio classification: the CLAP model. CLAP is a transformer-based model that takes both audio and text as inputs, and computes the *similarity* between the two. If we pass a text input that strongly correlates with an audio input, we'll get a high similarity score. Conversely, passing a text input that is completely unrelated to the audio input will return a low similarity.

We can use this similarity prediction for zero-shot audio classification by passing one audio input to the model and multiple candidate labels. The model will return a similarity score for each of the candidate labels, and we can pick the one that has the highest score as our prediction.

Let's take an example where we use one audio input from the Environmental Speech Challenge (ESC) dataset:

```
dataset = load_dataset("ashraq/esc50", split="train", streaming=True)
audio_sample = next(iter(dataset))["audio"]["array"]
```

We then define our candidate labels, which form the set of possible classification labels. The model will return a classification probability for each of the labels we define. This means we need to know *a-priori* the set of possible labels in our classification problem, such that the correct label is contained within the set and is thus assigned a valid probability score. Note that we can either pass the full set of labels to the model, or a hand-selected subset that we believe contains the correct label. Passing the full set of labels is going to be more exhaustive, but comes at the expense of lower classification accuracy since the classification space is larger (provided the correct label is our chosen subset of labels):

```
candidate_labels = ["Sound of a dog", "Sound of vacuum cleaner"]
```

We can run both through the model to find the candidate label that is *most similar* to the audio input:

```
classifier = pipeline(
    task="zero-shot-audio-classification", model="laion/clap-htsat-unfused"
)
classifier(audio_sample, candidate_labels=candidate_labels)
```

Output:

```
[, ]
```

Alright! The model seems pretty confident we have the sound of a dog - it predicts it with 99.96% probability, so we'll take that as our prediction. Let's confirm whether we were right by listening to the audio sample (don't turn up your volume too high or else you might get a jump!):

```
Audio(audio_sample, rate=16000)
```

Perfect! We have the sound of a dog barking 🐕, which aligns with the model's prediction. Have a play with different audio samples and different candidate labels - can you define a set of labels that give good generalisation across the ESC dataset? Hint: think about where you could find information on the possible sounds in ESC and construct your labels accordingly!

You might be wondering why we don't use the zero-shot audio classification pipeline for **all** audio classification tasks? It seems as though we can make predictions for any audio classification problem by defining appropriate class labels *a-priori*, thus bypassing the constraint that our classification task needs to match the labels that the model was pre-trained on. This comes down to the nature of the CLAP model used in the zero-shot pipeline: CLAP is pre-trained on *generic* audio classification data, similar to the environmental sounds in the ESC dataset, rather than specifically speech data, like we had in the LID task. If you gave it speech in English and speech in Spanish, CLAP would know that both examples were speech data 🗣️ But it wouldn't be able to differentiate between the languages in the same way a dedicated LID model is able to.

What next?

We've covered a number of different audio classification tasks and presented the most relevant datasets and models that you can download from the Hugging Face Hub and use in just several lines of code using the `pipeline()` class. These tasks included keyword spotting, language identification and zero-shot audio classification.

But what if we want to do something **new**? We've worked extensively on speech processing tasks, but this is only one aspect of audio classification. Another popular field of audio processing involves **music**. While music has inherently different features to speech, many of the same principles that we've learnt about already can be applied to music.

In the following section, we'll go through a step-by-step guide on how you can fine-tune a transformer model with 🤗 Transformers on the task of music classification. By the end of it, you'll have a fine-tuned checkpoint that you can plug into the `pipeline()` class, enabling you to classify songs in exactly the same way that we've classified speech here!

Build a demo with Gradio

In this final section on audio classification, we'll build a Gradio demo to showcase the music classification model that we just trained on the GTZAN dataset. The first thing to do is load up the fine-tuned checkpoint using the `pipeline()` class - this is very familiar now from the section on pre-trained models. You can change the `model_id` to the namespace of your fine-tuned model on the Hugging Face Hub:

```
from transformers import pipeline

model_id = "sanchit-gandhi/distilhubert-finetuned-gtzan"
pipe = pipeline("audio-classification", model=model_id)
```

Secondly, we'll define a function that takes the filepath for an audio input and passes it through the pipeline. Here, the pipeline automatically takes care of loading the audio file, resampling it to the correct sampling rate, and running inference with the model. We take the models predictions of `preds` and format them as a dictionary object to be displayed on the output:

```
def classify_audio(filepath):
    preds = pipe(filepath)
    outputs =
    for p in preds:
        outputs[p["label"]] = p["score"]
    return outputs
```

Finally, we launch the Gradio demo using the function we've just defined:

```
demo = gr.Interface(
    fn=classify_audio, inputs=gr.Audio(type="filepath"), outputs=gr.outputs.Label()
)
demo.launch(debug=True)
```

This will launch a Gradio demo similar to the one running on the Hugging Face Space:

Fine-tuning a model for music classification

In this section, we'll present a step-by-step guide on fine-tuning an encoder-only transformer model for music classification. We'll use a lightweight model for this demonstration and fairly small dataset, meaning the code is runnable end-to-end on any consumer grade GPU, including the T4 16GB GPU provided in the Google Colab free tier. The section includes various tips that you can try should you have a smaller GPU and encounter memory issues along the way.

The Dataset

To train our model, we'll use the GTZAN dataset, which is a popular dataset of 1,000 songs for music genre classification. Each song is a 30-second clip from one of 10 genres of music, spanning disco to metal. We can get the audio files and their corresponding labels from the Hugging Face Hub with the `load_dataset()` function from 🤗 Datasets:

```
from datasets import load_dataset

gtzan = load_dataset("marsyas/gtzan", "all")
gtzan
```

Output:

```
Dataset()
```

One of the recordings in GTZAN is corrupted, so it's been removed from the dataset. That's why we have 999 examples instead of 1,000.

GTZAN doesn't provide a predefined validation set, so we'll have to create one ourselves. The dataset is balanced across genres, so we can use the `train_test_split()` method to quickly create a 90/10 split as follows:

```
gtzan = gtzan["train"].train_test_split(seed=42, shuffle=True, test_size=0.1)
gtzan
```

Output:

```
DatasetDict()
    test: Dataset()
})
```

Great, now that we've got our training and validation sets, let's take a look at one of the audio files:

```
gtzan["train"][0]
```

Output:

```
,
    "genre": 7,
}
```

As we saw in Unit 1, the audio files are represented as 1-dimensional NumPy arrays, where the value of the array represents the amplitude at that timestep. For these songs, the sampling rate is 22,050 Hz, meaning there are 22,050 amplitude values sampled per second. We'll have to keep this in mind when using a pretrained model with a different sampling rate, converting the sampling rates ourselves to ensure they match. We can also see the genre is represented as an integer, or *class label*, which is the format the model will make it's predictions in. Let's use the `int2str()` method of the `genre` feature to map these integers to human-readable names:

```
id2label_fn = gtzan["train"].features["genre"].int2str
id2label_fn(gtzan["train"][0]["genre"])
```

Output:

```
'pop'
```

This label looks correct, since it matches the filename of the audio file. Let's now listen to a few more examples by using Gradio to create a simple interface with the `Blocks` API:

```
def generate_audio():
    example = gtzan["train"].shuffle()[0]
    audio = example["audio"]
    return (
        audio["sampling_rate"],
        audio["array"],
    ), id2label_fn(example["genre"])
```

```
with gr.Blocks() as demo:
    with gr.Column():
        for _ in range(4):
            audio, label = generate_audio()
            output = gr.Audio(audio, label=label)

demo.launch(debug=True)
```

From these samples we can certainly hear the difference between genres, but can a transformer do this too? Let's train a model to find out! First, we'll need to find a suitable pretrained model for this task. Let's see how we can do that.

Picking a pretrained model for audio classification

To get started, let's pick a suitable pretrained model for audio classification. In this domain, pretraining is typically carried out on large amounts of unlabeled audio data, using datasets like LibriSpeech and Voxpopuli. The best way to find these models on the Hugging Face Hub is to use the "Audio Classification" filter, as described in the previous section. Although models like Wav2Vec2 and HuBERT are very popular, we'll use a model called *DistilHuBERT*. This is a much smaller (or *distilled*) version of the HuBERT model, which trains around 73% faster, yet preserves most of the performance.

From audio to machine learning features

Preprocessing the data

Similar to tokenization in NLP, audio and speech models require the input to be encoded in a format that the model can process. In 🤗 Transformers, the conversion from audio to the input format is handled by the *feature extractor* of the model. Similar to tokenizers, 🤗 Transformers provides a convenient `AutoFeatureExtractor` class that can automatically select the correct feature extractor for a given model. To see how we can process our audio files, let's begin by instantiating the feature extractor for DistilHuBERT from the pre-trained checkpoint:

```
from transformers import AutoFeatureExtractor
```

```
model_id = "ntu-spml/distilhubert"
feature_extractor = AutoFeatureExtractor.from_pretrained(
    model_id, do_normalize=True, return_attention_mask=True
)
```

Since the sampling rate of the model and the dataset are different, we'll have to resample the audio file to 16,000 Hz before passing it to the feature extractor. We can do this by first obtaining the model's sample rate from the feature extractor:

```
sampling_rate = feature_extractor.sampling_rate
sampling_rate
```

Output:

```
16000
```

Next, we resample the dataset using the `cast_column()` method and `Audio` feature from 🤗 Datasets:

```
from datasets import Audio

gtzan = gtzan.cast_column("audio", Audio(sampling_rate=sampling_rate))
```

We can now check the first sample of the train-split of our dataset to verify that it is indeed at 16,000 Hz. 🤗 Datasets will resample the audio file *on-the-fly* when we load each audio sample:

```
gtzan["train"][0]
```

Output:

```
,
    "genre": 7,
}
```

Great! We can see that the sampling rate has been downsampled to 16kHz. The array values are also different, as we've now only got approximately one amplitude value for every 1.5 that we had before.

A defining feature of Wav2Vec2 and HuBERT like models is that they accept a float array corresponding to the raw waveform of the speech signal as an input. This is in contrast to other models, like Whisper, where we pre-process the raw audio waveform to spectrogram format.

We mentioned that the audio data is represented as a 1-dimensional array, so it's already in the right format to be read by the model (a set of continuous inputs at discrete time steps). So, what exactly does the feature extractor do?

Well, the audio data is in the right format, but we've imposed no restrictions on the values it can take. For our model to work optimally, we want to keep all the inputs within the same dynamic range. This is going to make sure we get a similar range of activations and gradients for our samples, helping with stability and convergence during training.

To do this, we *normalise* our audio data, by rescaling each sample to zero mean and unit variance, a process called *feature scaling*. It's exactly this feature normalisation that our feature extractor performs!

We can take a look at the feature extractor in operation by applying it to our first audio sample. First, let's compute the mean and variance of our raw audio data:

```
sample = gtzan["train"][0]["audio"]

print(f"Mean: , Variance: ")
```

Output:

```
Mean: 0.000185, Variance: 0.0493
```

We can see that the mean is close to zero already, but the variance is closer to 0.05. If the variance for the sample was larger, it could cause our model problems, since the dynamic range of the audio data would be very small and thus difficult to separate. Let's apply the feature extractor and see what the outputs look like:

```
inputs = feature_extractor(sample["array"], sampling_rate=sample["sampling_rate"])

print(f"inputs keys: ")

print(
    f"Mean: , Variance: "
)
```

Output:

```
inputs keys: ['input_values', 'attention_mask']
Mean: -4.53e-09, Variance: 1.0
```

Alright! Our feature extractor returns a dictionary of two arrays: `input_values` and `attention_mask`. The `input_values` are the preprocessed audio inputs that we'd pass to the HuBERT model. The `attention_mask` is used when we process a *batch* of audio inputs at once - it is used to tell the model where we have padded inputs of different lengths.

We can see that the mean value is now very much closer to zero, and the variance bang-on one! This is exactly the form we want our audio samples in prior to feeding them to the HuBERT model.

Note how we've passed the sampling rate of our audio data to our feature extractor. This is good practice, as the feature extractor performs a check under-the-hood to make sure the sampling rate of our audio data matches the sampling rate expected by the model. If the sampling rate of our audio data did not match the sampling rate of our model, we'd need to up-sample or down-sample the audio data to the correct sampling rate.

Great, so now we know how to process our resampled audio files, the last thing to do is define a function that we can apply to all the examples in the dataset. Since we expect the audio clips to be 30 seconds in length, we'll also truncate any longer clips by using the `max_length` and `truncation` arguments of the feature extractor as follows:

```
max_duration = 30.0


def preprocess_function(examples):
    audio_arrays = [x["array"] for x in examples["audio"]]
    inputs = feature_extractor(
        audio_arrays,
        sampling_rate=feature_extractor.sampling_rate,
        max_length=int(feature_extractor.sampling_rate * max_duration),
        truncation=True,
        return_attention_mask=True,
    )
    return inputs
```

With this function defined, we can now apply it to the dataset using the `map()` method. The `.map()` method supports working with batches of examples, which we'll enable by setting `batched=True`. The default batch size is 1000, but we'll reduce it to 100 to ensure the peak RAM stays within a sensible range for Google Colab's free tier:

```
gtzan_encoded = gtzan.map(
    preprocess_function,
    remove_columns=["audio", "file"],
    batched=True,
    batch_size=100,
    num_proc=1,
)
gtzan_encoded
```

Output:

```
DatasetDict()
    test: Dataset()
})
```

If you exhaust your device's RAM executing the above code, you can adjust the batch parameters to reduce the peak RAM usage. In particular, the following two arguments can be modified: * `batch_size`: defaults to 1000, but set to 100 above. Try reducing by a factor of 2 again to 50 * `writer_batch_size`: defaults to 1000. Try reducing it to 500, and if that doesn't work, then reduce it by a factor of 2 again to 250

To simplify the training, we've removed the `audio` and `file` columns from the dataset. The `input_values` column contains the encoded audio files, the `attention_mask` a binary mask of 0/1 values that indicate where we have padded the audio input, and the `genre` column contains the corresponding labels (or targets). To enable the `Trainer` to process the class labels, we need to rename the `genre` column to `label`:

```
gtzan_encoded = gtzan_encoded.rename_column("genre", "label")
```

Finally, we need to obtain the label mappings from the dataset. This mapping will take us from integer ids (e.g. `7`) to human-readable class labels (e.g. `"pop"`) and back again. In doing so, we can convert our model's integer id prediction into human-readable format, enabling us to use the model in any downstream application. We can do this by using the `int2str()` method as follows:

```
id2label =
label2id =

id2label["7"]
```

```
'pop'
```

OK, we've now got a dataset that's ready for training! Let's take a look at how we can train a model on this dataset.

Fine-tuning the model

To fine-tune the model, we'll use the `Trainer` class from 🤗 Transformers. As we've seen in other chapters, the `Trainer` is a high-level API that is designed to handle the most common training scenarios. In this case, we'll use the `Trainer` to fine-tune the model on GTZAN. To do this, we'll first need to load a model for this task. We can do this by using the `AutoModelForAudioClassification` class, which will automatically add the appropriate classification head to our pretrained DistilHuBERT model. Let's go ahead and instantiate the model:

```
from transformers import AutoModelForAudioClassification

num_labels = len(id2label)

model = AutoModelForAudioClassification.from_pretrained(
    model_id,
    num_labels=num_labels,
    label2id=label2id,
    id2label=id2label,
)
```

We strongly advise you to upload model checkpoints directly the Hugging Face Hub while training. The Hub provides: - Integrated version control: you can be sure that no model checkpoint is lost during training. - Tensorboard logs: track important metrics over the course of training. - Model cards: document what a model does and its intended use cases. - Community: an easy way to share and collaborate with the community! 🤗

Linking the notebook to the Hub is straightforward - it simply requires entering your Hub authentication token when prompted. Find your Hub authentication token here:

```
from huggingface_hub import notebook_login

notebook_login()
```

Output:

```
Login successful
Your token has been saved to /root/.huggingface/token
```

The next step is to define the training arguments, including the batch size, gradient accumulation steps, number of training epochs and learning rate:

```
from transformers import TrainingArguments

model_name = model_id.split("/")[-1]
batch_size = 8
gradient_accumulation_steps = 1
num_train_epochs = 10

training_args = TrainingArguments(
    f"-finetuned-gtzan",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=5e-5,
    per_device_train_batch_size=batch_size,
    gradient_accumulation_steps=gradient_accumulation_steps,
    per_device_eval_batch_size=batch_size,
    num_train_epochs=num_train_epochs,
    warmup_steps=100,
    logging_steps=5,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    fp16=True,
    push_to_hub=True,
)
```

```
Here we have set `push_to_hub=True` to enable automatic upload of our fine-tuned
checkpoints during training. Should you
not wish for your checkpoints to be uploaded to the Hub, you can set this to
`False`.
```

The last thing we need to do is define the metrics. Since the dataset is balanced, we'll use accuracy as our metric and load it using the 🤗 Evaluate library:

```
metric = evaluate.load("accuracy")


def compute_metrics(eval_pred):
    """Computes accuracy on a batch of predictions"""
    predictions = np.argmax(eval_pred.predictions, axis=1)
    return metric.compute(predictions=predictions, references=eval_pred.label_ids)
```

We've now got all the pieces! Let's instantiate the `Trainer` and train the model:

```
from transformers import Trainer

trainer = Trainer(
    model,
    training_args,
    train_dataset=gtzan_encoded["train"],
    eval_dataset=gtzan_encoded["test"],
    tokenizer=feature_extractor,
    compute_metrics=compute_metrics,
)

trainer.train()
```

Depending on your GPU, it is possible that you will encounter a CUDA `"out-of-memory"` error when you start training. In this case, you can reduce the `batch_size` incrementally by factors of 2 and employ `gradient_accumulation_steps` to compensate.

Output:

Training Loss	Epoch	Step	Validation Loss	Accuracy
1.7297	1.0	113	1.8011	0.44
1.24	2.0	226	1.3045	0.64
0.9805	3.0	339	0.9888	0.7
0.6853	4.0	452	0.7508	0.79
0.4502	5.0	565	0.6224	0.81
0.3015	6.0	678	0.5411	0.83
0.2244	7.0	791	0.6293	0.78
0.3108	8.0	904	0.5857	0.81
0.1644	9.0	1017	0.5355	0.83
0.1198	10.0	1130	0.5716	0.82

Training will take approximately 1 hour depending on your GPU or the one allocated to the Google Colab. Our best evaluation accuracy is 83% - not bad for just 10 epochs with 899 examples of training data! We could certainly improve upon this result by training for more epochs, using regularisation techniques such as *dropout*, or sub-diving each audio example from 30s into 15s segments to use a more efficient data pre-processing strategy.

The big question is how this compares to other music classification systems 🤔 For that, we can view the autoevaluate leaderboard, a leaderboard that categorises models by language and dataset, and subsequently ranks them according to their accuracy.

We can automatically submit our checkpoint to the leaderboard when we push the training results to the Hub - we simply have to set the appropriate key-word arguments (kwargs). You can change these values to match your dataset, language and model name accordingly:

```
kwargs = -finetuned-gtzan",
    "finetuned_from": model_id,
    "tasks": "audio-classification",
}
```

The training results can now be uploaded to the Hub. To do so, execute the `.push_to_hub` command:

```
trainer.push_to_hub(**kwargs)
```

This will save the training logs and model weights under `"your-username/distilhubert-finetuned-gtzan"`. For this example, check out the upload at `"sanchit-gandhi/distilhubert-finetuned-gtzan"`.

Share Model

You can now share this model with anyone using the link on the Hub. They can load it with the identifier `"your-username/distilhubert-finetuned-gtzan"` directly into the `pipeline()` class. For instance, to load the fine-tuned checkpoint `"sanchit-gandhi/distilhubert-finetuned-gtzan"`:

```
from transformers import pipeline

pipe = pipeline(
    "audio-classification", model="sanchit-gandhi/distilhubert-finetuned-gtzan"
)
```

Conclusion

In this section, we've covered a step-by-step guide for fine-tuning the DistilHuBERT model for music classification. While we focussed on the task of music classification and the GTZAN dataset, the steps presented here apply more generally to any audio classification task - the same script can be used for spoken language audio classification tasks like keyword spotting or language identification. You just need to swap out the dataset for one that corresponds to your task of interest! If you're

interested in fine-tuning other Hugging Face Hub models for audio classification, we encourage you to check out the other examples in the 🤗 Transformers repository.

In the next section, we'll take the model that you just fine-tuned and build a music classification demo that you can share on the Hugging Face Hub.

Hands-on exercise

It's time to get your hands on some Audio models and apply what you have learned so far. This exercise is one of the four hands-on exercises required to qualify for a course completion certificate.

Here are the instructions. In this unit, we demonstrated how to fine-tune a Hubert model on `marsyas/gtzan` dataset for music classification. Our example achieved 83% accuracy. Your task is to improve upon this accuracy metric.

Feel free to choose any model on the 🤗 Hub that you think is suitable for audio classification, and use the exact same dataset `marsyas/gtzan` to build your own classifier.

Your goal is to achieve 87% accuracy on this dataset with your classifier. You can choose the exact same model, and play with the training hyperparameters, or pick an entirely different model - it's up to you!

For your result to count towards your certificate, don't forget to push your model to Hub as was shown in this unit with the following `**kwargs` at the end of the training:

```
kwargs = -finetuned-gtzan",
    "finetuned_from": model_id,
    "tasks": "audio-classification",
}

trainer.push_to_hub(**kwargs)
```

Here are some additional resources that you may find helpful when working on this exercise: * Audio classification task guide in Transformers documentation * Hubert model documentation * M-CTC-T model documentation * Audio Spectrogram Transformer documentation * Wav2Vec2 documentation

Feel free to build a demo of your model, and share it on Discord! If you have questions, post them in the #audio-study-group channel.

Unit 4. Build a music genre classifier

What you'll learn and what you'll build

Audio classification is one of the most common applications of transformers in audio and speech processing. Like other classification tasks in machine learning, this task involves assigning one or more labels to an audio recording based on its content. For example, in the case of speech, we might want to detect when wake words like "Hey Siri" are spoken, or infer a key word like "temperature" from a spoken query like "What is the weather today?". Environmental sounds provide another example, where we might want to automatically distinguish between sounds such as "car horn", "siren", "dog barking", etc.

In this section, we'll look at how pre-trained audio transformers can be applied to a range of audio classification tasks. We'll then fine-tune a transformer model on the task of music classification, classifying songs into genres like "pop" and "rock". This is an important part of music streaming platforms like Spotify, which recommend songs that are similar to the ones the user is listening to.

By the end of this section, you'll know how to:

- Find suitable pre-trained models for audio classification tasks
- Use the 🤗 Datasets library and the Hugging Face Hub to select audio classification datasets
- Fine-tune a pretrained model to classify songs by genre
- Build a Gradio demo that lets you classify your own songs

Get your certificate of completion

The certification process is completely free. * To get a certificate of completion: you need to pass 3 out of 4 hands-on assignments. * To get a certificate of excellence: you need to pass 4 out of 4 hands-on assignments.

The requirements for each assignment are listed in the respective units: * Unit 4 Hands-on * Unit 5 Hands-on * Unit 6 Hands-on * Unit 7 Hands-on

For the assignments that require to train a model, make sure to push your model that meets the requirements to Hub with relevant `kwargs`. For the demo assignment in Unit 7, make sure that your demo is `public`.

For self-evaluation and to see what units you passed/not passed, you can use the following space:
Check My Progress - Audio Course

Once you qualify for a certificate, go to the Audio Course Certification space. This space implements additional checks to ensure your submissions meet the assessment criteria.

Type your Hugging Face username, your first name, last name in the text fields and click on the "Check if I pass and get the certificate" button.

If you passed 3 out of 4 hands-on assignments, you will receive the certificate of completion. If you passed 4 out of 4 hands-on assignments, you will receive the certificate of excellence.

You can download your certificate in pdf format and png format. Don't hesitate to share your certificate on Twitter (tag me @mariakhalusova and @huggingface) and on LinkedIn.

If you do not meet the certification criteria, don't be discouraged! Go back to the Check My Progress - Audio Course space to see which units you need to do again to get your certificate. If you are experiencing any issue with either of the spaces, let us know!

Congratulations!

You have worked hard to reach this point, and we'd like to congratulate you on completing this Audio course!

Throughout this course, you gained foundational understanding of audio data, explored new concepts, and developed practical skills working with Audio Transformers.

From the basics of working with audio data and pre-trained checkpoints via pipelines, to building real-world audio applications, you have discovered how you can build systems that can not only understand sound but also create it.

As this field is dynamic and ever-evolving, we encourage you to stay curious and continuously explore new models, research advancements, and new applications. When building your own new and exciting audio applications, make sure to always keep ethical implications in mind, and carefully consider the potential impact on individuals and society as a whole.

Thank you for joining this audio course, we hope you thoroughly enjoyed this educational experience, as much as we relished crafting it. Your feedback and contributions are welcome in the course GitHub repo.

To learn how you can obtain your well-deserved certificate of completion, if you successfully passed the hands-on assignments, check out the next page.

Finally, to stay connected with the Audio Course Team, you can follow us on Twitter:

- Maria Khalusova: @mariakhalusova
- Sanchit Gandhi: @sanchitgandhi99
- Matthijs Hollemans: @mhollemans
- Vaibhav (VB) Srivastav: @reach_vb

Stay curious and train Transformers! :)

Audio classification architectures

The goal of audio classification is to predict a class label for an audio input. The model can predict a single class label that covers the entire input sequence, or it can predict a label for every audio frame — typically every 20 milliseconds of input audio — in which case the model's output is a sequence of class label probabilities. An example of the former is detecting what bird is making a particular sound; an example of the latter is speaker diarization, where the model predicts which speaker is speaking at any given moment.

Classification using spectrograms

One of the easiest ways to perform audio classification is to pretend it's an image classification problem!

Recall that a spectrogram is a two-dimensional tensor of shape `(frequencies, sequence length)`. In the chapter on audio data we plotted these spectrograms as images. Guess what? We can literally treat the spectrogram as an image and pass it into a regular CNN classifier model such as ResNet and get very good predictions. Even better, we can use a image transformer model such as ViT.

This is what **Audio Spectrogram Transformer** does. It uses the ViT or Vision Transformer model, and passes it spectrograms as input instead of regular images.

Thanks to the transformer's self-attention layers, the model is better able to capture global context than a CNN is.

Just like ViT, the AST model splits the audio spectrogram into a sequence of partially overlapping image patches of 16×16 pixels. This sequence of patches is then projected into a sequence of embeddings, and these are given to the transformer encoder as input as usual. AST is an encoder-only transformer model and so the output is a sequence of hidden-states, one for each 16×16 input patch. On top of this is a simple classification layer with sigmoid activation to map the hidden-states to classification probabilities.

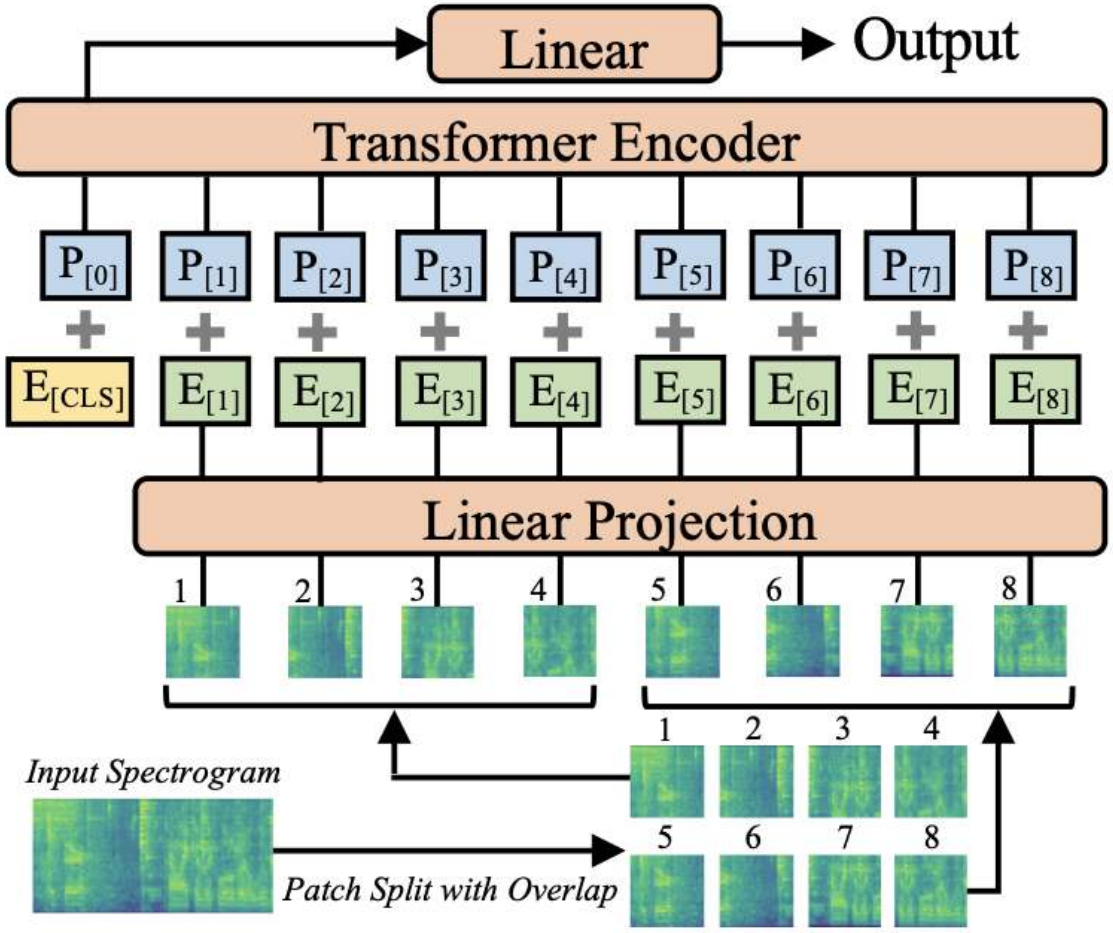


Image from the paper AST: Audio Spectrogram Transformer

💡 Even though here we pretend spectrograms are the same as images, there are important differences. For example, shifting the contents of an image up or down generally does not change the meaning of what is in the image. However, shifting a spectrogram up or down will change the frequencies that are in the sound and completely change its character. Images are invariant under translation but

spectrograms are not. Treating spectrograms as images can work very well in practice, but keep in mind they are not really the same thing.

Any transformer can be a classifier

In a previous section you've seen that CTC is an efficient technique for performing automatic speech recognition using an encoder-only transformer. Such CTC models already are classifiers, predicting probabilities for class labels from a tokenizer vocabulary. We can take a CTC model and turn it into a general-purpose audio classifier by changing the labels and training it with a regular cross-entropy loss function instead of the special CTC loss.

For example, HF Transformers has a `Wav2Vec2ForCTC` model but also `Wav2Vec2ForSequenceClassification` and `Wav2Vec2ForAudioFrameClassification`. The only differences between the architectures of these models is the size of the classification layer and the loss function used.

In fact, any encoder-only audio transformer model can be turned into an audio classifier by adding a classification layer on top of the sequence of hidden states. (Classifiers usually don't need a transformer decoder.)

To predict a single classification score for the entire sequence (`Wav2Vec2ForSequenceClassification`), the model takes the mean over the hidden-states and feeds that into the classification layer. The output is a single probability distribution.

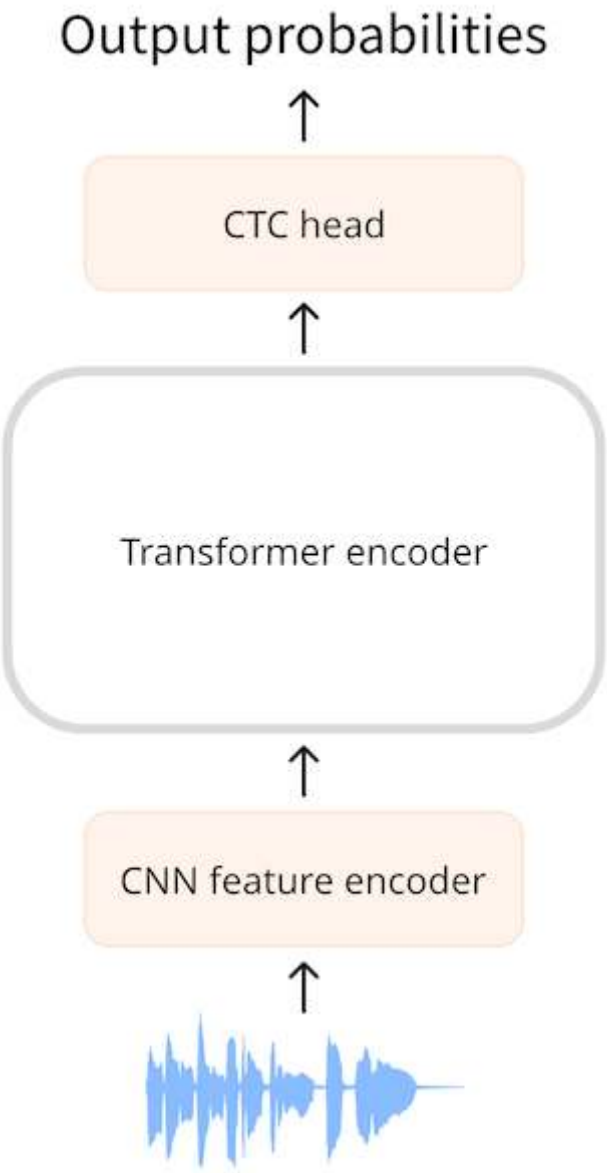
To make a separate classification for each audio frame (`Wav2Vec2ForAudioFrameClassification`), the classifier is run on the sequence of hidden-states, and so the output of the classifier is a sequence too.

CTC architectures

CTC or Connectionist Temporal Classification is a technique that is used with encoder-only transformer models for automatic speech recognition. Examples of such models are **Wav2Vec2**, **HuBERT** and **M-CTC-T**.

An encoder-only transformer is the simplest kind of transformer because it just uses the encoder portion of the model. The encoder reads the input sequence (the audio waveform) and maps this into a sequence of hidden-states, also known as the output embeddings.

With a CTC model, we apply an additional linear mapping on the sequence of hidden-states to get class label predictions. The class labels are the **characters of the alphabet** (a, b, c, ...). This way we're able to predict any word in the target language with a small classification head, as the vocabulary just needs to exist of 26 characters plus a few special tokens.



So far, this is very similar to what we do in NLP with a model such as BERT: an encoder-only transformer model maps our text tokens into a sequence of encoder hidden-states, and then we apply a linear mapping to get one class label prediction for each hidden-state.

Here's the rub: In speech, we don't know the **alignment** of the audio inputs and text outputs. We know that the order the speech is spoken in is the same as the order

that the text is transcribed in (the alignment is so-called monotonic), but we don't know how the characters in the transcription line up to the audio. This is where the CTC algorithm comes in.

💡 In NLP models the vocabulary is usually made up of thousands of tokens that describe not just individual characters but parts of words or even complete words. For CTC, however, a small vocabulary works best and we generally try to keep it to less than 50 characters. We don't care about the casing of the letters, so only using uppercase (or only lowercase) is sufficient. Numbers are spelled out, e.g. `"20"` becomes `"twenty"`. In addition to the letters, we need at least a word separator token (space) and a padding token. Just as with an NLP model, the padding token allows us to combine multiple examples in a batch, but it's also the token the model will predict for silences. In English, it's also useful to keep the `'` character — after all, `"it's"` and `"its"` have very different meanings.

Dude, where's my alignment?

Automatic speech recognition or ASR involves taking audio as input and producing text as output. We have a few choices for how to predict the text:

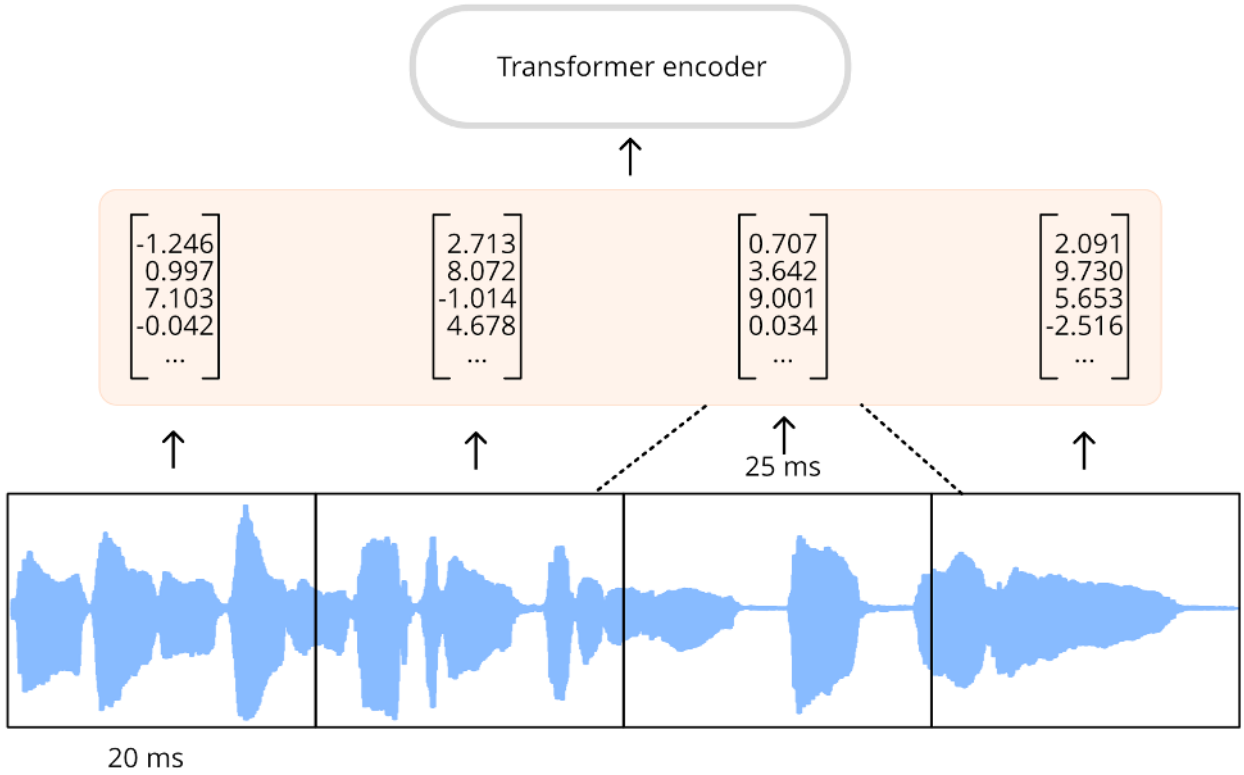
- as individual characters
- as phonemes
- as word tokens

An ASR model is trained on a dataset consisting of `(audio, text)` pairs where the text is a human-made transcription of the audio file. Generally the dataset does not include any timing information that says which word or syllable occurs where in the audio file. Since we can't rely on timing information during training, we don't have any idea how the input and output sequences should be aligned.

Let's suppose our input is a one-second audio file. In **Wav2Vec2**, the model first downsamples the audio input using the CNN feature encoder to a shorter sequence of hidden-states, where there is one hidden-state vector for every 20 milliseconds of audio. For one second of audio, we then forward a sequence of 50 hidden-states to the transformer encoder. (The audio segments extracted from the input sequence partially overlap, so even though one hidden-state vector is emitted every 20 ms, each hidden-state actually represent 25 ms of audio.)

The transformer encoder predicts one feature representation for each of these hidden-states, meaning we receive a sequence of 50 outputs from the transformer. Each of these outputs has a dimensionality of 768. The output sequence of the

transformer encoder in this example therefore has shape `(768, 50)`. As each of these predictions covers 25 ms of time, which is shorter than the duration of a phoneme, it makes sense to predict individual phonemes or characters but not entire words. CTC works best with a small vocabulary, so we'll predict characters.



To make text predictions, we map each of the 768-dimensional encoder outputs to our character labels using a linear layer (the "CTC head"). The model then predicts a `(50, 32)` tensor containing the logits, where 32 is the number of tokens in the vocabulary. Since we make one prediction for each of the features in the sequence, we end up with a total of 50 character predictions for each second of audio.

However, if we simply predict one character every 20 ms, our output sequence might look something like this:

```
BRIIONSAWWSOMEETHINGCLOSETOPANICONHHISOPPONENT'SSFAACEWHENTHEMANNFINALLLYRREECOGGNN
IIZEDHHISSERRRRORR ...
```

If you look closely, it somewhat resembles English but a lot of the characters have been duplicated. That's because the model needs to output *something* for every 20 ms of audio in the input sequence, and if a character is spread out over a period longer than 20 ms then it will appear multiple times in the output. There's no way to avoid this, especially since we don't know what the timing of the transcript is during training. CTC is a way to filter out these duplicates.

(In reality, the predicted sequence also contains a lot of padding tokens for when the model isn't quite sure what the sound represents, or for the empty space between characters. We removed these padding tokens from the example for clarity. The partial overlap between audio segments is another reason characters get duplicated in the output.)

The CTC algorithm

The key to the CTC algorithm is using a special token, often called the **blank token**. This is just another token that the model will predict and it's part of the vocabulary. In this example, the blank token is shown as `_`. This special token serves as a hard boundary between groups of characters.

The full output from the CTC model might be something like the following:

```
B_R_II_O_N_||_S_AWW_|||||_S_OMEE_TH_ING_||_C_L_O_S_E||TO|_P_A_N_I_C_||_ON||HHI_S||
_OP_P_O_N_EN_T_'SS||_F_AA_C_E||_W_H_EN||THE||M_A_NN_||||_F_I_N_AL_LL_Y||||
_RREE_C_O_GG_NN_II_Z_ED|||HHISS|||_ER_RRR_ORR||||
```

The `|` token is the word separator character. In the example we use `|` instead of a space making it easier to spot where the word breaks are, but it serves the same purpose.

The CTC blank character makes it possible to filter out the duplicate characters. For example let's look at the last word from the predicted sequence, `_ER_RRR_ORR`. Without the CTC blank token, the word looked like this:

```
ERRRRORR
```

If we were to simply remove duplicate characters, this would become `EROR`. That's clearly not the correct spelling. But with the CTC blank token we can remove the duplicates in each group, so that:

```
_ER_RRR_ORR
```

becomes:

```
_ER_R_OR
```

and now we remove the `_` blank token to get the final word:

```
ERROR
```

If we apply this logic to the entire text, including `|`, and replace the surviving `|` characters by spaces, the final CTC-decoded output is:

```
BRION SAW SOMETHING CLOSE TO PANIC ON HIS OPPONENT'S FACE WHEN THE MAN FINALLY
RECOGNIZED HIS ERROR
```

To recap, the model predicts one token (character) for every 20 ms of (partially overlapping) audio from the input waveform. This gives a lot of duplicates. Thanks to the CTC blank token, we can easily remove these duplicates without destroying the proper spelling of the words. This is a very simple and convenient way to solve the problem of aligning the output text with the input audio.

💡 In the actual Wav2Vec2 model, the CTC blank token is the same as the padding token `<pad>`. The model will predict many of these `<pad>` tokens, for example when there isn't a clear character to predict for the current 20 ms of audio. Using the same token for padding as for CTC blanking simplifies the decoding algorithm and it helps keep the vocab small.

Adding CTC to a transformer encoder model is easy: the output sequence from the encoder goes into a linear layer that projects the acoustic features to the vocabulary. The model is trained with a special CTC loss.

One downside of CTC is that it may output words that *sound* correct, but are not *spelled* correctly. After all, the CTC head only considers individual characters, not complete words. One way to improve the quality of the audio transcriptions is to use an external language model. This language model essentially acts as a spellchecker on top of the CTC output.

What's the difference between Wav2Vec2, HuBERT, M-CTC-T, ...?

All transformer-based CTC models have a very similar architecture: they use the transformer encoder (but not the decoder) with a CTC head on top. Architecture-wise they are more alike than different.

One difference between Wav2Vec2 and M-CTC-T is that the former works on raw audio waveforms while the latter uses mel spectrograms as input. The models also have been trained for different purposes. M-CTC-T, for example, is trained for

multilingual speech recognition, and therefore has a relatively large CTC head that includes Chinese characters in addition to other alphabets.

Wav2Vec2 & HuBERT use the exact same architecture but are trained in very different ways. Wav2Vec2 is pre-trained like BERT's masked language modeling, by predicting speech units for masked parts of the audio. HuBERT takes the BERT inspiration a step further and learns to predict "discrete speech units", which are analogous to tokens in a text sentence, so that speech can be treated using established NLP techniques.

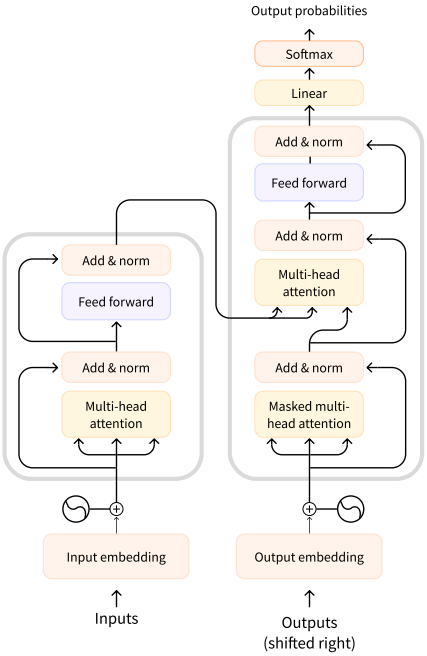
To clarify, the models highlighted here aren't the only transformer-based CTC models. There are many others, but now you know they all work in a similar way.

Unit 3. Transformer architectures for audio

In this course we will primarily consider transformer models and how they can be applied to audio tasks. While you don't need to know the inner details of these models, it's useful to understand the main concepts that make them work, so here's a quick refresher. For a deep dive into transformers, check out our LLM Course.

How does a transformer work?

The original transformer model was designed to translate written text from one language into another. Its architecture looked like this:



On the left is the **encoder** and on the right is the **decoder**.

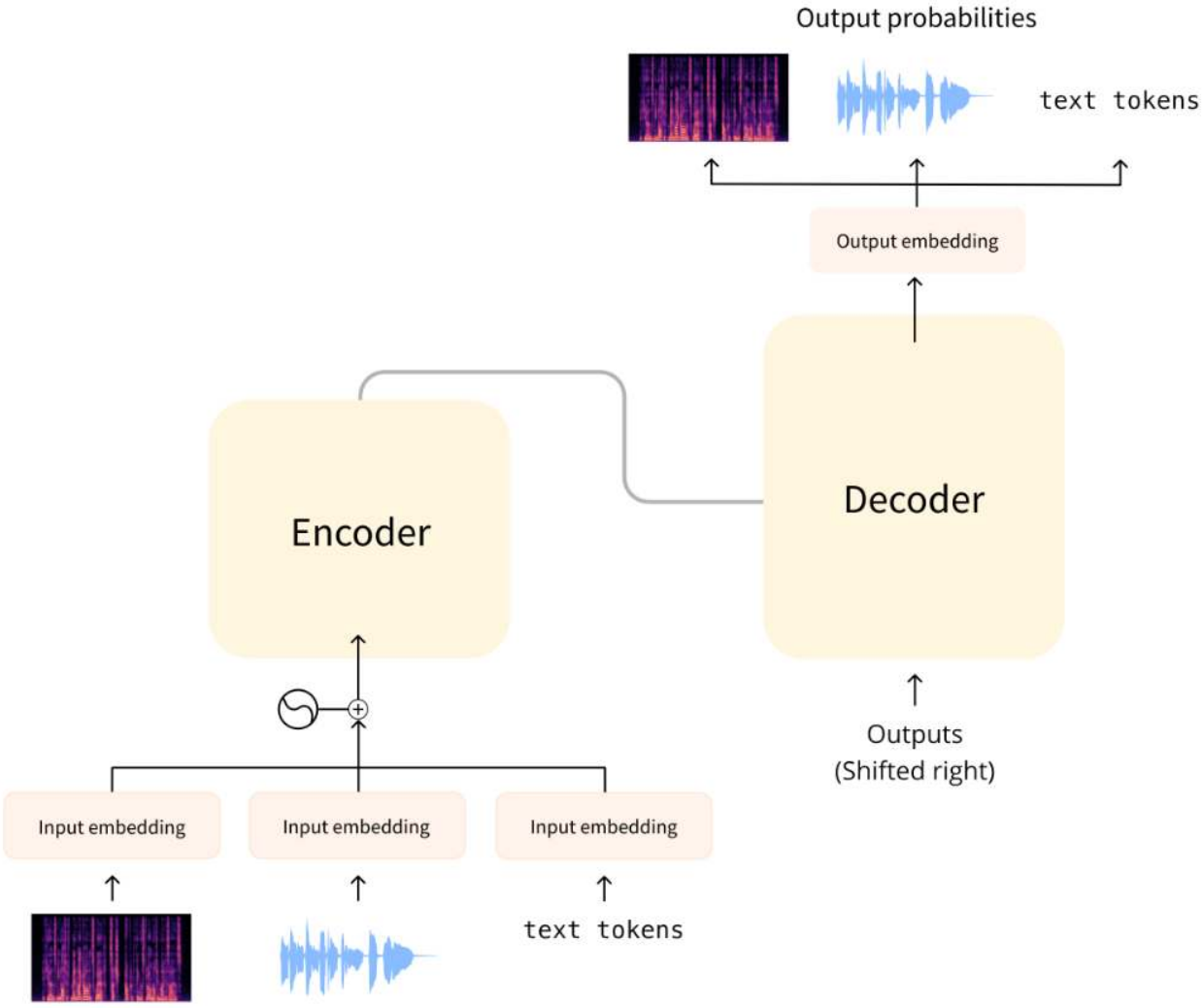
- The encoder receives an input, in this case a sequence of text tokens, and builds a representation of it (its features). This part of the model is trained to acquire understanding from the input.
- The decoder uses the encoder's representation (the features) along with other inputs (the previously predicted tokens) to generate a target sequence. This part of the model is trained to generate outputs. In the original design, the output sequence consisted of text tokens.

There are also transformer-based models that only use the encoder part (good for tasks that require understanding of the input, such as classification), or only the decoder part (good for tasks such as text generation). An example of an encoder-only model is BERT; an example of a decoder-only model is GPT2.

A key feature of transformer models is that they are built with special layers called **attention layers**. These layers tell the model to pay specific attention to certain elements in the input sequence and ignore others when computing the feature representations.

Using transformers for audio

The audio models we'll cover in this course typically have a standard transformer architecture as shown above, but with a slight modification on the input or output side to allow for audio data instead of text. Since all these models are transformers at heart, they will have most of their architecture in common and the main differences are in how they are trained and used.



For audio tasks, the input and/or output sequences may be audio instead of text:

- Automatic speech recognition (ASR): The input is speech, the output is text.
- Speech synthesis (TTS): The input is text, the output is speech.
- Audio classification: The input is audio, the output is a class probability — one for each element in the sequence or a single class probability for the entire sequence.
- Voice conversion or speech enhancement: Both the input and output are audio.

There are a few different ways to handle audio so it can be used with a transformer. The main consideration is whether to use the audio in its raw form — as a waveform — or to process it as a spectrogram instead.

Model inputs

The input to an audio model can be either text or sound. The goal is to convert this input into an embedding vector that can be processed by the transformer architecture.

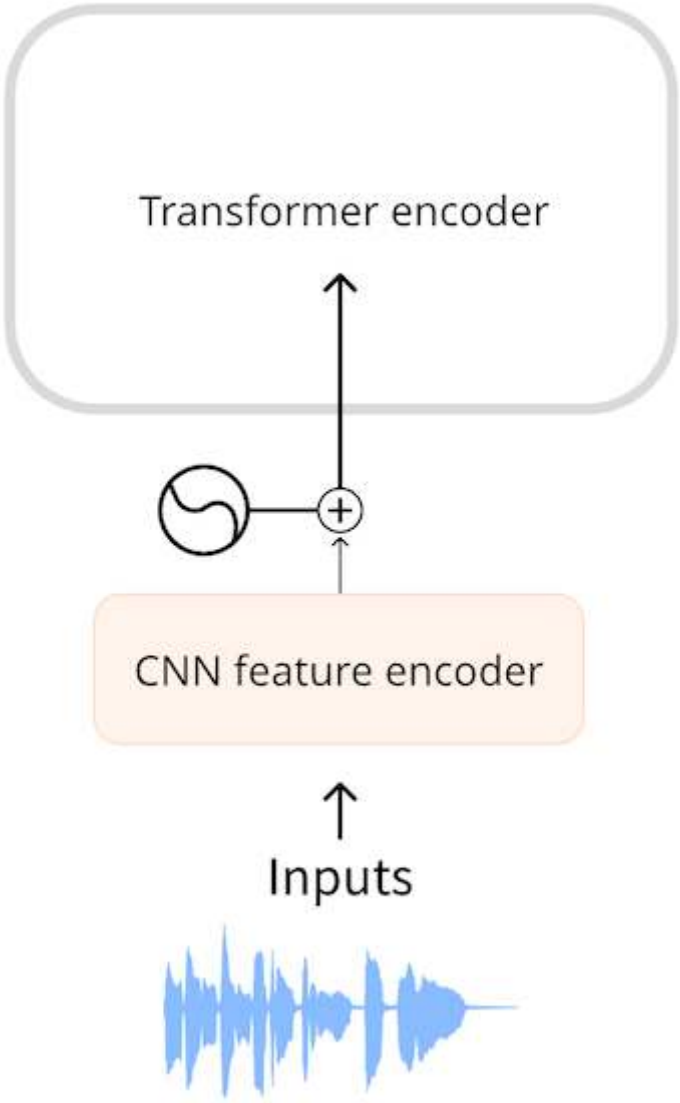
Text input

A text-to-speech model takes text as input. This works just like the original transformer or any other NLP model: The input text is first tokenized, giving a sequence of text tokens. This sequence is sent through an input embedding layer to convert the tokens into 512-dimensional vectors. Those embedding vectors are then passed into the transformer encoder.

Waveform input

An automatic speech recognition model takes audio as input. To be able to use a transformer for ASR, we first need to convert the audio into a sequence of embedding vectors somehow.

Models such as **Wav2Vec2** and **HuBERT** use the audio waveform directly as the input to the model. As you've seen in the chapter on audio data, a waveform is a one-dimensional sequence of floating-point numbers, where each number represents the sampled amplitude at a given time. This raw waveform is first normalized to zero mean and unit variance, which helps to standardize audio samples across different volumes (amplitudes).



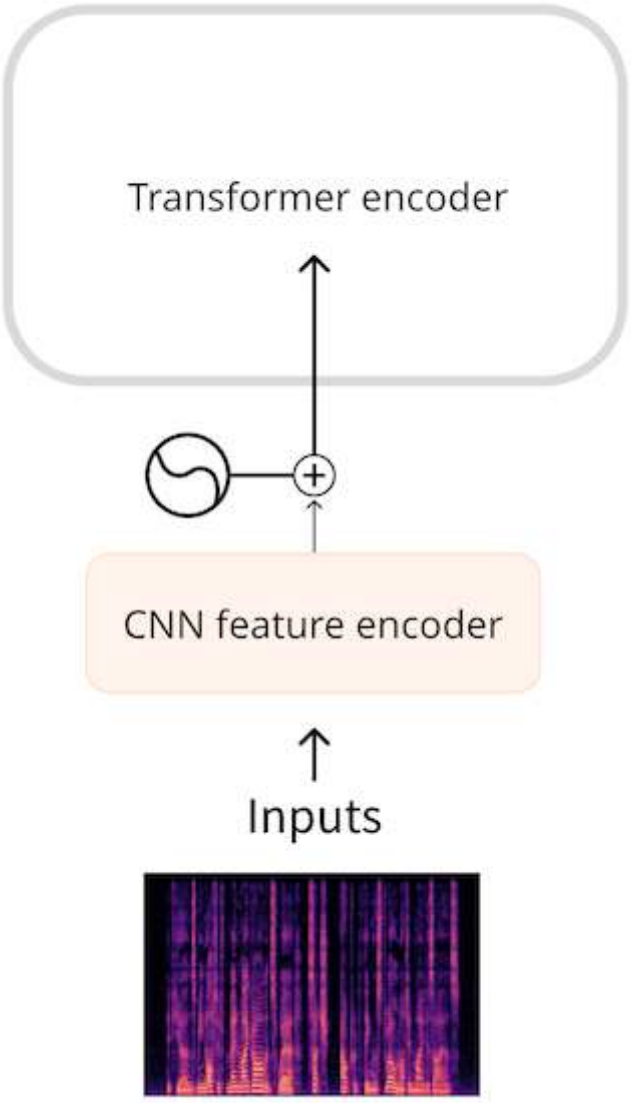
After normalizing, the sequence of audio samples is turned into an embedding using a small convolutional neural network, known as the feature encoder. Each of the convolutional layers in this network processes the input sequence, subsampling the audio to reduce the sequence length, until the final convolutional layer outputs a 512-dimensional vector with the embedding for each 25 ms of audio. Once the input sequence has been transformed into a sequence of such embeddings, the transformer will process the data as usual.

Spectrogram input

One downside of using the raw waveform as input is that they tend to have long sequence lengths. For example, thirty seconds of audio at a sampling rate of 16 kHz

gives an input of length `30 * 16000 = 480000`. Longer sequence lengths require more computations in the transformer model, and so higher memory usage.

Because of this, raw audio waveforms are not usually the most efficient form of representing an audio input. By using a spectrogram, we get the same amount of information but in a more compressed form.



Models such as **Whisper** first convert the waveform into a log-mel spectrogram. Whisper always splits the audio into 30-second segments, and the log-mel spectrogram for each segment has shape `(80, 3000)` where 80 is the number of mel bins and 3000 is the sequence length. By converting to a log-mel spectrogram we've reduced the amount of input data, but more importantly, this is a much shorter sequence than the raw waveform. The log-mel spectrogram is then processed by a small CNN into a sequence of embeddings, which goes into the transformer as usual.

In both cases, waveform as well as spectrogram input, there is a small network in front of the transformer that converts the input into embeddings and then the transformer takes over to do its thing.

Model outputs

The transformer architecture outputs a sequence of hidden-state vectors, also known as the output embeddings. Our goal is to transform these vectors into a text or audio output.

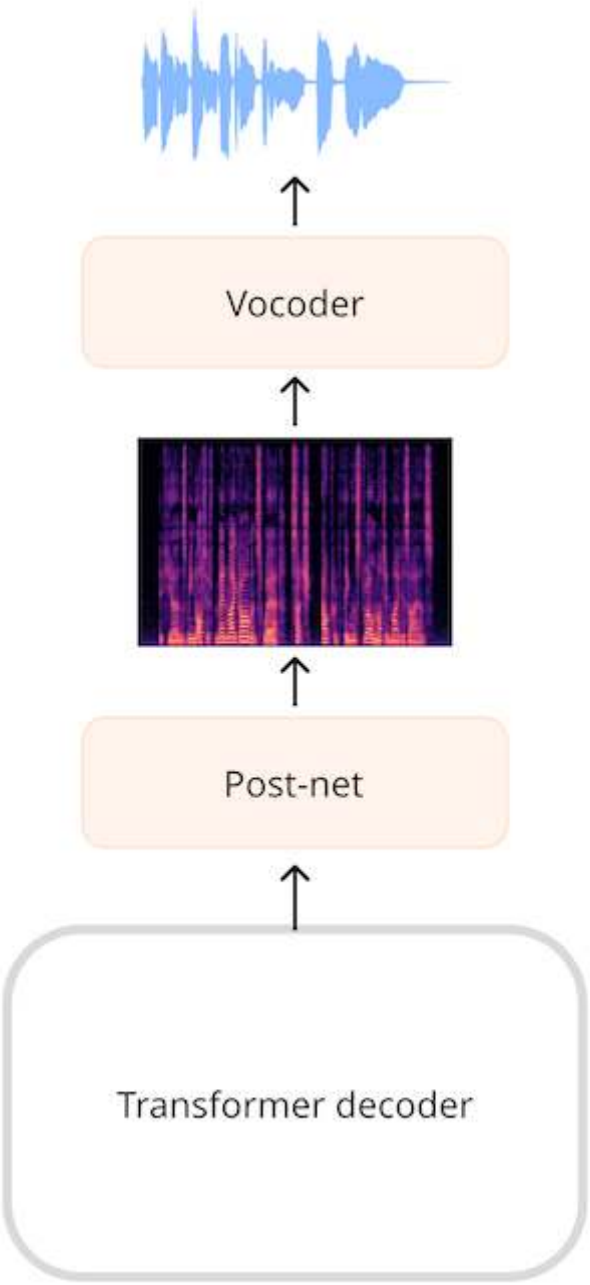
Text output

The goal of an automatic speech recognition model is to predict a sequence of text tokens. This is done by adding a language modeling head — typically a single linear layer — followed by a softmax on top of the transformer's output. This predicts the probabilities over the text tokens in the vocabulary.

Spectrogram output

For models that generate audio, such as a text-to-speech (TTS) model, we'll have to add layers that can produce an audio sequence. It's very common to generate a spectrogram and then use an additional neural network, known as a vocoder, to turn this spectrogram into a waveform.

In the **SpeechT5** TTS model, for example, the output from the transformer network is a sequence of 768-element vectors. A linear layer projects that sequence into a log-mel spectrogram. A so-called post-net, made up of additional linear and convolutional layers, refines the spectrogram by reducing noise. The vocoder then makes the final audio waveform.



💡 If you take an existing waveform and apply the Short-Time Fourier Transform or STFT, it is possible to perform the inverse operation, the ISTFT, to get the original waveform again. This works because the spectrogram created by the STFT contains both amplitude and phase information, and both are needed to reconstruct the waveform. However, audio models that generate their output as a spectrogram typically only predict the amplitude information, not the phase. To turn such a spectrogram into a waveform, we have to somehow estimate the phase information. That's what a vocoder does.

Waveform output

It's also possible for models to directly output a waveform instead of a spectrogram as an intermediate step, but we currently don't have any models in 🤗 Transformers that do this.

Conclusion

In summary: Most audio transformer models are more alike than different — they're all built on the same transformer architecture and attention layers, although some models will only use the encoder portion of the transformer while others use both the encoder and decoder.

You've also seen how to get audio data into and out of transformer models. To perform the different audio tasks of ASR, TTS, and so on, we can simply swap out the layers that pre-process the inputs into embeddings, and swap out the layers that post-process the predicted embeddings into outputs, while the transformer backbone stays the same.

Next, let's look at a few different ways these models can be trained to do automatic speech recognition.

Check your understanding of the course material

1. What is a vocoder?

<Question choices=, ,

```
]}
```

/>

2. Wav2Vec2 is an example of

<Question choices=, ,

```
]}
```

/>

3. What does a blank token in CTC algorithm do?

<Question choices=, ,

```
]}
```

/>

4. Which of the following statements about CTC models is FALSE?

<Question choices=, ,

```
]}
```

/>

5. Whisper is an example of

<Question choices=, ,

```
]}
```

/>

6. What is the easiest way to perform audio classification?

<Question choices=, ,

```
]}
```

/>

7. True or false? When treating spectrograms as images for classification, you will always benefit from image data augmentation techniques, such as shifting an image, cropping it, or resizing.

<Question choices=,

```
]}
```

/>

Seq2Seq architectures

The CTC models discussed in the previous section used only the encoder part of the transformer architecture. When we also add the decoder to create an encoder-decoder model, this is referred to as a **sequence-to-sequence** model or seq2seq for short. The model maps a sequence of one kind of data to a sequence of another kind of data.

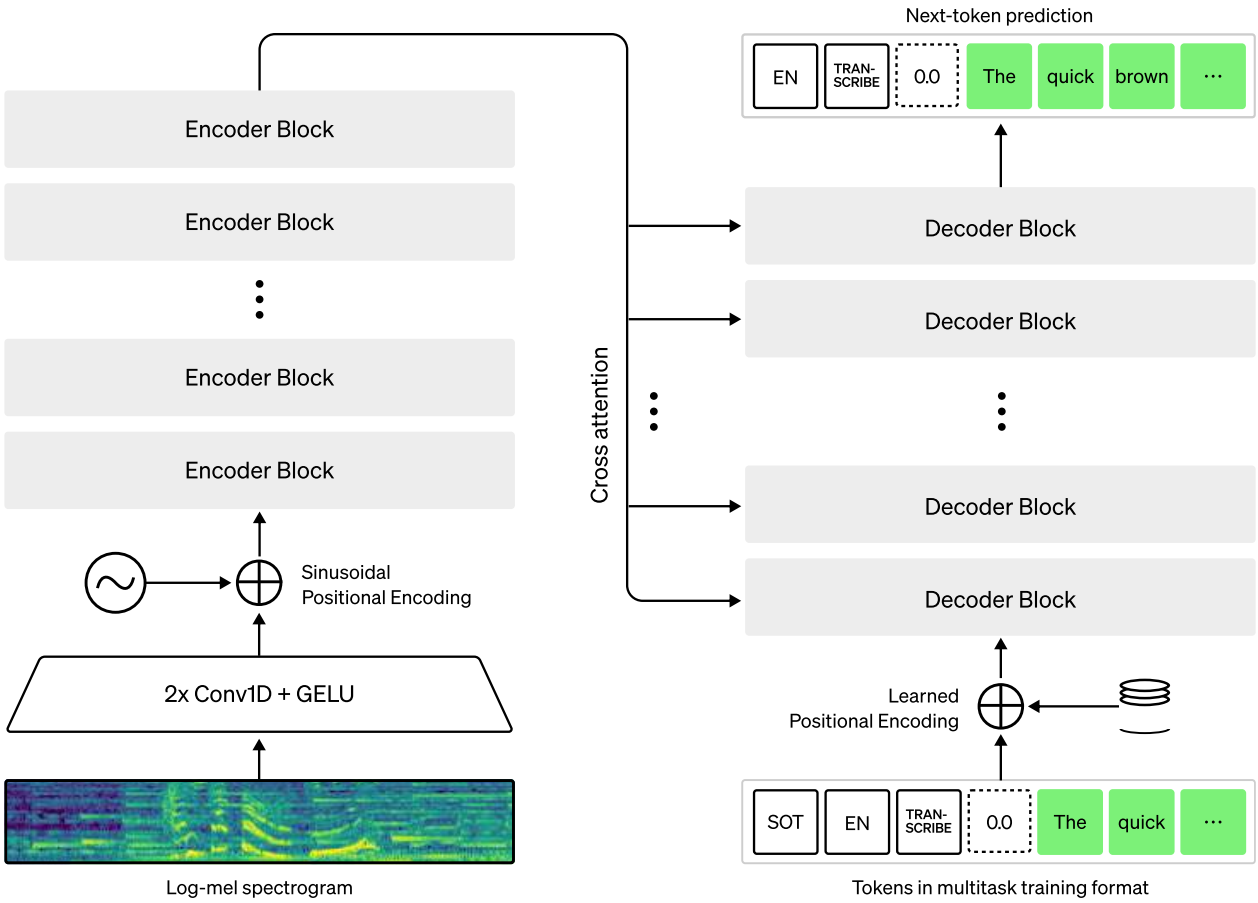
With encoder-only transformer models, the encoder made a prediction for each element in the input sequence. Therefore, both input and output sequences will always have the same length. In the case of CTC models such as Wav2Vec2 the input waveform was first downsampled, but there still was one prediction for every 20 ms of audio.

With a seq2seq model, there is no such one-to-one correspondence and the input and output sequences can have different lengths. That makes seq2seq models suitable for NLP tasks such as text summarization or translation between different languages — but also for audio tasks such as speech recognition.

The architecture of a decoder is very similar to that of an encoder, and both use similar layers with self-attention as the main feature. However, the decoder performs a different task than the encoder. To see how this works, let's examine how a seq2seq model can do automatic speech recognition.

Automatic speech recognition

The architecture of **Whisper** is as follows (figure courtesy of OpenAI Whisper blog):



This should look quite familiar. On the left is the **transformer encoder**. This takes as input a log-mel spectrogram and encodes that spectrogram to form a sequence of encoder hidden states that extract important features from the spoken speech. This hidden-states tensor represents the input sequence as a whole and effectively encodes the "meaning" of the input speech.

💡 It's common for these seq2seq models to use spectrograms as input. However, a seq2seq model can also be designed to work directly on audio waveforms.

The output of the encoder is then passed into the **transformer decoder**, shown on the right, using a mechanism called **cross-attention**. This is like self-attention but attends over the encoder output. From this point on, the encoder is no longer needed.

The decoder predicts a sequence of text tokens in an **autoregressive** manner, a single token at a time, starting from an initial sequence that just has a "start" token in it (`SOT` in the case of Whisper). At each following timestep, the previous output sequence is fed back into the decoder as the new input sequence. In this manner,

the decoder emits one new token at a time, steadily growing the output sequence, until it predicts an "end" token or a maximum number of timesteps is reached.

While the architecture of the decoder is mostly identical to that of the encoder, there are two big differences:

1. the decoder has a cross-attention mechanism that allows it to look at the encoder's representation of the input sequence
2. the decoder's attention is causal — the decoder isn't allowed to look into the future.

In this design, the decoder plays the role of a **language model**, processing the hidden-state representations from the encoder and generating the corresponding text transcriptions. This is a more powerful approach than CTC, even if the CTC model is combined with an external language model, as the seq2seq system can be trained end-to-end with the same training data and loss function, giving greater flexibility and generally superior performance.

💡 Whereas a CTC model outputs a sequence of individual characters, the tokens predicted by Whisper are full words or portions of words. It uses the tokenizer from GPT-2 and has 50k+ unique tokens. A seq2seq model can therefore output a much shorter sequence than a CTC model for the same transcription.

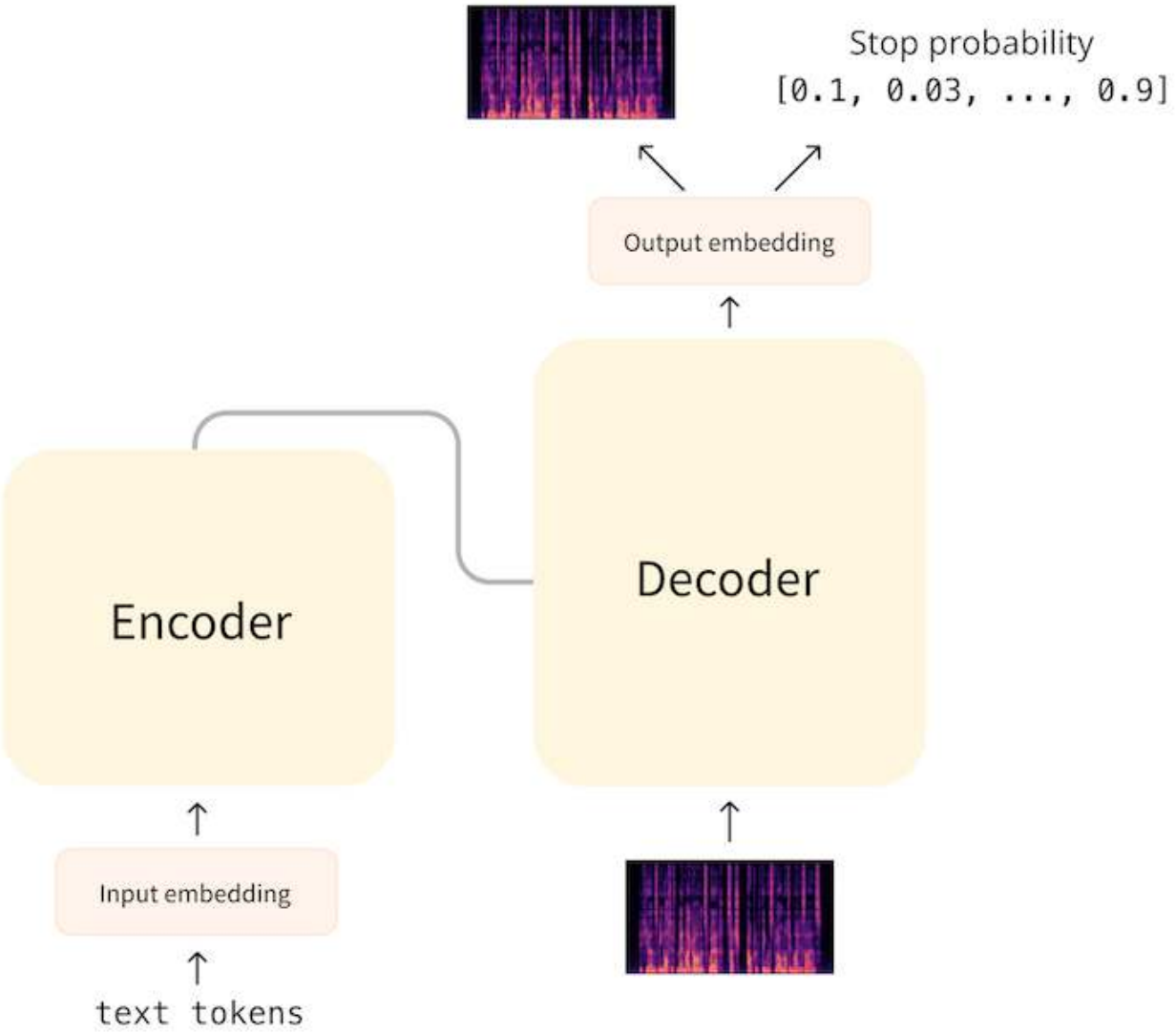
A typical loss function for a seq2seq ASR model is the cross-entropy loss, as the final layer of the model predicts a probability distribution over the possible tokens. This is usually combined with techniques such as beam search to generate the final sequence. The metric for speech recognition is WER or word error rate, which measures how many substitutions, insertions, and deletions are necessary to turn the predicted text into the target text — the fewer, the better the score.

Text-to-speech

It may not surprise you: A seq2seq model for TTS works essentially the same as described above but with the inputs and outputs switched around! The transformer encoder takes in a sequence of text tokens and extracts a sequence of hidden-states that represent the input text. The transformer decoder applies cross-attention to the encoder output and predicts a spectrogram.

💡 Recall that a spectrogram is made by taking the frequency spectrum of successive time slices of an audio waveform and stacking them together. In other words, a spectrogram is a sequence where the elements are (log-mel) frequency spectra, one for each timestep.

With the ASR model, the decoder was kickstarted using a sequence that just has the special "start" token in it. For the TTS model, we can start the decoding with a spectrogram of length one that is all zeros that acts as the "start token". Given this initial spectrogram and the cross-attentions over the encoder's hidden-state representations, the decoder then predicts the next timeslice for this spectrogram, steadily growing the spectrogram one timestep at a time.



But how does the decoder know when to stop? In the **SpeechT5** model this is handled by making the decoder predict a second sequence. This contains the probability that the current timestep is the last one. While generating audio at inference time, if this probability is over a certain threshold (say 0.5), the decoder is indicating that the spectrogram is finished and the generation loop should end.

After the decoding finishes and we have an output sequence containing the spectrogram, SpeechT5 uses a so-called **post-net** that is made up of several convolution layers to refine the spectrogram.

During training of the TTS model, the targets are also spectrograms and the loss is L1 or MSE. At inference time we want to convert the output spectrogram into an audio waveform, so that we can actually listen to it. For this an external model is used, the **vocoder**. This vocoder is not part of the seq2seq architecture and is trained separately.

What makes TTS difficult is that it's a one-to-many mapping. With speech-to-text there is only one correct output text that corresponds to the input speech, but with text-to-speech the input text can be mapped to many possible speech sounds. Different speakers may choose to emphasize different parts of the sentence, for example. This makes TTS models hard to evaluate. Because of this, the L1 or MSE loss value isn't actually very meaningful — there are multiple ways to represent the same text to a spectrogram. This is why TTS models are typically evaluated by human listeners, using a metric known as MOS or mean opinion score.

Conclusion

The seq2seq approach is more powerful than an encoder-only model. By separating the encoding of the input sequence from the decoding of the output sequence, the alignment of audio and text is less of a problem.

However, an encoder-decoder model is also slower as the decoding process happens one step at a time, rather than all at once. The longer the sequence, the slower the prediction. Autoregressive models can also get stuck in repetitions or skip words. Techniques such as beam search can help improve the quality of the predictions, but also slow down decoding even more.

Supplemental reading and resources

If you'd like to further explore different Transformer architectures, and learn about their various applications in speech processing, check out this recent paper:

Transformers in Speech Processing: A Survey

by Siddique Latif, Aun Zaidi, Heriberto Cuayahuitl, Fahad Shamshad, Moazzam Shoukat, Junaid Qadir

"The remarkable success of transformers in the field of natural language processing has sparked the interest of the speech-processing community, leading to an exploration of their potential for modeling long-range dependencies within speech

sequences. Recently, transformers have gained prominence across various speech-related domains, including automatic speech recognition, speech synthesis, speech translation, speech para-linguistics, speech enhancement, s poken dialogue systems, and numerous multimodal applications. In this paper, we present a comprehensive survey that aims to bridge research studies from diverse subfields within speech technology. By consolidating findings from across the speech technology landscape, we provide a valuable resource for researchers interested in harnessing the power of transformers to advance the field. We identify the challenges encountered by transformers in speech processing while also offering insights into potential solutions to address these issues."

arxiv.org/abs/2303.11607

Automatic speech recognition with a pipeline

Automatic Speech Recognition (ASR) is a task that involves transcribing speech audio recording into text. This task has numerous practical applications, from creating closed captions for videos to enabling voice commands for virtual assistants like Siri and Alexa.

In this section, we'll use the `automatic-speech-recognition` pipeline to transcribe an audio recording of a person asking a question about paying a bill using the same MINDS-14 dataset as before.

To get started, load the dataset and upsample it to 16kHz as described in Audio classification with a pipeline, if you haven't done that yet.

To transcribe an audio recording, we can use the `automatic-speech-recognition` pipeline from 🤗 Transformers. Let's instantiate the pipeline:

```
from transformers import pipeline

asr = pipeline("automatic-speech-recognition")
```

Next, we'll take an example from the dataset and pass its raw data to the pipeline:

```
example = minds[0]
asr(example["audio"]["array"])
```

Output:

```
```

Let's compare this output to what the actual transcription for this example is:

```
example["english_transcription"]
```

Output:

```
"I would like to pay my electricity bill using my card can you please assist"
```

The model seems to have done a pretty good job at transcribing the audio! It only got one word wrong ("card") compared to the original transcription, which is pretty good considering the speaker has an Australian accent, where the letter "r" is often silent. Having said that, I wouldn't recommend trying to pay your next electricity bill with a fish!

By default, this pipeline uses a model trained for automatic speech recognition for English language, which is fine in this example. If you'd like to try transcribing other subsets of MINDS-14 in different language, you can find a pre-trained ASR model on the 🤗 Hub. You can filter the models list by task first, then by language. Once you have found the model you like, pass it's name as the `model` argument to the pipeline.

Let's try this for the German split of the MINDS-14. Load the "de-DE" subset:

```
from datasets import load_dataset
from datasets import Audio

minds = load_dataset("PolyAI/minds14", name="de-DE", split="train")
minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
```

Get an example and see what the transcription is supposed to be:

```
example = minds[0]
example["transcription"]
```

Output:

```
"ich möchte gerne Geld auf mein Konto einzahlen"
```

Find a pre-trained ASR model for German language on the 🤗 Hub, instantiate a pipeline, and transcribe the example:

```
from transformers import pipeline

asr = pipeline("automatic-speech-recognition", model="maxidl/wav2vec2-large-xlsr-
german")
asr(example["audio"]["array"])
```

Output:

```

```

Also, stimmt's!

When working on solving your own task, starting with a simple pipeline like the ones we've shown in this unit is a valuable tool that offers several benefits: - a pre-trained model may exist that already solves your task really well, saving you plenty of time - pipeline() takes care of all the pre/post-processing for you, so you don't have to worry about getting the data into the right format for a model - if the result isn't ideal, this still gives you a quick baseline for future fine-tuning - once you fine-tune a model on your custom data and share it on Hub, the whole community will be able to use it quickly and effortlessly via the `pipeline()` method making AI more accessible.

Audio classification with a pipeline

Audio classification involves assigning one or more labels to an audio recording based on its content. The labels could correspond to different sound categories, such as music, speech, or noise, or more specific categories like bird song or car engine sounds.

Before diving into details on how the most popular audio transformers work, and before fine-tuning a custom model, let's see how you can use an off-the-shelf pre-trained model for audio classification with only a few lines of code with 🤗 Transformers.

Let's go ahead and use the same MINDS-14 dataset that you have explored in the previous unit. If you recall, MINDS-14 contains recordings of people asking an e-banking system questions in several languages and dialects, and has the `intent_class` for each recording. We can classify the recordings by intent of the call.

Just as before, let's start by loading the `en-AU` subset of the data to try out the pipeline, and upsample it to 16kHz sampling rate which is what most speech models require.

```
from datasets import load_dataset
from datasets import Audio

minds = load_dataset("PolyAI/minds14", name="en-AU", split="train")
minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
```

To classify an audio recording into a set of classes, we can use the `audio-classification` pipeline from 🤗 Transformers. In our case, we need a model that's been fine-tuned for intent classification, and specifically on the MINDS-14 dataset. Luckily for us, the Hub has a model that does just that! Let's load it by using the `pipeline()` function:

```
from transformers import pipeline

classifier = pipeline(
    "audio-classification",
    model="anton-l/xtreme_s_xlsr_300m_minds14",
)
```

This pipeline expects the audio data as a NumPy array. All the preprocessing of the raw audio data will be conveniently handled for us by the pipeline. Let's pick an example to try it out:

```
example = minds[0]
```

If you recall the structure of the dataset, the raw audio data is stored in a NumPy array under `["audio"]["array"]`, let's pass it straight to the `classifier`:

```
classifier(example["audio"]["array"])
```

Output:

```
[
    ,
    ,
    ,
    ,
    ,
]
```

The model is very confident that the caller intended to learn about paying their bill. Let's see what the actual label for this example is:

```
id2label = minds.features["intent_class"].int2str
id2label(example["intent_class"])
```

Output:

```
"pay_bill"
```

Hooray! The predicted label was correct! Here we were lucky to find a model that can classify the exact labels that we need. A lot of the times, when dealing with a classification task, a pre-trained model's set of classes is not exactly the same as the classes you need the model to distinguish. In this case, you can fine-tune a pre-trained model to "calibrate" it to your exact set of class labels. We'll learn how to do this in the upcoming units. Now, let's take a look at another very common task in speech processing, *automatic speech recognition*.

Hands-on exercise

This exercise is not graded and is intended to help you become familiar with the tools and libraries that you will be using throughout the rest of the course. If you are already experienced in using Google Colab, 🤗 Datasets, librosa and 🤗 Transformers, you may choose to skip this exercise.

1. Create a Google Colab notebook.
2. Use 🤗 Datasets to load the train split of the `facebook/voxpopuli` dataset in language of your choice in streaming mode.
3. Get the third example from the `train` part of the dataset and explore it. Given the features that this example has, what kinds of audio tasks can you use this dataset for?
4. Plot this example's waveform and spectrogram.
5. Go to 🤗 Hub, explore pretrained models and find one that can be used for automatic speech recognition for the language that you have picked earlier. Instantiate a corresponding pipeline with the model you found, and transcribe the example.
6. Compare the transcription that you get from the pipeline to the transcription provided in the example.

If you struggle with this exercise, feel free to take a peek at an example solution. Discovered something interesting? Found a cool model? Got a beautiful spectrogram? Feel free to share your work and discoveries on Twitter!

In the next chapters you'll learn more about various audio transformer architectures and will train your own model!

Unit 2. A gentle introduction to audio applications

Welcome to the second unit of the Hugging Face audio course! Previously, we explored the fundamentals of audio data and learned how to work with audio datasets using the 🤗 Datasets and 🤗 Transformers libraries. We discussed various concepts such as sampling rate, amplitude, bit depth, waveform, and spectrograms, and saw how to preprocess data to prepare it for a pre-trained model.

At this point you may be eager to learn about the audio tasks that 🤗 Transformers can handle, and you have all the foundational knowledge necessary to dive in! Let's take a look at some of the mind-blowing audio task examples:

- **Audio classification**: easily categorize audio clips into different categories. You can identify whether a recording is of a barking dog or a meowing cat, or what music genre a song belongs to.
- **Automatic speech recognition**: transform audio clips into text by transcribing them automatically. You can get a text representation of a recording of someone speaking, like "How are you doing today?". Rather useful for note taking!
- **Speaker diarization**: Ever wondered who's speaking in a recording? With 🤗 Transformers, you can identify which speaker is talking at any given time in an audio clip. Imagine being able to differentiate between "Alice" and "Bob" in a recording of them having a conversation.
- **Text to speech**: create a narrated version of a text that can be used to produce an audio book, help with accessibility, or give a voice to an NPC in a game. With 🤗 Transformers, you can easily do that!

In this unit, you'll learn how to use pre-trained models for some of these tasks using the `pipeline()` function from 🤗 Transformers. Specifically, we'll see how the pre-trained models can be used for audio classification, automatic speech recognition and audio generation. Let's get started!

Audio generation with a pipeline

Audio generation encompasses a versatile set of tasks that involve producing an audio output. The tasks that we will look into here are speech generation (aka "text-to-speech") and music generation. In text-to-speech, a model transforms a piece of text into lifelike spoken language sound, opening the door to applications such as virtual assistants, accessibility tools for the visually impaired, and personalized audiobooks. On the other hand, music generation can enable creative expression, and finds its use mostly in entertainment and game development industries.

In 🤗 Transformers, you'll find a pipeline that covers both of these tasks. This pipeline is called `"text-to-audio"`, but for convenience, it also has a `"text-to-speech"` alias. Here we'll use both, and you are free to pick whichever seems more applicable for your task.

Let's explore how you can use this pipeline to start generating audio narration for texts, and music with just a few lines of code.

This pipeline is new to 🤗 Transformers and comes part of the version 4.32 release. Thus you'll need to upgrade the library to the latest version to get the feature:

```
pip install --upgrade transformers
```

Generating speech

Let's begin by exploring text-to-speech generation. First, just as it was the case with audio classification and automatic speech recognition, we'll need to define the pipeline. We'll define a text-to-speech pipeline since it best describes our task, and use the `suno/bark-small` checkpoint:

```
from transformers import pipeline

pipe = pipeline("text-to-speech", model="suno/bark-small")
```

The next step is as simple as passing some text through the pipeline. All the preprocessing will be done for us under the hood:

```
text = "Ladybugs have had important roles in culture and religion, being associated
with luck, love, fertility and prophecy. "
output = pipe(text)
```

In a notebook, we can use the following code snippet to listen to the result:

```
from IPython.display import Audio

Audio(output["audio"].squeeze(), rate=output["sampling_rate"])
```

The model that we're using with the pipeline, Bark, is actually multilingual, so we can easily substitute the initial text with a text in, say, French, and use the pipeline in the exact same way. It will pick up on the language all by itself:

```
fr_text = "Contrairement à une idée répandue, le nombre de points sur les élytres
d'une coccinelle ne correspond pas à son âge, ni en nombre d'années, ni en nombre
de mois. "
output = pipe(fr_text)
Audio(output["audio"].squeeze(), rate=output["sampling_rate"])
```

Not only is this model multilingual, it can also generate audio with non-verbal communications and singing. Here's how you can make it sing:

```
song = "♪ In the jungle, the mighty jungle, the ladybug was seen. ♪ "
output = pipe(song)
Audio(output["audio"].squeeze(), rate=output["sampling_rate"])
```

We'll dive deeper into Bark specifics in the later unit dedicated to Text-to-speech, and will also show how you can use other models for this task. Now, let's generate some music!

Generating music

Just as before, we'll begin by instantiating a pipeline. For music generation, we'll define a text-to-audio pipeline, and initialise it with the pretrained checkpoint `facebook/musicgen-small`

```
music_pipe = pipeline("text-to-audio", model="facebook/musicgen-small")
```

Let's create a text description of the music we'd like to generate:

```
text = "90s rock song with electric guitar and heavy drums"
```

We can control the length of the generated output by passing an additional `max_new_tokens` parameter to the model.

```
forward_params =

output = music_pipe(text, forward_params=forward_params)
Audio(output["audio"][0], rate=output["sampling_rate"])
```

Join the community!

We invite you to join our vibrant and supportive community on Discord. You will have the opportunity to connect with like-minded learners, exchange ideas, and get valuable feedback on your hands-on exercises. You can ask questions, share resources, and collaborate with others.

Our team is also active on Discord, and they are available to provide support and guidance when you need it. Joining our community is an excellent way to stay motivated, engaged, and connected, and we look forward to seeing you there!

What is Discord?

Discord is a free chat platform. If you've used Slack, you'll find it quite similar. The Hugging Face Discord server is a home to a thriving community of 18 000 AI experts, learners and enthusiasts that you can be a part of.

Navigating Discord

Once you sign up to our Discord server, you'll need to pick the topics you're interested in by clicking `#role-assignment` at the left. You can choose as many different categories as you like. To join other learners of this course, make sure to click "ML for Audio and Speech". Explore the channels and share a few things about you in the `#introduce-yourself` channel.

Audio course channels

There are many channels focused on various topics on our Discord server. You'll find people discussing papers, organizing events, sharing their projects and ideas, brainstorming, and so much more.

As an audio course learner, you may find the following set of channels particularly relevant:

- `#audio-announcements`: updates about the course, news from the Hugging Face related to everything audio, event announcements, and more.
- `#audio-study-group`: a place to exchange ideas, ask questions about the course and start discussions.
- `#audio-discuss`: a general place to have discussions about things related to audio.

In addition to joining the `#audio-study-group`, feel free to create your own study group, learning together is always easier!

Get ready to take the course

We hope that you are excited to get started with the course, and we have designed this page to make sure you have everything you need to jump right in!

Step 1. Sign up

To stay up to date with all the updates and special social events, sign up to the course.

👉 SIGN UP

Step 2. Get a Hugging Face account

If you don't yet have one, create a Hugging Face account (it's free). You'll need it to complete hands-on tasks, to receive your certificate of completion, to explore pre-trained models, to access datasets and more.

👉 CREATE HUGGING FACE ACCOUNT

Step 3. Brush up on fundamentals (if you need to)

We assume that you are familiar with deep learning basics, and have general familiarity with transformers. If you need to brush up on your understanding of transformers, check out our NLP Course.

Step 4. Check your setup

To go through the course materials you will need: - A computer with an internet connection - Google Colab for hands-on exercises. The free version is enough. If you have never used Google Colab before, check out this official introduction notebook.

As an alternative to the free tier of Google Colab, you can use your own local setup, or Kaggle Notebooks. Kaggle Notebooks offer a fixed number of GPU hours and have similar functionality to Google Colab, however, there are differences when it comes to sharing your models on 🤗 Hub (e.g. for completing assignments). If you decide to use Kaggle Notebooks as your tool of choice, check out the example Kaggle notebook created by @michaelshekasta. This notebook illustrates how you can train and share your trained model on 🤗 Hub.

Step 5. Join the community

Sign up to our Discord server, the place where you can exchange ideas with your classmates and reach out to us (the Hugging Face team).

👉 JOIN THE COMMUNITY ON DISCORD

To learn more about our community on Discord and how to make the most of it, check out the next page.

Welcome to the Hugging Face Audio course!

Dear learner,

Welcome to this course on using transformers for audio. Time and again transformers have proven themselves as one of the most powerful and versatile deep learning architectures, capable of achieving state-of-the-art results in a wide range of tasks, including natural language processing, computer vision, and more recently, audio processing.

In this course, we will explore how transformers can be applied to audio data. You'll learn how to use them to tackle a range of audio-related tasks. Whether you are interested in speech recognition, audio classification, or generating speech from text, transformers and this course have got you covered.

To give you a taste of what these models can do, say a few words in the demo below and watch the model transcribe it in real-time!

I

Throughout the course, you will gain an understanding of the specifics of working with audio data, you'll learn about different transformer architectures, and you'll train your own audio transformers leveraging powerful pre-trained models.

This course is designed for learners with a background in deep learning, and general familiarity with transformers. No expertise in audio data processing is required. If you need to brush up on your understanding of transformers, check out our NLP Course that goes into much detail on the transformer basics.

Meet the course team

Sanchit Gandhi, Machine Learning Research Engineer at Hugging Face

Hi! I'm Sanchit and I'm a machine learning research engineer for audio in the open-source team at Hugging Face 🤗. My primary focus is automatic speech recognition and translation, with the current goal of making speech models faster, lighter and easier to use.

Matthijs Hollemans, Machine Learning Engineer at Hugging Face

I'm Matthijs, and I'm a machine learning engineer for audio in the open source team at Hugging Face. I'm also the author of a book on how to write sound synthesizers, and I create audio plug-ins in my spare time.

Maria Khalusova, Documentation & Courses at Hugging Face

I'm Maria, and I create educational content and documentation to make Transformers and other open-source tools even more accessible. I break down complex technical concepts and help folks get started with cutting-edge technologies.

Vaibhav Srivastav, ML Developer Advocate Engineer at Hugging Face

I'm Vaibhav (VB) and I'm a Developer Advocate Engineer for Audio in the Open Source team at Hugging Face. I research low-resource Text to Speech and help bring SoTA speech research to the masses.

Course structure

The course is structured into several units that covers various topics in depth:

- Unit 1: learn about the specifics of working with audio data, including audio processing techniques and data preparation.
- Unit 2: get to know audio applications and learn how to use 🤗 Transformers pipelines for different tasks, such as audio classification and speech recognition.
- Unit 3: explore audio transformer architectures, learn how they differ, and what tasks they are best suited for.
- Unit 4: learn how to build your own music genre classifier.
- Unit 5: delve into speech recognition and build a model to transcribe meeting recordings.
- Unit 6: learn how to generate speech from text.
- Unit 7: learn how to build real-world audio applications with transformers.

Each unit includes a theoretical component, where you will gain a deep understanding of the underlying concepts and techniques. Throughout the course, we provide quizzes to help you test your knowledge and reinforce your learning. Some chapters also include hands-on exercises, where you will have the opportunity to apply what you have learned.

By the end of the course, you will have a strong foundation in using transformers for audio data and will be well-equipped to apply these techniques to a wide range of audio-related tasks.

The course units will be released in several consecutive blocks with the following publishing schedule:

Units	Publishing date
Unit 0, Unit 1, and Unit 2	June 14, 2023
Unit 3, Unit 4	June 21, 2023
Unit 5	June 28, 2023
Unit 6	July 5, 2023
Unit 7, Unit 8	July 12, 2023

Learning paths and certification

There is no right or wrong way to take this course. All the materials in this course are 100% free, public and open-source. You can take the course at your own pace, however, we recommend going through the units in their order.

If you'd like to get certified upon the course completion, we offer two options:

Certificate type	Requirements
Certificate of completion	Complete 80% of the hands-on exercises according to instructions.
Certificate of honors	Complete 100% of the hands-on exercises according to instructions.

Each hands-on exercise outlines its completion criteria. Once you have completed enough hands-on exercises to qualify for either of the certificates, refer to the last unit of the course to learn how you can get your certificate. Good luck!

Sign up to the course

The units of this course will be released gradually over the course of a few weeks. We encourage you to sign up to the course updates so that you don't miss new units when they are released. Learners who sign up to the course updates will also be the first ones to learn about special social events that we plan to host.

SIGN UP

Enjoy the course!

Live sessions and workshops

New Audio Transformers Course: Live Launch Event with Paige Bailey (DeepMind), Seokhwan Kim (Amazon Alexa AI), and Brian McFee (Librosa)

The recording of a Live AMA with the Hugging Face Audio course team:

Pre-trained models for automatic speech recognition

In this section, we'll cover how to use the `pipeline()` to leverage pre-trained models for speech recognition. In Unit 2, we introduced the `pipeline()` as an easy way of running speech recognition tasks, with all pre- and post-processing handled under-the-hood and the flexibility to quickly experiment with any pre-trained checkpoint on the Hugging Face Hub. In this Unit, we'll go a level deeper and explore the different attributes of speech recognition models and how we can use them to tackle a range of different tasks.

As detailed in Unit 3, speech recognition model broadly fall into one of two categories:

1. Connectionist Temporal Classification (CTC): *encoder-only* models with a linear classification (CTC) head on top
2. Sequence-to-sequence (Seq2Seq): *encoder-decoder* models, with a cross-attention mechanism between the encoder and decoder

Prior to 2022, CTC was the more popular of the two architectures, with encoder-only models such as Wav2Vec2, HuBERT and XLSR achieving breakthoughs in the pre-training / fine-tuning paradigm for speech. Big corporations, such as Meta and Microsoft, pre-trained the encoder on vast amounts of unlabelled audio data for many days or weeks. Users could then take a pre-trained checkpoint, and fine-tune it with a CTC head on as little as **10 minutes** of labelled speech data to achieve strong performance on a downstream speech recognition task.

However, CTC models have their shortcomings. Appending a simple linear layer to an encoder gives a small, fast overall model, but can be prone to phonetic spelling errors. We'll demonstrate this for the Wav2Vec2 model below.

Probing CTC Models

Let's load a small excerpt of the LibriSpeech ASR dataset to demonstrate Wav2Vec2's speech transcription capabilities:

```
from datasets import load_dataset

dataset = load_dataset(
    "hf-internal-testing/librispeech_asr_dummy", "clean", split="validation"
```

```
)
dataset
```

Output:

```
Dataset()
```

We can pick one of the 73 audio samples and inspect the audio sample as well as the transcription:

```
from IPython.display import Audio

sample = dataset[2]

print(sample["text"])
Audio(sample["audio"]["array"], rate=sample["audio"]["sampling_rate"])
```

Output:

```
HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH CHRISTMAS AND ROAST BEEF
LOOMING BEFORE US SIMILES DRAWN FROM EATING AND ITS RESULTS OCCUR MOST READILY TO
THE MIND
```

Alright! Christmas and roast beef, sounds great! 🎄 Having chosen a data sample, we now load a fine-tuned checkpoint into the `pipeline()`. For this, we'll use the official Wav2Vec2 base checkpoint fine-tuned on 100 hours of LibriSpeech data:

```
from transformers import pipeline

pipe = pipeline("automatic-speech-recognition", model="facebook/wav2vec2-
base-100h")
```

Next, we'll take an example from the dataset and pass its raw data to the pipeline. Since the `pipeline` *consumes* any dictionary that we pass it (meaning it cannot be re-used), we'll pass a copy of the data. This way, we can safely re-use the same audio sample in the following examples:

```
pipe(sample["audio"].copy())
```

Output:

```

```

We can see that the Wav2Vec2 model does a pretty good job at transcribing this sample - at a first glance it looks generally correct. Let's put the target and prediction side-by-side and highlight the differences:

```
Target:      HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH CHRISTMAS AND
ROAST BEEF LOOMING BEFORE US SIMILES DRAWN FROM EATING AND ITS RESULTS OCCUR MOST
READILY TO THE MIND
Prediction:  HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH
**CHRISTMAUS** AND **ROSE** BEEF LOOMING BEFORE US **SIMALYIS** DRAWN FROM EATING
AND ITS RESULTS OCCUR MOST READILY TO THE MIND
```

Comparing the target text to the predicted transcription, we can see that all words *sound* correct, but some are not spelled accurately. For example:

- *CHRISTMAUS* vs. *CHRISTMAS*
- *ROSE* vs. *ROAST*
- *SIMALYIS* vs. *SIMILES*

This highlights the shortcoming of a CTC model. A CTC model is essentially an 'acoustic-only' model: it consists of an encoder which forms hidden-state representations from the audio inputs, and a linear layer which maps the hidden-states to characters:

This means that the system almost entirely bases its prediction on the acoustic input it was given (the phonetic sounds of the audio), and so has a tendency to transcribe the audio in a phonetic way (e.g. *CHRISTMAUS*). It gives less importance to the language modelling context of previous and successive letters, and so is prone to phonetic spelling errors. A more intelligent model would identify that *CHRISTMAUS* is not a valid word in the English vocabulary, and correct it to *CHRISTMAS* when making its predictions. We're also missing two big features in our prediction - casing and punctuation - which limits the usefulness of the model's transcriptions to real-world applications.

Graduation to Seq2Seq

Cue Seq2Seq models! As outlined in Unit 3, Seq2Seq models are formed of an encoder and decoder linked via a cross-attention mechanism. The encoder plays the same role as before, computing hidden-state representations of the audio inputs, while the decoder plays the role of a **language model**. The decoder processes the entire sequence of hidden-state representations from the encoder and generates the corresponding text transcriptions. With global context of the audio input, the decoder is able to use language modelling context as it makes its predictions,

correcting for spelling mistakes on-the-fly and thus circumventing the issue of phonetic predictions.

There are two downsides to Seq2Seq models: 1. They are inherently slower at decoding, since the decoding process happens one step at a time, rather than all at once 2. They are more data hungry, requiring significantly more training data to reach convergence

In particular, the need for large amounts of training data has been a bottleneck in the advancement of Seq2Seq architectures for speech. Labelled speech data is difficult to come by, with the largest annotated datasets at the time clocking in at just 10,000 hours. This all changed in 2022 upon the release of **Whisper**. Whisper is a pre-trained model for speech recognition published in September 2022 by the authors Alec Radford et al. from OpenAI. Unlike its CTC predecessors, which were pre-trained entirely on **un-labelled** audio data, Whisper is pre-trained on a vast quantity of **labelled** audio-transcription data, 680,000 hours to be precise.

This is an order of magnitude more data than the un-labelled audio data used to train Wav2Vec 2.0 (60,000 hours). What is more, 117,000 hours of this pre-training data is multilingual (or "non-English") data. This results in checkpoints that can be applied to over 96 languages, many of which are considered *low-resource*, meaning the language lacks a large corpus of data suitable for training.

When scaled to 680,000 hours of labelled pre-training data, Whisper models demonstrate a strong ability to generalise to many datasets and domains. The pre-trained checkpoints achieve competitive results to state-of-the-art pipe systems, with near 3% word error rate (WER) on the test-clean subset of LibriSpeech pipe and a new state-of-the-art on TED-LIUM with 4.7% WER (*c.f.* Table 8 of the Whisper paper).

Of particular importance is Whisper's ability to handle long-form audio samples, its robustness to input noise and ability to predict cased and punctuated transcriptions. This makes it a viable candidate for real-world speech recognition systems.

The remainder of this section will show you how to use the pre-trained Whisper models for speech recognition using 🤗 Transformers. In many situations, the pre-trained Whisper checkpoints are extremely performant and give great results, thus we encourage you to try using the pre-trained checkpoints as a first step to solving any speech recognition problem. Through fine-tuning, the pre-trained checkpoints can be adapted for specific datasets and languages to further improve upon these results. We'll demonstrate how to do this in the upcoming subsection on fine-tuning.

The Whisper checkpoints come in five configurations of varying model sizes. The smallest four are trained on either English-only or multilingual data. The largest checkpoint is multilingual only. All nine of the pre-trained checkpoints are available on the Hugging Face Hub. The checkpoints are summarised in the following table with links to the models on the Hub. "VRAM" denotes the required GPU memory to run the model with the minimum batch size of 1. "Rel Speed" is the relative speed of a checkpoint compared to the largest model. Based on this information, you can select a checkpoint that is best suited to your hardware.

Size	Parameters	VRAM / GB	Rel Speed	English-only	Multilingual
tiny	39 M	1.4	32	✓	✓
base	74 M	1.5	16	✓	✓
small	244 M	2.3	6	✓	✓
medium	769 M	4.2	2	✓	✓
large	1550 M	7.5	1	x	✓

Let's load the Whisper Base checkpoint, which is of comparable size to the Wav2Vec2 checkpoint we used previously. Preempting our move to multilingual speech recognition, we'll load the multilingual variant of the base checkpoint. We'll also load the model on the GPU if available, or CPU otherwise. The `pipeline()` will subsequently take care of moving all inputs / outputs from the CPU to the GPU as required:

```
from transformers import pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
pipe = pipeline(
    "automatic-speech-recognition", model="openai/whisper-base", device=device
)
```

Great! Now let's transcribe the audio as before. The only change we make is passing an extra argument, `max_new_tokens`, which tells the model the maximum number of tokens to generate when making its prediction:

```
pipe(sample["audio"], max_new_tokens=256)
```

Output:

```
```

Easy enough! The first thing you'll notice is the presence of both casing and punctuation. Immediately this makes the transcription easier to read compared to the un-cased and un-punctuated transcription from Wav2Vec2. Let's put the transcription side-by-side with the target:

```
Target:     HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH CHRISTMAS AND
ROAST BEEF LOOMING BEFORE US SIMILES DRAWN FROM EATING AND ITS RESULTS OCCUR MOST
READILY TO THE MIND
Prediction: He tells us that at this festive season of the year, with **Christmas**
and **roast** beef looming before us, **similarly** is drawn from eating and its
results occur most readily to the mind.
```

Whisper has done a great job at correcting the phonetic errors we saw from Wav2Vec2 - both *Christmas* and *roast* are spelled correctly. We see that the model still struggles with *SIMILES*, being incorrectly transcribed as *similarly*, but this time the prediction is a valid word from the English vocabulary. Using a larger Whisper checkpoint can help further reduce transcription errors, at the expense of requiring more compute and a longer transcription time.

We've been promised a model that can handle 96 languages, so lets leave English speech recognition for now and go global 🌎! The Multilingual LibriSpeech (MLS) dataset is the multilingual equivalent of the LibriSpeech dataset, with labelled audio data in six languages. We'll load one sample from the Spanish split of the MLS dataset, making use of *streaming* mode so that we don't have to download the entire dataset:

```
dataset = load_dataset(
    "facebook/multilingual_librispeech", "spanish", split="validation",
streaming=True
)
sample = next(iter(dataset))
```

Again, we'll inspect the text transcription and take a listen to the audio segment:

```
print(sample["text"])
Audio(sample["audio"]["array"], rate=sample["audio"]["sampling_rate"])
```

Output:

```
entonces te delelitarás en jehová y yo te haré subir sobre las alturas de la tierra
y te daré á comer la heredad de jacob tu padre porque la boca de jehová lo ha
hablado
```

This is the target text that we're aiming for with our Whisper transcription. Although we now know that we can probably do better this, since our model is also going to predict punctuation and casing, neither of which are present in the reference. Let's forward the audio sample to the pipeline to get our text prediction. One thing to note is that the pipeline *consumes* the dictionary of audio inputs that we input, meaning the dictionary can't be re-used. To circumvent this, we'll pass a *copy* of the audio sample, so that we can re-use the same audio sample in the proceeding code examples:

```
pipe(sample["audio"].copy(), max_new_tokens=256, generate_kwargs=)
```

Output:

```

```

Great - this looks very similar to our reference text (arguably better since it has punctuation and casing!). You'll notice that we forwarded the `"task"` as a *generate key-word argument* (generate kwarg). Setting the `"task"` to `"transcribe"` forces Whisper to perform the task of *speech recognition*, where the audio is transcribed in the same language that the speech was spoken in. Whisper is also capable of performing the closely related task of *speech translation*, where the audio in Spanish can be translated to text in English. To achieve this, we set the `"task"` to `"translate"`:

```
pipe(sample["audio"], max_new_tokens=256, generate_kwargs=)
```

Output:

```

```

Now that we know we can toggle between speech recognition and speech translation, we can pick our task depending on our needs. Either we recognise from audio in language X to text in the same language X (e.g. Spanish audio to Spanish text), or we translate from audio in any language X to text in English (e.g. Spanish audio to English text).

To read more about how the `"task"` argument is used to control the properties of the generated text, refer to the model card for the Whisper base model.

Long-Form Transcription and Timestamps

So far, we've focussed on transcribing short audio samples of less than 30 seconds. We mentioned that one of the appeals of Whisper was its ability to work on long audio samples. We'll tackle this task here!

Let's create a long audio file by concatenating sequential samples from the MLS dataset. Since the MLS dataset is curated by splitting long audiobook recordings into shorter segments, concatenating samples is one way of reconstructing longer audiobook passages. Consequently, the resulting audio should be coherent across the entire sample.

We'll set our target audio length to 5 minutes, and stop concatenating samples once we hit this value:

```
target_length_in_m = 5

# convert from minutes to seconds (* 60) to num samples (* sampling rate)
sampling_rate = pipe.feature_extractor.sampling_rate
target_length_in_samples = target_length_in_m * 60 * sampling_rate

# iterate over our streaming dataset, concatenating samples until we hit our target
long_audio = []
for sample in dataset:
    long_audio.extend(sample["audio"]["array"])
    if len(long_audio) > target_length_in_samples:
        break

long_audio = np.asarray(long_audio)

# how did we do?
seconds = len(long_audio) / 16000
minutes, seconds = divmod(seconds, 60)
print(f"Length of audio sample is  minutes  seconds")
```

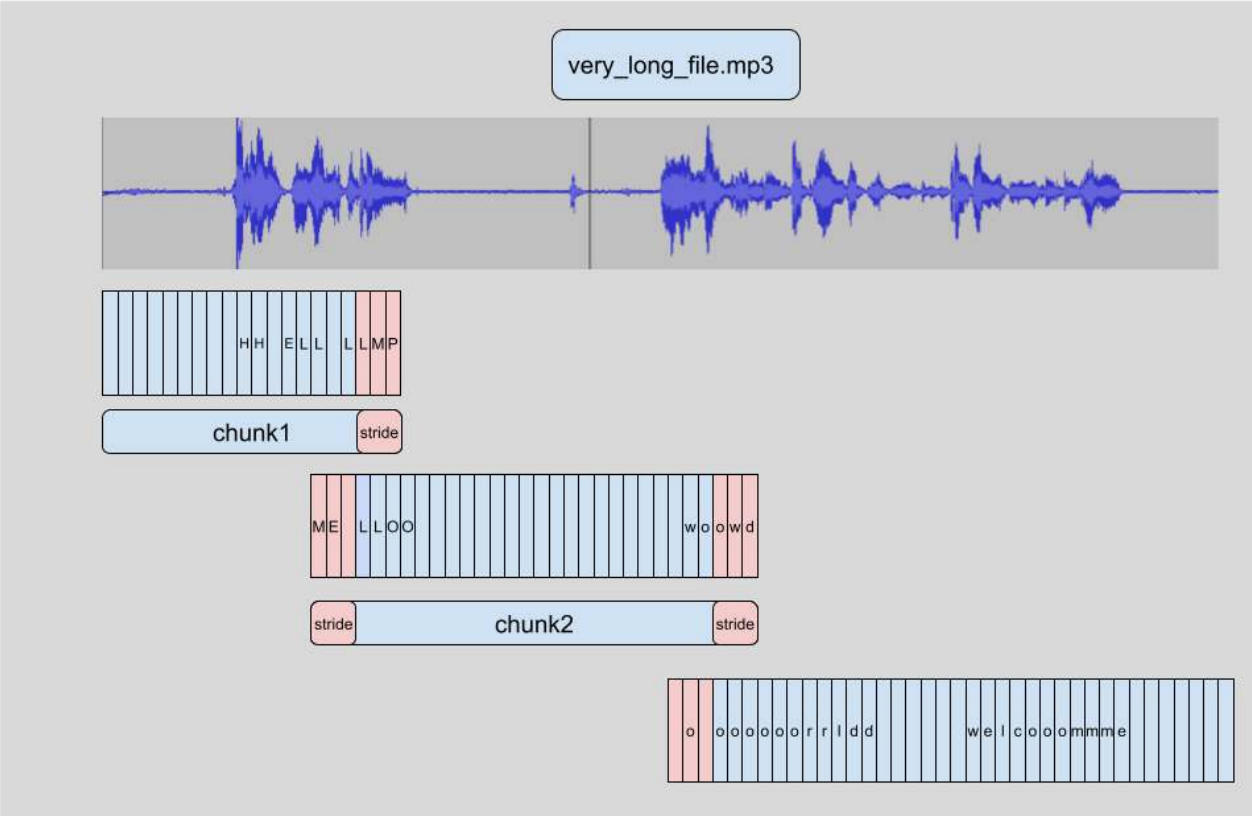
Output:

```
Length of audio sample is 5.0 minutes 17.22 seconds
```

Alright! 5 minutes and 17 seconds of audio to transcribe. There are two problems with forwarding this long audio sample directly to the model: 1. Whisper is inherently designed to work with 30 second samples: anything shorter than 30s is padded to 30s with silence, anything longer than 30s is truncated to 30s by cutting

of the extra audio, so if we pass our audio directly we'll only get the transcription for the first 30s 2. Memory in a transformer network scales with the sequence length squared: doubling the input length quadruples the memory requirement, so passing super long audio files is bound to lead to an out-of-memory (OOM) error

The way long-form transcription works in 🤗 Transformers is by *chunking* the input audio into smaller, more manageable segments. Each segment has a small amount of overlap with the previous one. This allows us to accurately stitch the segments back together at the boundaries, since we can find the overlap between segments and merge the transcriptions accordingly:



The advantage of chunking the samples is that we don't need the result of chunk \(i \) to transcribe the subsequent chunk \(i + 1 \). The stitching is done after we have transcribed all the chunks at the chunk boundaries, so it doesn't matter which order we transcribe chunks in. The algorithm is entirely **stateless**, so we can even do chunk \(i + 1 \) at the same time as chunk \(i \)! This allows us to *batch* the chunks and run them through the model in parallel, providing a large computational speed-up compared to transcribing them sequentially. To read more about chunking in 🤗 Transformers, you can refer to this blog post.

To activate long-form transcriptions, we have to add one additional argument when we call the pipeline. This argument, `chunk_length_s`, controls the length of the

chunked segments in seconds. For Whisper, 30 second chunks are optimal, since this matches the input length Whisper expects.

To activate batching, we need to pass the argument `batch_size` to the pipeline. Putting it all together, we can transcribe the long audio sample with chunking and batching as follows:

```
pipe(
    long_audio,
    max_new_tokens=256,
    generate_kwargs=,
    chunk_length_s=30,
    batch_size=8,
)
```

Output:

```
,
    chunk_length_s=30,
    batch_size=8,
    return_timestamps=True,
)["chunks"]
```

Output:

```
[,
 ,
 ,
 ...
```

And voila! We have our predicted text as well as corresponding timestamps.

Summary

Whisper is a strong pre-trained model for speech recognition and translation. Compared to Wav2Vec2, it has higher transcription accuracy, with outputs that contain punctuation and casing. It can be used to transcribe speech in English as well as 96 other languages, both on short audio segments and longer ones through *chunking*. These attributes make it a viable model for many speech recognition and translation tasks without the need for fine-tuning. The `pipeline()` method provides an easy way of running inference in one-line API calls with control over the generated predictions.

While the Whisper model performs extremely well on many high-resource languages, it has lower transcription and translation accuracy on low-resource languages, i.e. those with less readily available training data. There is also varying performance across different accents and dialects of certain languages, including lower accuracy for speakers of different genders, races, ages or other demographic criteria (*c.f.* Whisper paper).

To boost the performance on low-resource languages, accents or dialects, we can take the pre-trained Whisper model and train it on a small corpus of appropriately selected data, in a process called *fine-tuning*. We'll show that with as little as ten hours of additional data, we can improve the performance of the Whisper model by over 100% on a low-resource language. In the next section, we'll cover the process behind selecting a dataset for fine-tuning.

Choosing a dataset

As with any machine learning problem, our model is only as good as the data that we train it on. Speech recognition datasets vary considerably in how they are curated and the domains that they cover. To pick the right dataset, we need to match our criteria with the features that a dataset offers.

Before we pick a dataset, we first need to understand the key defining features.

Features of speech datasets

1. Number of hours

Simply put, the number of training hours indicates how large the dataset is. It’s analogous to the number of training examples in an NLP dataset. However, bigger datasets aren’t necessarily better. If we want a model that generalises well, we want a **diverse** dataset with lots of different speakers, domains and speaking styles.

2. Domain

The domain entails where the data was sourced from, whether it be audiobooks, podcasts, YouTube or financial meetings. Each domain has a different distribution of data. For example, audiobooks are recorded in high-quality studio conditions (with no background noise) and text that is taken from written literature. Whereas for

YouTube, the audio likely contains more background noise and a more informal style of speech.

We need to match our domain to the conditions we anticipate at inference time. For instance, if we train our model on audiobooks, we can't expect it to perform well in noisy environments.

3. Speaking style

The speaking style falls into one of two categories:

- Narrated: read from a script
- Spontaneous: un-scripted, conversational speech

The audio and text data reflect the style of speaking. Since narrated text is scripted, it tends to be spoken articulately and without any errors:

```
“Consider the task of training a model on a speech recognition dataset”
```

Whereas for spontaneous speech, we can expect a more colloquial style of speech, with the inclusion of repetitions, hesitations and false-starts:

```
“Let’s uhh let's take a look at how you'd go about training a model on uhm a sp-
speech recognition dataset”
```

4. Transcription style

The transcription style refers to whether the target text has punctuation, casing or both. If we want a system to generate fully formatted text that could be used for a publication or meeting transcription, we require training data with punctuation and casing. If we just require the spoken words in an un-formatted structure, neither punctuation nor casing are necessary. In this case, we can either pick a dataset without punctuation or casing, or pick one that has punctuation and casing and then subsequently remove them from the target text through pre-processing.

A summary of datasets on the Hub

Here is a summary of the most popular English speech recognition datasets on the Hugging Face Hub:

Dataset	Train Hours	Domain	Speaking Style	Casing	Punctuation	License	Recommende Use
LibriSpeech	960	Audiobook	Narrated	❌	❌	CC-BY-4.0	Academic benchmarks
Common Voice 11	3000	Wikipedia	Narrated	✅	✅	CC0-1.0	Non-native speakers
VoxPopuli	540	European Parliament	Oratory	❌	✅	CC0	Non-native speakers
TED-LIUM	450	TED talks	Oratory	❌	❌	CC-BY-NC-ND 3.0	Technical topic
GigaSpeech	10000	Audiobook, podcast, YouTube	Narrated, spontaneous	❌	✅	apache-2.0	Robustness ov multiple domains
SPGISpeech	5000	Financial meetings	Oratory, spontaneous	✅	✅	User Agreement	Fully formatted transcriptions
Earnings-22	119	Financial meetings	Oratory, spontaneous	✅	✅	CC-BY-SA-4.0	Diversity of accents
AMI	100	Meetings	Spontaneous	✅	✅	CC-BY-4.0	Noisy speech conditions

This table serves as a reference for selecting a dataset based on your criterion. Below is an equivalent table for multilingual speech recognition. Note that we omit the train hours column, since this varies depending on the language for each dataset, and replace it with the number of languages per dataset:

Dataset	Languages	Domain	Speaking Style	Casing	Punctuation	License	Recommen Usage
Multilingual LibriSpeech	6	Audiobooks	Narrated	❌	❌	CC-BY-4.0	Academic benchmarks
Common Voice 13	108	Wikipedia text & crowd-sourced speech	Narrated	✅	✅	CC0-1.0	Diverse spe set
VoxPopuli	15		Spontaneous	❌	✅	CC0	

Dataset	Languages	Domain	Speaking Style	Casing	Punctuation	License	Recommen Usage
		European Parliament recordings					European languages
FLEURS	101	European Parliament recordings	Spontaneous	❌	❌	CC-BY-4.0	Multilingual evaluation

For a detailed breakdown of the audio datasets covered in both tables, refer to the blog post A Complete Guide to Audio Datasets. While there are over 180 speech recognition datasets on the Hub, it may be possible that there isn't a dataset that matches your needs. In this case, it's also possible to use your own audio data with 🤗 Datasets. To create a custom audio dataset, refer to the guide Create an audio dataset. When creating a custom audio dataset, consider sharing the final dataset on the Hub so that others in the community can benefit from your efforts - the audio community is inclusive and wide-ranging, and others will appreciate your work as you do theirs.

Alright! Now that we've gone through all the criterion for selecting an ASR dataset, let's pick one for the purpose of this tutorial. We know that Whisper already does a pretty good job at transcribing data in high-resource languages (such as English and Spanish), so we'll focus ourselves on low-resource multilingual transcription. We want to retain Whisper's ability to predict punctuation and casing, so it seems from the second table that Common Voice 13 is a great candidate dataset!

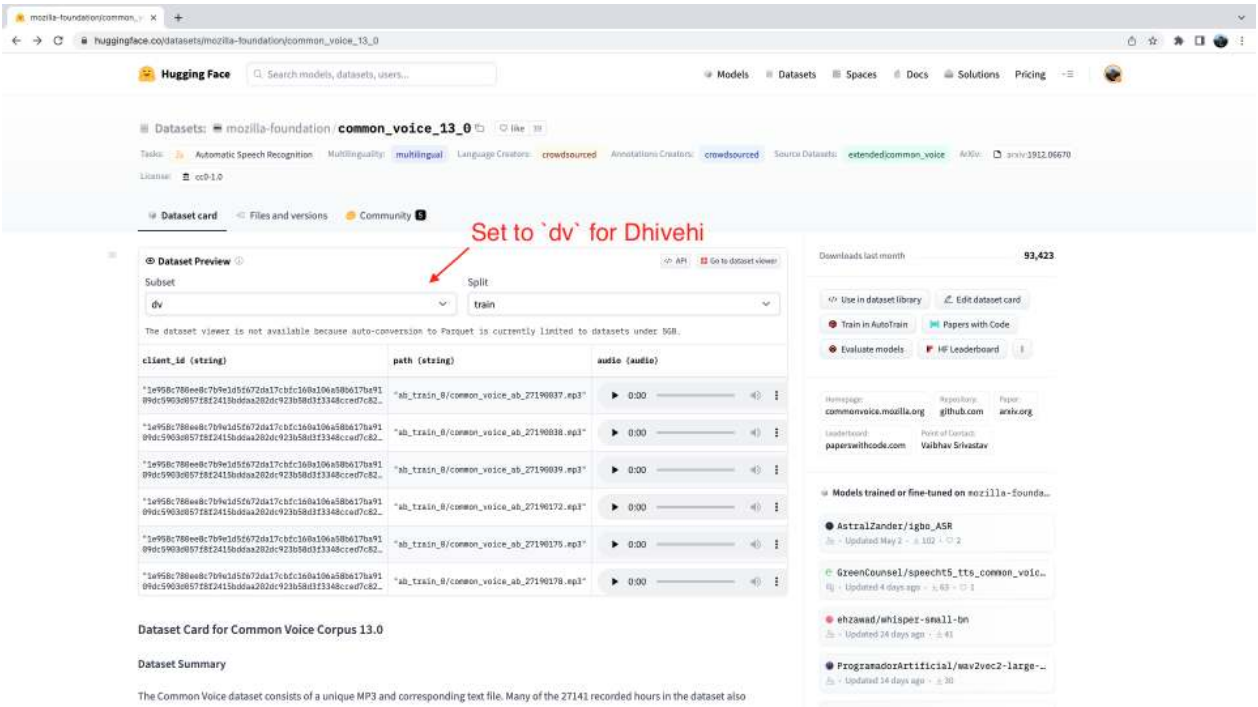
Common Voice 13

Common Voice 13 is a crowd-sourced dataset where speakers record text from Wikipedia in various languages. It forms part of the Common Voice series, a collection of Common Voice datasets released by Mozilla Foundation. At the time of writing, Common Voice 13 is the latest edition of the dataset, with the most languages and hours per language out of any release to date.

We can get the full list of languages for the Common Voice 13 dataset by checking-out the dataset page on the Hub: mozilla-foundation/common_voice_13_0. The first time you view this page, you'll be asked to accept the terms of use. After that, you'll be given full access to the dataset.

Once we've provided authentication to use the dataset, we'll be presented with the dataset preview. The dataset preview shows us the first 100 samples of the dataset for each language. What's more, it's loaded up with audio samples ready for us to listen to in real time. For this Unit, we'll select *Dhivehi* (or *Maldivian*), an Indo-Aryan language spoken in the South Asian island country of the Maldives. While we're selecting Dhivehi for this tutorial, the steps covered here apply to any one of the 108 languages in the Common Voice 13 dataset, and more generally to any one of the 180+ audio datasets on the Hugging Face Hub, so there's no restriction on language or dialect.

We can select the Dhivehi subset of Common Voice 13 by setting the subset to `dv` using the dropdown menu (`dv` being the language identifier code for Dhivehi):



If we hit the play button on the first sample, we can listen to the audio and see the corresponding text. Have a scroll through the samples for the train and test sets to get a better feel for the audio and text data that we're dealing with. You can tell from the intonation and style that the recordings are taken from narrated speech. You'll also likely notice the large variation in speakers and recording quality, a common trait of crowd-sourced data.

The Dataset Preview is a brilliant way of experiencing audio datasets before committing to using them. You can pick any dataset on the Hub, scroll through the samples and listen to the audio for the different subsets and splits, gauging whether it's the right dataset for your needs. Once you've selected a dataset, it's trivial to load the data so that you can start using it.

Now, I personally don't speak Dhivehi, and expect the vast majority of readers not to either! To know if our fine-tuned model is any good, we'll need a rigorous way of *evaluating* it on unseen data and measuring its transcription accuracy. We'll cover exactly this in the next section!

Build a demo with Gradio

Now that we've fine-tuned a Whisper model for Dhivehi speech recognition, let's go ahead and build a Gradio demo to showcase it to the community!

The first thing to do is load up the fine-tuned checkpoint using the `pipeline()` class - this is very familiar now from the section on pre-trained models. You can change the `model_id` to the namespace of your fine-tuned model on the Hugging Face Hub, or one of the pre-trained Whisper models to perform zero-shot speech recognition:

```
from transformers import pipeline

model_id = "sanchit-gandhi/whisper-small-dv"  # update with your model id
pipe = pipeline("automatic-speech-recognition", model=model_id)
```

Secondly, we'll define a function that takes the filepath for an audio input and passes it through the pipeline. Here, the pipeline automatically takes care of loading the audio file, resampling it to the correct sampling rate, and running inference with the model. We can then simply return the transcribed text as the output of the function. To ensure our model can handle audio inputs of arbitrary length, we'll enable *chunking* as described in the section on pre-trained models:

```
def transcribe_speech(filepath):
    output = pipe(
        filepath,
        max_new_tokens=256,
        generate_kwargs=,  # update with the language you've fine-tuned on
        chunk_length_s=30,
        batch_size=8,
    )
    return output["text"]
```

We'll use the Gradio blocks feature to launch two tabs on our demo: one for microphone transcription, and the other for file upload.

```
demo = gr.Blocks()

mic_transcribe = gr.Interface(
```

```
    fn=transcribe_speech,
    inputs=gr.Audio(sources="microphone", type="filepath"),
    outputs=gr.components.Textbox(),
)

file_transcribe = gr.Interface(
    fn=transcribe_speech,
    inputs=gr.Audio(sources="upload", type="filepath"),
    outputs=gr.components.Textbox(),
)
```

Finally, we launch the Gradio demo using the two blocks that we've just defined:

```
with demo:
    gr.TabbedInterface(
        [mic_transcribe, file_transcribe],
        ["Transcribe Microphone", "Transcribe Audio File"],
    )

demo.launch(debug=True)
```

This will launch a Gradio demo similar to the one running on the Hugging Face Space:

Should you wish to host your demo on the Hugging Face Hub, you can use this Space as a template for your fine-tuned model.

Click the link to duplicate the template demo to your account: https://huggingface.co/spaces/course-demos/whisper-small?duplicate=true

We recommend giving your space a similar name to your fine-tuned model (e.g. whisper-small-dv-demo) and setting the visibility to "Public".

Once you've duplicated the Space to your account, click "Files and versions" -> "app.py" -> "edit". Then change the model identifier to your fine-tuned model (line 6). Scroll to the bottom of the page and click "Commit changes to main". The demo will reboot, this time using your fine-tuned model. You can share this demo with your friends and family so that they can use the model that you've trained!

Checkout our video tutorial to get a better understanding of how to duplicate the Space 👉 YouTube Video

We look forward to seeing your demos on the Hub!

Evaluation metrics for ASR

If you're familiar with the Levenshtein distance from NLP, the metrics for assessing speech recognition systems will be familiar! Don't worry if you're not, we'll go through the explanations start-to-finish to make sure you know the different metrics and understand what they mean.

When assessing speech recognition systems, we compare the system's predictions to the target text transcriptions, annotating any errors that are present. We categorise these errors into one of three categories: 1. Substitutions (S): where we transcribe the **wrong word** in our prediction ("sit" instead of "sat") 2. Insertions (I): where we add an **extra word** in our prediction 3. Deletions (D): where we **remove a word** in our prediction

These error categories are the same for all speech recognition metrics. What differs is the level at which we compute these errors: we can either compute them on the *word level* or on the *character level*.

We'll use a running example for each of the metric definitions. Here, we have a *ground truth* or *reference* text sequence:

```
reference = "the cat sat on the mat"
```

And a predicted sequence from the speech recognition system that we're trying to assess:

```
prediction = "the cat sit on the"
```

We can see that the prediction is pretty close, but some words are not quite right. We'll evaluate this prediction against the reference for the three most popular speech recognition metrics and see what sort of numbers we get for each.

Word Error Rate

The *word error rate (WER)* metric is the 'de facto' metric for speech recognition. It calculates substitutions, insertions and deletions on the *word level*. This means errors are annotated on a word-by-word basis. Take our example:

Reference:	**the**	**cat**	**sat**	**on**	**the**	**mat**
Prediction:	the	cat	**sit**	on	the	

Reference:	**the**	**cat**	**sat**	**on**	**the**	**mat**
Label:	✅	✅	S	✅	✅	D

Here, we have: * 1 substitution ("sit" instead of "sat") * 0 insertions * 1 deletion ("mat" is missing)

This gives 2 errors in total. To get our error rate, we divide the number of errors by the total number of words in our reference (N), which for this example is 6:

$$ \begin WER &= \frac \ &= \frac \ &= 0.333 \end $$

Alright! So we have a WER of 0.333, or 33.3%. Notice how the word "sit" only has one character that is wrong, but the entire word is marked incorrect. This is a defining feature of the WER: spelling errors are penalised heavily, no matter how minor they are.

The WER is defined such that *lower is better*: a lower WER means there are fewer errors in our prediction, so a perfect speech recognition system would have a WER of zero (no errors).

Let's see how we can compute the WER using 🤗 Evaluate. We'll need two packages to compute our WER metric: 🤗 Evaluate for the API interface, and JIWER to do the heavy lifting of running the calculation:

```
pip install --upgrade evaluate jiwer
```

Great! We can now load up the WER metric and compute the figure for our example:

```
from evaluate import load

wer_metric = load("wer")

wer = wer_metric.compute(references=[reference], predictions=[prediction])

print(wer)
```

Print Output:

```
0.3333333333333333
```

0.33, or 33.3%, as expected! We now know what's going on under-the-hood with this WER calculation.

Now, here's something that's quite confusing... What do you think the upper limit of the WER is? You would expect it to be 1 or 100% right? Nuh uh! Since the WER is the ratio of errors to number of words (N), there is no upper limit on the WER! Let's take an example were we predict 10 words and the target only has 2 words. If all of our predictions were wrong (10 errors), we'd have a WER of 10 / 2 = 5, or 500%! This is something to bear in mind if you train an ASR system and see a WER of over 100%. Although if you're seeing this, something has likely gone wrong... 😅

Inverse Real-Time Factor (RTFx)

While WER measures the accuracy of transcriptions, the *inverse real-time factor (RTFx)* measures the speed of an ASR system. RTFx is the inverse ratio of processing time to audio duration:

$$ \text = \frac}} $$

For example, if it takes 10 seconds to transcribe 100 seconds of audio, the RTFx is 100/10 = 10. An RTFx greater than 1.0 means the system can transcribe audio faster than real-time, which is essential for live transcription applications like video conferencing or live captioning. An RTFx of 1.0 means the system processes at exactly real-time speed, while values below 1.0 indicate slower-than-real-time processing.

Key points about RTFx: * **Higher is better**: Higher RTFx means faster processing * **RTFx > 1.0**: Faster than real-time (good for streaming applications) * **RTFx = 1.0**: Processes at exactly real-time speed * **RTFx < 1.0**: Slower than real-time (may be acceptable for batch processing)

RTFx is hardware-dependent and varies based on factors like: - Model size (larger models typically have lower RTFx) - Hardware acceleration (GPU vs CPU) - Batch size - Audio characteristics (sampling rate, number of channels)

When evaluating ASR systems, it's important to consider both WER and RTFx together. A model with excellent WER but very low RTFx may not be practical for real-time applications, while a model with slightly higher WER but high RTFx might be more suitable for latency-sensitive use cases.

Word Accuracy

We can flip the WER around to give us a metric where *higher is better*. Rather than measuring the word error rate, we can measure the *word accuracy (WAcc)* of our system:

$$ \begin WAcc = 1 - WER \nonumber \end $$

The WAcc is also measured on the word-level, it's just the WER reformulated as an accuracy metric rather than an error metric. The WAcc is very infrequently quoted in the speech literature - we think of our system predictions in terms of word errors, and so prefer error rate metrics that are more associated with these error type annotations.

Character Error Rate

It seems a bit unfair that we marked the entire word for "sit" wrong when in fact only one letter was incorrect. That's because we were evaluating our system on the word level, thereby annotating errors on a word-by-word basis. The *character error rate (CER)* assesses systems on the *character level*. This means we divide up our words into their individual characters, and annotate errors on a character-by-character basis:

Reference:	**t**	**h**	**e**		**c**	**a**	**t**		**s**	**a**	**t**		**o**	**n**		**t**	**h**	**e**		**m**	**a**
Prediction:	t	h	e		c	a	t		s	**i**	t		o	n		t	h	e			
Label:	✅	✅	✅	✅	✅	✅	✅	✅	✅	S	✅	✅	✅	✅	✅	✅	✅	✅	D	D	D

We can see now that for the word "sit", the "s" and "t" are marked as correct. It's only the "i" which is labelled as a substitution error (S). Thus, we reward our system for the partially correct prediction 🤝

In our example, we have 1 character substitution, 0 insertions, and 4 deletions. In total, we have 22 characters. So, our CER is:

$$ \begin CER &= \frac \ &= \frac \ &= 0.227 \end $$

Right! We have a CER of 0.227, or 22.7%. Notice how this is lower than our WER - we penalised the spelling error much less.

Which metric should I use?

In general, the WER is used far more than the CER for assessing speech systems. This is because the WER requires systems to have greater understanding of the context of the predictions. In our example, "sit" is in the wrong tense. A system that understands the relationship between the verb and tense of the sentence would have predicted the correct verb tense of "sat". We want to encourage this level of

understanding from our speech systems. So although the WER is less forgiving than the CER, it's also more conducive to the kinds of intelligible systems we want to develop. Therefore, we typically use the WER and would encourage you to as well! However, there are circumstances where it is not possible to use the WER. Certain languages, such as Mandarin and Japanese, have no notion of 'words', and so the WER is meaningless. Here, we revert to using the CER.

In our example, we only used one sentence when computing the WER. We would typically use an entire test set consisting of several thousand sentences when evaluating a real system. When evaluating over multiple sentences, we aggregate S, I, D and N across all sentences, and then compute the WER according to the formula defined above. This gives a better estimate of the WER for unseen data.

Normalisation

If we train an ASR model on data with punctuation and casing, it will learn to predict casing and punctuation in its transcriptions. This is great when we want to use our model for actual speech recognition applications, such as transcribing meetings or dictation, since the predicted transcriptions will be fully formatted with casing and punctuation, a style referred to as *orthographic*.

However, we also have the option of *normalising* the dataset to remove any casing and punctuation. Normalising the dataset makes the speech recognition task easier: the model no longer needs to distinguish between upper and lower case characters, or have to predict punctuation from the audio data alone (e.g. what sound does a semi-colon make?). Because of this, the word error rates are naturally lower (meaning the results are better). The Whisper paper demonstrates the drastic effect that normalising transcriptions can have on WER results (*c.f.* Section 4.4 of the Whisper paper). While we get lower WERs, the model isn't necessarily better for production. The lack of casing and punctuation makes the predicted text from the model significantly harder to read. Take the example from the previous section, where we ran Wav2Vec2 and Whisper on the same audio sample from the LibriSpeech dataset. The Wav2Vec2 model predicts neither punctuation nor casing, whereas Whisper predicts both. Comparing the transcriptions side-by-side, we see that the Whisper transcription is far easier to read:

```
Wav2Vec2:  HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH CHRISTMAUS AND
ROSE BEEF LOOMING BEFORE US SIMALYIS DRAWN FROM EATING AND ITS RESULTS OCCUR MOST
READILY TO THE MIND
Whisper:   He tells us that at this festive season of the year, with Christmas and
```

```
roast beef looming before us, similarly is drawn from eating and its results occur
most readily to the mind.
```

The Whisper transcription is orthographic and thus ready to go - it's formatted as we'd expect for a meeting transcription or dictation script with both punctuation and casing. On the contrary, we would need to use additional post-processing to restore punctuation and casing in our Wav2Vec2 predictions if we wanted to use it for downstream applications.

There is a happy medium between normalising and not normalising: we can train our systems on orthographic transcriptions, and then normalise the predictions and targets before computing the WER. This way, we train our systems to predict fully formatted text, but also benefit from the WER improvements we get by normalising the transcriptions.

The Whisper model was released with a normaliser that effectively handles the normalisation of casing, punctuation and number formatting among others. Let's apply the normaliser to the Whisper transcriptions to demonstrate how we can normalise them:

```
from transformers.models.whisper.english_normalizer import BasicTextNormalizer

normalizer = BasicTextNormalizer()

prediction = " He tells us that at this festive season of the year, with Christmas
and roast beef looming before us, similarly is drawn from eating and its results
occur most readily to the mind."
normalized_prediction = normalizer(prediction)

normalized_prediction
```

Output:

```
' he tells us that at this festive season of the year with christmas and roast beef
looming before us similarly is drawn from eating and its results occur most readily
to the mind '
```

Great! We can see that the text has been fully lower-cased and all punctuation removed. Let's now define the reference transcription and then compute the normalised WER between the reference and prediction:

```
reference = "HE TELLS US THAT AT THIS FESTIVE SEASON OF THE YEAR WITH CHRISTMAS AND
ROAST BEEF LOOMING BEFORE US SIMILES DRAWN FROM EATING AND ITS RESULTS OCCUR MOST
READILY TO THE MIND"
normalized_referece = normalizer(reference)
```

```
wer = wer_metric.compute(
    references=[normalized_referece], predictions=[normalized_prediction]
)
wer
```

Output:

```
0.0625
```

6.25% - that's about what we'd expect for the Whisper base model on the LibriSpeech validation set. As we see here, we've predicted an orthographic transcription, but benefited from the WER boost obtained by normalising the reference and prediction prior to computing the WER.

The choice of how you normalise the transcriptions is ultimately down to your needs. We recommend training on orthographic text and evaluating on normalised text to get the best of both worlds.

Putting it all together

Alright! We've covered three topics so far in this Unit: pre-trained models, dataset selection and evaluation. Let's have some fun and put them together in one end-to-end example 🚀 We're going to set ourselves up for the next section on fine-tuning by evaluating the pre-trained Whisper model on the Common Voice 13 Dhivehi test set. We'll use the WER number we get as a *baseline* for our fine-tuning run, or a target number that we'll try and beat 🥊

First, we'll load the pre-trained Whisper model using the `pipeline()` class. This process will be extremely familiar by now! The only new thing we'll do is load the model in half-precision (float16) if running on a GPU - this will speed up inference at almost no cost to WER accuracy.

```
from transformers import pipeline


if torch.cuda.is_available():
    device = "cuda:0"
    torch_dtype = torch.float16
else:
    device = "cpu"
    torch_dtype = torch.float32

pipe = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-small",
```

```
    torch_dtype=torch_dtype,
    device=device,
)
```

Next, we'll load the Dhivehi test split of Common Voice 13. You'll remember from the previous section that the Common Voice 13 is *gated*, meaning we had to agree to the dataset terms of use before gaining access to the dataset. We can now link our Hugging Face account to our notebook, so that we have access to the dataset from the machine we're currently using.

Linking the notebook to the Hub is straightforward - it simply requires entering your Hub authentication token when prompted. Find your Hub authentication token here and enter it when prompted:

```
from huggingface_hub import notebook_login

notebook_login()
```

Great! Once we've linked the notebook to our Hugging Face account, we can proceed with downloading the Common Voice dataset. This will take a few minutes to download and pre-process, fetching the data from the Hugging Face Hub and preparing it automatically on your notebook:

```
from datasets import load_dataset

common_voice_test = load_dataset(
    "mozilla-foundation/common_voice_13_0", "dv", split="test"
)
```

If you face an authentication issue when loading the dataset, ensure that you have accepted the dataset's terms of use on the Hugging Face Hub through the following link: https://huggingface.co/datasets/mozilla-foundation/common_voice_13_0

Evaluating over an entire dataset can be done in much the same way as over a single example - all we have to do is **loop** over the input audios, rather than inferring just a single sample. To do this, we first transform our dataset into a `KeyDataset`. All this does is pick out the particular dataset column that we want to forward to the model (in our case, that's the `"audio"` column), ignoring the rest (like the target transcriptions, which we don't want to use for inference). We then iterate over this transformed datasets, appending the model outputs to a list to save the predictions. The following code cell will take approximately five minutes if running on a GPU with half-precision, peaking at 12GB memory:

```
from tqdm import tqdm
from transformers.pipelines.pt_utils import KeyDataset

all_predictions = []

# run streamed inference
for prediction in tqdm(
    pipe(
        KeyDataset(common_voice_test, "audio"),
        max_new_tokens=128,
        generate_kwargs=,
        batch_size=32,
    ),
    total=len(common_voice_test),
):
    all_predictions.append(prediction["text"])
```

If you experience a CUDA out-of-memory (OOM) when running the above cell, incrementally reduce the `batch_size` by factors of 2 until you find a batch size that fits your device.

And finally, we can compute the WER. Let's first compute the orthographic WER, i.e. the WER without any post-processing:

```
from evaluate import load

wer_metric = load("wer")

wer_ortho = 100 * wer_metric.compute(
    references=common_voice_test["sentence"], predictions=all_predictions
)
wer_ortho
```

Output:

```
167.29577268612022
```

Okay... 167% essentially means our model is outputting garbage 😜 Not to worry, it'll be our aim to improve this by fine-tuning the model on the Dhivehi training set!

Next, we'll evaluate the normalised WER, i.e. the WER with normalisation post-processing. We have to filter out samples that would be empty after normalisation, as otherwise the total number of words in our reference (N) would be zero, which would give a division by zero error in our calculation:

```
from transformers.models.whisper.english_normalizer import BasicTextNormalizer
```

```
normalizer = BasicTextNormalizer()

# compute normalised WER
all_predictions_norm = [normalizer(pred) for pred in all_predictions]
all_references_norm = [normalizer(label) for label in
common_voice_test["sentence"]]

# filtering step to only evaluate the samples that correspond to non-zero
references
all_predictions_norm = [
    all_predictions_norm[i]
    for i in range(len(all_predictions_norm))
    if len(all_references_norm[i]) > 0
]
all_references_norm = [
    all_references_norm[i]
    for i in range(len(all_references_norm))
    if len(all_references_norm[i]) > 0
]

wer = 100 * wer_metric.compute(
    references=all_references_norm, predictions=all_predictions_norm
)

wer
```

Output:

```
125.69809089960707
```

Again we see the drastic reduction in WER we achieve by normalising our references and predictions: the baseline model achieves an orthographic test WER of 168%, while the normalised WER is 126%.

Right then! These are the numbers that we want to try and beat when we fine-tune the model, in order to improve the Whisper model for Dhivehi speech recognition. Continue reading to get hands-on with a fine-tuning example 🚀

Fine-tuning the ASR model

In this section, we'll cover a step-by-step guide on fine-tuning Whisper for speech recognition on the Common Voice 13 dataset. We'll use the 'small' version of the model and a relatively lightweight dataset, enabling you to run fine-tuning fairly quickly on any 16GB+ GPU with low disk space requirements, such as the 16GB T4 GPU provided in the Google Colab free tier.

Should you have a smaller GPU or encounter memory issues during training, you can follow the suggestions provided for reducing memory usage. Conversely, should you have access to a larger GPU, you can amend the training arguments to maximise your throughput. Thus, this guide is accessible regardless of your GPU specifications!

Likewise, this guide outlines how to fine-tune the Whisper model for the Dhivehi language. However, the steps covered here generalise to any language in the Common Voice dataset, and more generally to any ASR dataset on the Hugging Face Hub. You can tweak the code to quickly switch to a language of your choice and fine-tune a Whisper model in your native tongue 🌍

Right! Now that's out the way, let's get started and kick-off our fine-tuning pipeline!

Prepare Environment

We strongly advise you to upload model checkpoints directly the Hugging Face Hub while training. The Hub provides: - Integrated version control: you can be sure that no model checkpoint is lost during training. - Tensorboard logs: track important metrics over the course of training. - Model cards: document what a model does and its intended use cases. - Community: an easy way to share and collaborate with the community! 🤗

Linking the notebook to the Hub is straightforward - it simply requires entering your Hub authentication token when prompted. Find your Hub authentication token here and enter it when prompted:

```
from huggingface_hub import notebook_login

notebook_login()
```

Output:

```
Login successful
Your token has been saved to /root/.huggingface/token
```

Load Dataset

Common Voice 13 contains approximately ten hours of labelled Dhivehi data, three of which is held-out test data. This is extremely little data for fine-tuning, so we'll be

relying on leveraging the extensive multilingual ASR knowledge acquired by Whisper during pre-training for the low-resource Dhivehi language.

Using 🤗 Datasets, downloading and preparing data is extremely simple. We can download and prepare the Common Voice 13 splits in just one line of code. Since Dhivehi is very low-resource, we'll combine the `train` and `validation` splits to give approximately seven hours of training data. We'll use the three hours of `test` data as our held-out test set:

```
from datasets import load_dataset, DatasetDict

common_voice = DatasetDict()

common_voice["train"] = load_dataset(
    "mozilla-foundation/common_voice_13_0", "dv", split="train+validation"
)
common_voice["test"] = load_dataset(
    "mozilla-foundation/common_voice_13_0", "dv", split="test"
)

print(common_voice)
```

Output:

```
DatasetDict()
    test: Dataset()
})
```

You can change the language identifier from `"dv"` to a language identifier of your choice. To see all possible languages in Common Voice 13, check out the dataset card on the Hugging Face Hub: https://huggingface.co/datasets/mozilla-foundation/common_voice_13_0

Most ASR datasets only provide input audio samples (`audio`) and the corresponding transcribed text (`sentence`). Common Voice contains additional metadata information, such as `accent` and `locale`, which we can disregard for ASR. Keeping the notebook as general as possible, we only consider the input audio and transcribed text for fine-tuning, discarding the additional metadata information:

```
common_voice = common_voice.select_columns(["audio", "sentence"])
```

Feature Extractor, Tokenizer and Processor

The ASR pipeline can be de-composed into three stages:

1. The feature extractor which pre-processes the raw audio-inputs to log-mel spectrograms
2. The model which performs the sequence-to-sequence mapping
3. The tokenizer which post-processes the predicted tokens to text

In 🤗 Transformers, the Whisper model has an associated feature extractor and tokenizer, called WhisperFeatureExtractor and WhisperTokenizer respectively. To make our lives simple, these two objects are wrapped under a single class, called the WhisperProcessor. We can call the WhisperProcessor to perform both the audio pre-processing and the text token post-processing. In doing so, we only need to keep track of two objects during training: the processor and the model.

When performing multilingual fine-tuning, we need to set the `"language"` and `"task"` when instantiating the processor. The `"language"` should be set to the source audio language, and the task to `"transcribe"` for speech recognition or `"translate"` for speech translation. These arguments modify the behaviour of the tokenizer, and should be set correctly to ensure the target labels are encoded properly.

We can see all possible languages supported by Whisper by importing the list of languages:

```
from transformers.models.whisper.tokenization_whisper import TO_LANGUAGE_CODE

TO_LANGUAGE_CODE
```

If you scroll through this list, you'll notice that many languages are present, but Dhivehi is one of few that is not! This means that Whisper was not pre-trained on Dhivehi. However, this doesn't mean that we can't fine tune Whisper on it. In doing so, we'll be teaching Whisper a new language, one that the pre-trained checkpoint does not support. That's pretty cool, right!

When you fine-tune it on a new language, Whisper does a good job at leveraging its knowledge of the other 96 languages it’s pre-trained on. Largely speaking, all modern languages will be linguistically similar to at least one of the 96 languages Whisper already knows, so we'll fall under this paradigm of cross-lingual knowledge representation.

What we need to do to fine-tune Whisper on a new language is find the language **most similar** that Whisper was pre-trained on. The Wikipedia article for Dhivehi

states that Dhivehi is closely related to the Sinhalese language of Sri Lanka. If we check the language codes again, we can see that Sinhalese is present in the Whisper language set, so we can safely set our language argument to `"sinhalese"`.

Right! We'll load our processor from the pre-trained checkpoint, setting the language to `"sinhalese"` and task to `"transcribe"` as explained above:

```
from transformers import WhisperProcessor

processor = WhisperProcessor.from_pretrained(
    "openai/whisper-small", language="sinhalese", task="transcribe"
)
```

It's worth reiterating that in most circumstances, you'll find that the language you want to fine-tune on is in the set of pre-training languages, in which case you can simply set the language directly as your source audio language! Note that both of these arguments should be omitted for English-only fine-tuning, where there is only one option for the language (`"English"`) and task (`"transcribe"`).

Pre-Process the Data

Let's have a look at the dataset features. Pay particular attention to the `"audio"` column - this details the sampling rate of our audio inputs:

```
common_voice["train"].features
```

Output:

```

```

Since our input audio is sampled at 48kHz, we need to *downsample* it to 16kHz prior to passing it to the Whisper feature extractor, 16kHz being the sampling rate expected by the Whisper model.

We'll set the audio inputs to the correct sampling rate using dataset's `cast_column` method. This operation does not change the audio in-place, but rather signals to datasets to resample audio samples on-the-fly when they are loaded:

```
from datasets import Audio

sampling_rate = processor.feature_extractor.sampling_rate
common_voice = common_voice.cast_column("audio",
Audio(sampling_rate=sampling_rate))
```

Now we can write a function to prepare our data ready for the model:

1. We load and resample the audio data on a sample-by-sample basis by calling `sample["audio"]`. As explained above, 🤗 Datasets performs any necessary resampling operations on the fly.
2. We use the feature extractor to compute the log-mel spectrogram input features from our 1-dimensional audio array.
3. We encode the transcriptions to label ids through the use of the tokenizer.

```
def prepare_dataset(example):
    audio = example["audio"]

    example = processor(
        audio=audio["array"],
        sampling_rate=audio["sampling_rate"],
        text=example["sentence"],
    )

    # compute input length of audio sample in seconds
    example["input_length"] = len(audio["array"]) / audio["sampling_rate"]

    return example
```

We can apply the data preparation function to all of our training examples using 🤗 Datasets' `.map` method. We'll remove the columns from the raw training data (the audio and text), leaving just the columns returned by the `prepare_dataset` function:

```
common_voice = common_voice.map(
    prepare_dataset, remove_columns=common_voice.column_names["train"], num_proc=1
)
```

Finally, we filter any training data with audio samples longer than 30s. These samples would otherwise be truncated by the Whisper feature-extractor which could affect the stability of training. We define a function that returns `True` for samples that are less than 30s, and `False` for those that are longer:

```
max_input_length = 30.0


def is_audio_in_length_range(length):
    return length < max_input_length
```

We apply our filter function to all samples of our training dataset through 🤗 Datasets' `.filter` method:

```
common_voice["train"] = common_voice["train"].filter(
    is_audio_in_length_range,
    input_columns=["input_length"],
)
```

Let's check how much training data we removed through this filtering step:

```
common_voice["train"]
```

Output

```
Dataset()
```

Alright! In this case we actually have the same number of samples as before, so there were no samples longer than 30s. This might not be the case if you switch languages, so it's best to keep this filter step in-place for robustness. With that, we have our data fully prepared for training! Let's continue and take a look at how we can use this data to fine-tune Whisper.

Training and Evaluation

Now that we've prepared our data, we're ready to dive into the training pipeline. The 🤗 Trainer will do much of the heavy lifting for us. All we have to do is:

- Define a data collator: the data collator takes our pre-processed data and prepares PyTorch tensors ready for the model.
- Evaluation metrics: during evaluation, we want to evaluate the model using the word error rate (WER) metric. We need to define a `compute_metrics` function that handles this computation.
- Load a pre-trained checkpoint: we need to load a pre-trained checkpoint and configure it correctly for training.
- Define the training arguments: these will be used by the 🤗 Trainer in constructing the training schedule.

Once we've fine-tuned the model, we will evaluate it on the test data to verify that we have correctly trained it to transcribe speech in Dhivehi.

Define a Data Collator

The data collator for a sequence-to-sequence speech model is unique in the sense that it treats the `input_features` and `labels` independently: the `input_features` must be handled by the feature extractor and the `labels` by the tokenizer.

The `input_features` are already padded to 30s and converted to a log-Mel spectrogram of fixed dimension, so all we have to do is convert them to batched PyTorch tensors. We do this using the feature extractor's `.pad` method with `return_tensors=pt`. Note that no additional padding is applied here since the inputs are of fixed dimension, the `input_features` are simply converted to PyTorch tensors.

On the other hand, the `labels` are un-padded. We first pad the sequences to the maximum length in the batch using the tokenizer's `.pad` method. The padding tokens are then replaced by `-100` so that these tokens are **not** taken into account when computing the loss. We then cut the start of transcript token from the beginning of the label sequence as we append it later during training.

We can leverage the `WhisperProcessor` we defined earlier to perform both the feature extractor and the tokenizer operations:

```
from dataclasses import dataclass
from typing import Any, Dict, List, Union


@dataclass
class DataCollatorSpeechSeq2SeqWithPadding:
    processor: Any

    def __call__(
        self, features: List[Dict[str, Union[List[int], torch.Tensor]]]
    ) -> Dict[str, torch.Tensor]:
        # split inputs and labels since they have to be of different lengths and
need different padding methods
        # first treat the audio inputs by simply returning torch tensors
        input_features = [
             for feature in features
        ]
        batch = self.processor.feature_extractor.pad(input_features,
return_tensors="pt")

        # get the tokenized label sequences
        label_features = [ for feature in features]
        # pad the labels to max length
        labels_batch = self.processor.tokenizer.pad(label_features,
return_tensors="pt")

        # replace padding with -100 to ignore loss correctly
        labels = labels_batch["input_ids"].masked_fill(
```

```
            labels_batch.attention_mask.ne(1), -100
        )

        # if bos token is appended in previous tokenization step,
        # cut bos token here as it's append later anyways
        if (labels[:, 0] ==
self.processor.tokenizer.bos_token_id).all().cpu().item():
            labels = labels[:, 1:]

        batch["labels"] = labels

        return batch
```

We can now initialise the data collator we've just defined:

```
data_collator = DataCollatorSpeechSeq2SeqWithPadding(processor=processor)
```

Onwards!

Evaluation Metrics

Next, we define the evaluation metric we'll use on our evaluation set. We'll use the Word Error Rate (WER) metric introduced in the section on Evaluation, the 'de-facto' metric for assessing ASR systems.

We'll load the WER metric from 🤗 Evaluate:

```
metric = evaluate.load("wer")
```

We then simply have to define a function that takes our model predictions and returns the WER metric. This function, called `compute_metrics`, first replaces `-100` with the `pad_token_id` in the `label_ids` (undoing the step we applied in the data collator to ignore padded tokens correctly in the loss). It then decodes the predicted and label ids to strings. Finally, it computes the WER between the predictions and reference labels. Here, we have the option of evaluating with the 'normalised' transcriptions and predictions, which have punctuation and casing removed. We recommend you follow this to benefit from the WER improvement obtained by normalising the transcriptions.

```
from transformers.models.whisper.english_normalizer import BasicTextNormalizer

normalizer = BasicTextNormalizer()


def compute_metrics(pred):
    pred_ids = pred.predictions
```

```
    label_ids = pred.label_ids

    # replace -100 with the pad_token_id
    label_ids[label_ids == -100] = processor.tokenizer.pad_token_id

    # we do not want to group tokens when computing the metrics
    pred_str = processor.batch_decode(pred_ids, skip_special_tokens=True)
    label_str = processor.batch_decode(label_ids, skip_special_tokens=True)

    # compute orthographic wer
    wer_ortho = 100 * metric.compute(predictions=pred_str, references=label_str)

    # compute normalised WER
    pred_str_norm = [normalizer(pred) for pred in pred_str]
    label_str_norm = [normalizer(label) for label in label_str]
    # filtering step to only evaluate the samples that correspond to non-zero
references:
    pred_str_norm = [
        pred_str_norm[i] for i in range(len(pred_str_norm)) if
len(label_str_norm[i]) > 0
    ]
    label_str_norm = [
        label_str_norm[i]
        for i in range(len(label_str_norm))
        if len(label_str_norm[i]) > 0
    ]

    wer = 100 * metric.compute(predictions=pred_str_norm,
references=label_str_norm)

    return
```

Load a Pre-Trained Checkpoint

Now let's load the pre-trained Whisper small checkpoint. Again, this is trivial through use of 🤗 Transformers!

```
from transformers import WhisperForConditionalGeneration

model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-small")
```

We'll set `use_cache` to `False` for training since we're using gradient checkpointing and the two are incompatible. We'll also override two generation arguments to control the behaviour of the model during inference: we'll force the language and task tokens during generation by setting the `language` and `task` arguments, and also re-enable cache for generation to speed-up inference time:

```
from functools import partial
```

```
# disable cache during training since it's incompatible with gradient checkpointing
model.config.use_cache = False

# set language and task for generation and re-enable cache
model.generate = partial(
    model.generate, language="sinhalese", task="transcribe", use_cache=True
)
```

Define the Training Configuration

In the final step, we define all the parameters related to training. Here, we set the number of training steps to 500. This is enough steps to see a big WER improvement compared to the pre-trained Whisper model, while ensuring that fine-tuning can be run in approximately 45 minutes on a Google Colab free tier. For more detail on the training arguments, refer to the Seq2SeqTrainingArguments docs.

```
from transformers import Seq2SeqTrainingArguments

training_args = Seq2SeqTrainingArguments(
    output_dir="./whisper-small-dv",  # name on the HF Hub
    per_device_train_batch_size=16,
    gradient_accumulation_steps=1,  # increase by 2x for every 2x decrease in batch
size
    learning_rate=1e-5,
    lr_scheduler_type="constant_with_warmup",
    warmup_steps=50,
    max_steps=500,  # increase to 4000 if you have your own GPU or a Colab paid
plan
    gradient_checkpointing=True,
    fp16=True,
    fp16_full_eval=True,
    evaluation_strategy="steps",
    per_device_eval_batch_size=16,
    predict_with_generate=True,
    generation_max_length=225,
    save_steps=500,
    eval_steps=500,
    logging_steps=25,
    report_to=["tensorboard"],
    load_best_model_at_end=True,
    metric_for_best_model="wer",
    greater_is_better=False,
    push_to_hub=True,
)
```

If you do not want to upload the model checkpoints to the Hub, set `push_to_hub=False`.

We can forward the training arguments to the 🤗 Trainer along with our model, dataset, data collator and `compute_metrics` function:

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
    args=training_args,
    model=model,
    train_dataset=common_voice["train"],
    eval_dataset=common_voice["test"],
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    tokenizer=processor,
)
```

And with that, we're ready to start training!

Training

To launch training, simply execute:

```
trainer.train()
```

Training will take approximately 45 minutes depending on your GPU or the one allocated to the Google Colab. Depending on your GPU, it is possible that you will encounter a CUDA `"out-of-memory"` error when you start training. In this case, you can reduce the `per_device_train_batch_size` incrementally by factors of 2 and employ `gradient_accumulation_steps` to compensate.

Output:

Training Loss	Epoch	Step	Validation Loss	Wer Ortho	Wer
0.136	1.63	500	0.1727	63.8972	14.0661

Our final WER is 14.1% - not bad for seven hours of training data and just 500 training steps! That amounts to a 112% improvement versus the pre-trained model! That means we've taken a model that previously had no knowledge about Dhivehi, and fine-tuned it to recognise Dhivehi speech with adequate accuracy in under one hour 🤯

The big question is how this compares to other ASR systems. For that, we can view the autoevaluate leaderboard, a leaderboard that categorises models by language and dataset, and subsequently ranks them according to their WER.

Looking at the leaderboard, we see that our model trained for 500 steps convincingly beats the pre-trained Whisper Small checkpoint that we evaluated in the previous section. Nice job 👏

We see that there are a few checkpoints that do better than the one we trained. The beauty of the Hugging Face Hub is that it's a *collaborative* platform - if we don't have the time or resources to perform a longer training run ourselves, we can load a checkpoint that someone else in the community has trained and been kind enough to share (making sure to thank them for it!). You'll be able to load these checkpoints in exactly the same way as the pre-trained ones using the `pipeline` class as we did previously! So there's nothing stopping you cherry-picking the best model on the leaderboard to use for your task!

We can automatically submit our checkpoint to the leaderboard when we push the training results to the Hub - we simply have to set the appropriate key-word arguments (kwargs). You can change these values to match your dataset, language and model name accordingly:

```
kwargs =
```

The training results can now be uploaded to the Hub. To do so, execute the `push_to_hub` command:

```
trainer.push_to_hub(**kwargs)
```

This will save the training logs and model weights under `"your-username/the-name-you-picked"`. For this example, check out the upload at `sanchit-gandhi/whisper-small-dv`.

While the fine-tuned model yields satisfactory results on the Common Voice 13 Dhivehi test data, it is by no means optimal. The purpose of this guide is to demonstrate how to fine-tune an ASR model using the 🤗 Trainer for multilingual speech recognition.

If you have access to your own GPU or are subscribed to a Google Colab paid plan, you can increase `max_steps` to 4000 steps to improve the WER further by training for more steps. Training for 4000 steps will take approximately 3-5 hours depending on your GPU and yield WER results approximately 3% lower than training for 500 steps. If you decide to train for 4000 steps, we also recommend changing the learning rate scheduler to a *linear* schedule (set `lr_scheduler_type="linear"`), as this will yield an additional performance boost over long training runs.

The results could likely be improved further by optimising the training hyperparameters, such as *learning rate* and *dropout*, and using a larger pre-trained checkpoint (`medium` or `large`). We leave this as an exercise to the reader.

Sharing Your Model

You can now share this model with anyone using the link on the Hub. They can load it with the identifier `"your-username/the-name-you-picked"` directly into the `pipeline()` object. For instance, to load the fine-tuned checkpoint "sanchit-gandhi/whisper-small-dv":

```
from transformers import pipeline

pipe = pipeline("automatic-speech-recognition", model="sanchit-gandhi/whisper-small-dv")
```

Conclusion

In this section, we covered a step-by-step guide on fine-tuning the Whisper model for speech recognition 🤗 Datasets, Transformers and the Hugging Face Hub. We first loaded the Dhivehi subset of the Common Voice 13 dataset and pre-processed it by computing log-mel spectrograms and tokenising the text. We then defined a data collator, evaluation metric and training arguments, before using the 🤗 Trainer to train and evaluate our model. We finished by uploading the fine-tuned model to the Hugging Face Hub, and showcased how to share and use it with the `pipeline()` class.

If you followed through to this point, you should now have a fine-tuned checkpoint for speech recognition, well done! 🥳 Even more importantly, you're equipped with all the tools you need to fine-tune the Whisper model on any speech recognition dataset or domain. So what are you waiting for! Pick one of the datasets covered in the section Choosing a Dataset or select a dataset of your own, and see whether you can get state-of-the-art performance! The leaderboard is waiting for you...

Hands-on exercise

In this unit, we explored the challenges of fine-tuning ASR models, acknowledging the time and resources required to fine-tune a model like Whisper (even a small checkpoint) on a new language. To provide a hands-on experience, we have

designed an exercise that allows you to navigate the process of fine-tuning an ASR model while using a smaller dataset. The main goal of this exercise is to familiarize you with the process rather than expecting production-level results. We have intentionally set a low metric to ensure that even with limited resources, you should be able to achieve it.

Here are the instructions: * Fine-tune the `"openai/whisper-tiny"` model using the American English ("en-US") subset of the `"PolyAI/minds14"` dataset. * Use the first **450 examples for training**, and the rest for evaluation. Ensure you set `num_proc=1` when pre-processing the dataset using the `.map` method (this will ensure your model is submitted correctly for assessment). * To evaluate the model, use the `wer` and `wer_ortho` metrics as described in this Unit. However, *do not* convert the metric into percentages by multiplying by 100 (E.g. if WER is 42%, we'll expect to see the value of 0.42 in this exercise).

Once you have fine-tuned a model, make sure to upload it to the 🤗 Hub with the following `kwargs`:

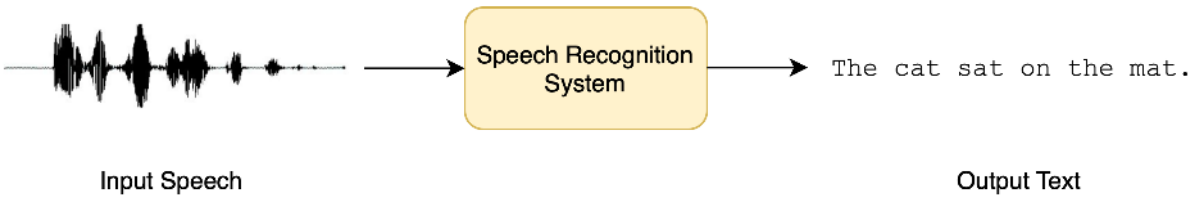
```
kwargs =
```

You will pass this assignment if your model's normalised WER (`wer`) is lower than **0.37**.

Feel free to build a demo of your model, and share it on Discord! If you have questions, post them in the #audio-study-group channel.

What you'll learn and what you'll build

In this section, we'll take a look at how Transformers can be used to convert spoken speech into text, a task known *speech recognition*.



Speech recognition, also known as automatic speech recognition (ASR) or speech-to-text (STT), is one of the most popular and exciting spoken language processing

tasks. It’s used in a wide range of applications, including dictation, voice assistants, video captioning and meeting transcriptions.

You’ve probably made use of a speech recognition system many times before without realising! Consider the digital assistant in your smartphone device (Siri, Google Assistant, Alexa). When you use these assistants, the first thing that they do is transcribe your spoken speech to written text, ready to be used for any downstream tasks (such as finding you the weather 🌤️).

Have a play with the speech recognition demo below. You can either record yourself using your microphone, or drag and drop an audio sample for transcription:

Speech recognition is a challenging task as it requires joint knowledge of audio and text. The input audio might have lots of background noise and be spoken by speakers with different accents, making it difficult to pick out the spoken speech. The written text might have characters which don’t have an acoustic sound, such as punctuation, which are difficult to infer from audio alone. These are all hurdles we have to tackle when building effective speech recognition systems!

Now that we’ve defined our task, we can begin looking into speech recognition in more detail. By the end of this Unit, you'll have a good fundamental understanding of the different pre-trained speech recognition models available and how to use them with the 🤗 Transformers library. You'll also know the procedure for fine-tuning an ASR model on a domain or language of choice, enabling you to build a performant system for whatever task you encounter. You'll be able to showcase your model to your friends and family by building a live demo, one that takes any spoken speech and converts it to text!

Specifically, we’ll cover:

- Pre-trained models for speech recognition
- Choosing a dataset
- Evaluation and metrics for speech recognition
- How to fine-tune an ASR system with the Trainer API
- Building a demo
- Hands-on exercise

Supplemental reading and resources

This unit provided a hands-on introduction to speech recognition, one of the most popular tasks in the audio domain. Want to learn more? Here you will find additional resources that will help you deepen your understanding of the topics and enhance your learning experience.

- Whisper Talk by Jong Wook Kim: a presentation on the Whisper model, explaining the motivation, architecture, training and results, delivered by Whisper author Jong Wook Kim
- End-to-End Speech Benchmark (ESB): a paper that comprehensively argues for using the orthographic WER as opposed to the normalised WER for evaluating ASR systems and presents an accompanying benchmark
- Fine-Tuning Whisper for Multilingual ASR: an in-depth blog post that explains how the Whisper model works in more detail, and the pre- and post-processing steps involved with the feature extractor and tokenizer
- Fine-tuning MMS Adapter Models for Multi-Lingual ASR: an end-to-end guide for fine-tuning Meta AI's new MMS speech recognition models, freezing the base model weights and only fine-tuning a small number of *adapter* layers
- Boosting Wav2Vec2 with n-grams in 🤗 Transformers: a blog post for combining CTC models with external language models (LMs) to combat spelling and punctuation errors

Evaluating text-to-speech models

During the training time, text-to-speech models optimize for the mean-square error loss (or mean absolute error) between the predicted spectrogram values and the generated ones. Both MSE and MAE encourage the model to minimize the difference between the predicted and target spectrograms. However, since TTS is a one-to-many mapping problem, i.e. the output spectrogram for a given text can be represented in many different ways, the evaluation of the resulting text-to-speech (TTS) models is much more difficult.

Unlike many other computational tasks that can be objectively measured using quantitative metrics, such as accuracy or precision, evaluating TTS relies heavily on subjective human analysis.

One of the most commonly employed evaluation methods for TTS systems is conducting qualitative assessments using mean opinion scores (MOS). MOS is a

subjective scoring system that allows human evaluators to rate the perceived quality of synthesized speech on a scale from 1 to 5. These scores are typically gathered through listening tests, where human participants listen to and rate the synthesized speech samples.

One of the main reasons why objective metrics are challenging to develop for TTS evaluation is the subjective nature of speech perception. Human listeners have diverse preferences and sensitivities to various aspects of speech, including pronunciation, intonation, naturalness, and clarity. Capturing these perceptual nuances with a single numerical value is a daunting task. At the same time, the subjectivity of the human evaluation makes it challenging to compare and benchmark different TTS systems.

Furthermore, this kind of evaluation may overlook certain important aspects of speech synthesis, such as naturalness, expressiveness, and emotional impact. These qualities are difficult to quantify objectively but are highly relevant in applications where the synthesized speech needs to convey human-like qualities and evoke appropriate emotional responses.

In summary, evaluating text-to-speech models is a complex task due to the absence of one truly objective metric. The most common evaluation method, mean opinion scores (MOS), relies on subjective human analysis. While MOS provides valuable insights into the quality of synthesized speech, it also introduces variability and subjectivity.

Fine-tuning SpeechT5

Now that you are familiar with the text-to-speech task and internal workings of the SpeechT5 model that was pre-trained on English language data, let's see how we can fine-tune it to another language.

House-keeping

Make sure that you have a GPU if you want to reproduce this example. In a notebook, you can check with the following command:

```
nvidia-smi
```

In our example we will be using approximately 40 hours of training data. If you'd like to follow along using the Google Colab free tier GPU, you will need to reduce the amount of training data to approximately 10-15 hours, and reduce the number of training steps.

You'll also need some additional dependencies:

```
pip install transformers datasets soundfile speechbrain accelerate
```

Finally, don't forget to log in to your Hugging Face account so that you could upload and share your model with the community:

```
from huggingface_hub import notebook_login

notebook_login()
```

The dataset

For this example we'll take the Dutch (`nl`) language subset of the VoxPopuli dataset. VoxPopuli is a large-scale multilingual speech corpus consisting of data sourced from 2009-2020 European Parliament event recordings. It contains labelled audio-transcription data for 15 European languages. While we will be using the Dutch language subset, feel free to pick another subset.

This is an automated speech recognition (ASR) dataset, so, as mentioned before, it is not the most suitable option for training TTS models. However, it will be good enough for this exercise.

Let's load the data:

```
from datasets import load_dataset, Audio

dataset = load_dataset("qmeeus/voxpopuli", "nl", split="train")
len(dataset)
```

Output:

```
20968
```

20968 examples should be sufficient for fine-tuning. SpeechT5 expects audio data to have a sampling rate of 16 kHz, so make sure the examples in the dataset meet this requirement:

```
dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))
```

Preprocessing the data

Let's begin by defining the model checkpoint to use and loading the appropriate processor that contains both tokenizer, and feature extractor that we will need to prepare the data for training:

```
from transformers import SpeechT5Processor

checkpoint = "microsoft/speecht5_tts"
processor = SpeechT5Processor.from_pretrained(checkpoint)
```

Text cleanup for SpeechT5 tokenization

First, for preparing the text, we'll need the tokenizer part of the processor, so let's get it:

```
tokenizer = processor.tokenizer
```

Let's take a look at an example:

```
dataset[0]
```

Output:

```
,
 'text': 'dat kan naar mijn gevoel alleen met een brede meerderheid die wij samen
zoeken.',
 'gender': 'female',
 'speaker_id': '1122',
 'is_gold_transcript': True,
 'accent': 'None'}
```

The dataset examples contain a `text` feature which we'll use as the text input. It's important to know that the SpeechT5 tokenizer doesn't have any tokens for numbers, so if your text contains numbers, you may need to write them out as text.

Because SpeechT5 was trained on the English language, it may not recognize certain characters in the Dutch dataset. If left as is, these characters will be converted to `<unk>` tokens. However, in Dutch, certain characters like `à` are used to

stress syllables. In order to preserve the meaning of the text, we can replace this character with a regular `a`.

To identify unsupported tokens, extract all unique characters in the dataset using the `SpeechT5Tokenizer` which works with characters as tokens. To do this, we'll write the `extract_all_chars` mapping function that concatenates the transcriptions from all examples into one string and converts it to a set of characters. Make sure to set `batched=True` and `batch_size=-1` in `dataset.map()` so that all transcriptions are available at once for the mapping function.

```
def extract_all_chars(batch):
    all_text = " ".join(batch["text"])
    vocab = list(set(all_text))
    return


vocabs = dataset.map(
    extract_all_chars,
    batched=True,
    batch_size=-1,
    keep_in_memory=True,
    remove_columns=dataset.column_names,
)

dataset_vocab = set(vocabs["vocab"][0])
tokenizer_vocab =
```

Now you have two sets of characters: one with the vocabulary from the dataset and one with the vocabulary from the tokenizer. To identify any unsupported characters in the dataset, you can take the difference between these two sets. The resulting set will contain the characters that are in the dataset but not in the tokenizer.

```
dataset_vocab - tokenizer_vocab
```

Output:

```

```

To handle the unsupported characters identified in the previous step, we can define a function that maps these characters to valid tokens. Note that spaces are already replaced by `_` in the tokenizer and don't need to be handled separately.

```
replacements = [
    ("à", "a"),
    ("ç", "c"),
    ("è", "e"),
```

```
    ("ë", "e"),
    ("í", "i"),
    ("ï", "i"),
    ("ö", "o"),
    ("ü", "u"),
]


def cleanup_text(inputs):
    for src, dst in replacements:
        inputs["text"] = inputs["text"].replace(src, dst)
    return inputs


dataset = dataset.map(cleanup_text)
```

Now that we have dealt with special characters in the text, it's time to shift the focus to the audio data.

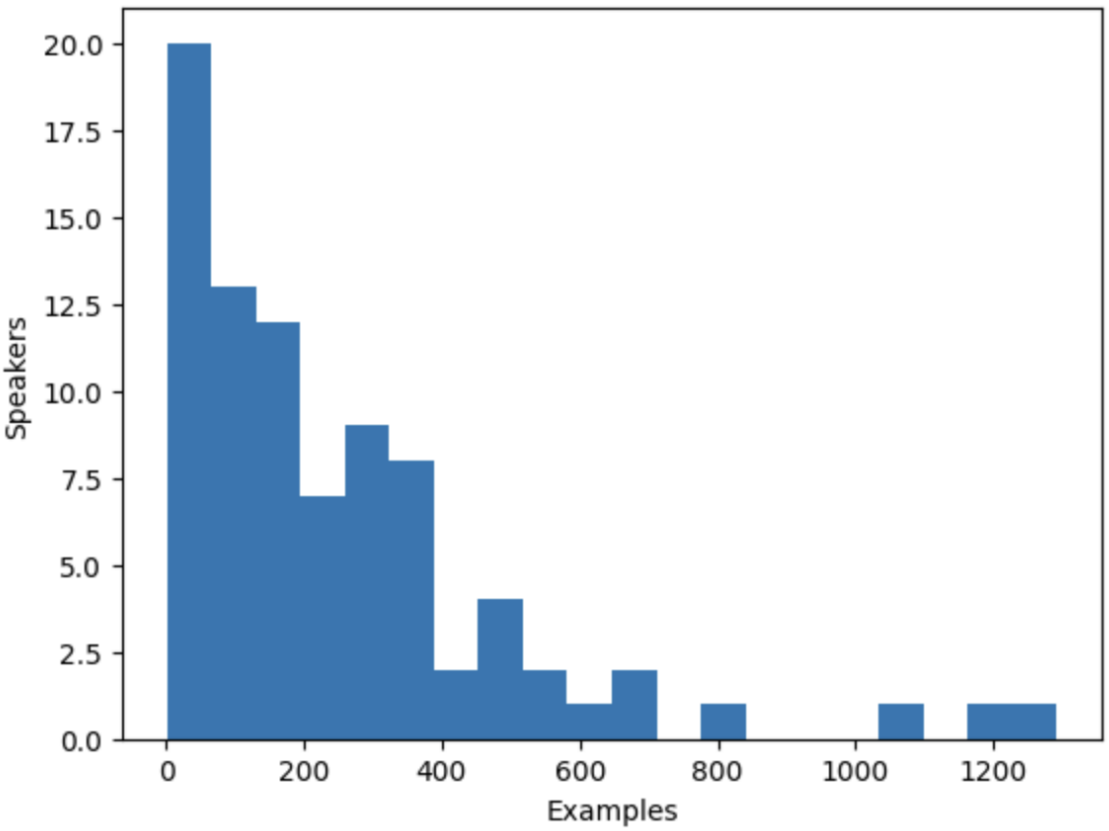
Speakers

The VoxPopuli dataset includes speech from multiple speakers, but how many speakers are represented in the dataset? To determine this, we can count the number of unique speakers and the number of examples each speaker contributes to the dataset. With a total of 20,968 examples in the dataset, this information will give us a better understanding of the distribution of speakers and examples in the data.

```
from collections import defaultdict

speaker_counts = defaultdict(int)

for speaker_id in dataset["speaker_id"]:
    speaker_counts[speaker_id] += 1
```

By plotting a histogram you can get a sense of how much data there is for each speaker.

```
plt.figure()
plt.hist(speaker_counts.values(), bins=20)
plt.ylabel("Speakers")
plt.xlabel("Examples")
plt.show()
```



The histogram reveals that approximately one-third of the speakers in the dataset have fewer than 100 examples, while around ten speakers have more than 500 examples. To improve training efficiency and balance the dataset, we can limit the data to speakers with between 100 and 400 examples.

```
def select_speaker(speaker_id):
    return 100 <= speaker_counts[speaker_id] <= 400


dataset = dataset.filter(select_speaker, input_columns=["speaker_id"])
```

Let's check how many speakers remain:

```
len(set(dataset["speaker_id"]))
```

Output:

```
42
```

Let's see how many examples are left:

```
len(dataset)
```

Output:

```
9973
```

You are left with just under 10,000 examples from approximately 40 unique speakers, which should be sufficient.

Note that some speakers with few examples may actually have more audio available if the examples are long. However, determining the total amount of audio for each speaker requires scanning through the entire dataset, which is a time-consuming process that involves loading and decoding each audio file. As such, we have chosen to skip this step here.

Speaker embeddings

To enable the TTS model to differentiate between multiple speakers, you'll need to create a speaker embedding for each example. The speaker embedding is an additional input into the model that captures a particular speaker's voice characteristics. To generate these speaker embeddings, use the pre-trained spkrec-xvect-voxceleb model from SpeechBrain.

Create a function `create_speaker_embedding()` that takes an input audio waveform and outputs a 512-element vector containing the corresponding speaker embedding.

```
from speechbrain.pretrained import EncoderClassifier

spk_model_name = "speechbrain/spkrec-xvect-voxceleb"

device = "cuda" if torch.cuda.is_available() else "cpu"
speaker_model = EncoderClassifier.from_hparams(
    source=spk_model_name,
    run_opts=,
    savedir=os.path.join("/tmp", spk_model_name),
)


def create_speaker_embedding(waveform):
    with torch.no_grad():
        speaker_embeddings = speaker_model.encode_batch(torch.tensor(waveform))
        speaker_embeddings = torch.nn.functional.normalize(speaker_embeddings,
dim=2)
        speaker_embeddings = speaker_embeddings.squeeze().cpu().numpy()
    return speaker_embeddings
```

It's important to note that the `speechbrain/spkrec-xvect-voxceleb` model was trained on English speech from the VoxCeleb dataset, whereas the training examples in this guide are in Dutch. While we believe that this model will still generate reasonable speaker embeddings for our Dutch dataset, this assumption may not hold true in all cases.

For optimal results, we would need to train an X-vector model on the target speech first. This will ensure that the model is better able to capture the unique voice characteristics present in the Dutch language. If you'd like to train your own X-vector model, you can use this script as an example.

Processing the dataset

Finally, let's process the data into the format the model expects. Create a `prepare_dataset` function that takes in a single example and uses the `SpeechT5Processor` object to tokenize the input text and load the target audio into a log-mel spectrogram. It should also add the speaker embeddings as an additional input.

```
def prepare_dataset(example):
    audio = example["audio"]

    example = processor(
        text=example["text"],
        audio_target=audio["array"],
        sampling_rate=audio["sampling_rate"],
        return_attention_mask=False,
    )

    # strip off the batch dimension
    example["labels"] = example["labels"][0]

    # use SpeechBrain to obtain x-vector
    example["speaker_embeddings"] = create_speaker_embedding(audio["array"])

    return example
```

Verify the processing is correct by looking at a single example:

```
processed_example = prepare_dataset(dataset[0])
list(processed_example.keys())
```

Output:

```
['input_ids', 'labels', 'stop_labels', 'speaker_embeddings']
```

Speaker embeddings should be a 512-element vector:

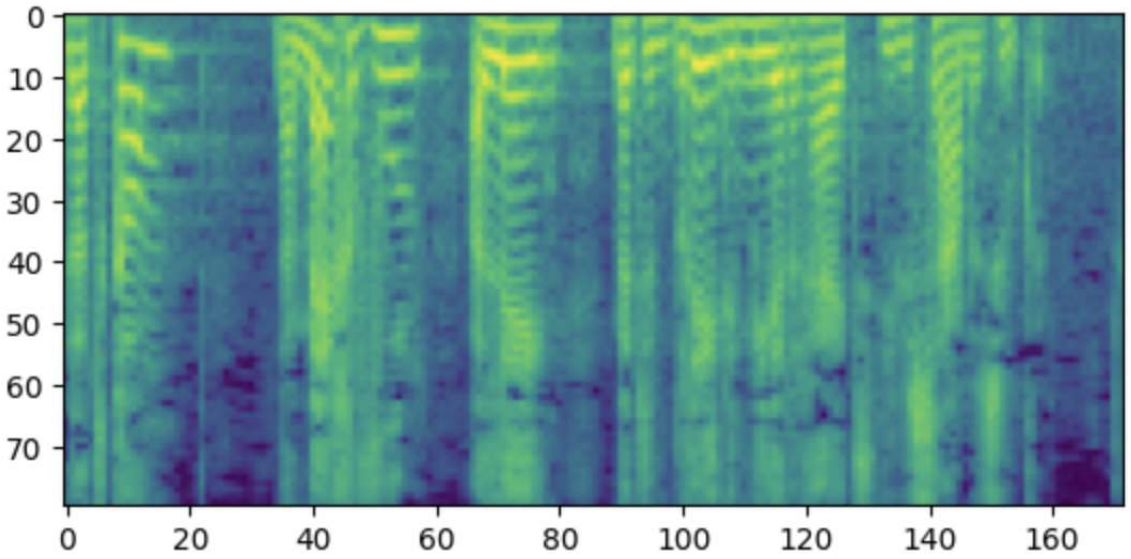
```
processed_example["speaker_embeddings"].shape
```

Output:

```
(512,)
```

The labels should be a log-mel spectrogram with 80 mel bins.

```
plt.figure()
plt.imshow(processed_example["labels"].T)
plt.show()
```



Side note: If you find this spectrogram confusing, it may be due to your familiarity with the convention of placing low frequencies at the bottom and high frequencies at the top of a plot. However, when plotting spectrograms as an image using the matplotlib library, the y-axis is flipped and the spectrograms appear upside down.

Now we need to apply the processing function to the entire dataset. This will take between 5 and 10 minutes.

```
dataset = dataset.map(prepare_dataset, remove_columns=dataset.column_names)
```

You'll see a warning saying that some examples in the dataset are longer than the maximum input length the model can handle (600 tokens). Remove those examples

from the dataset. Here we go even further and to allow for larger batch sizes we remove anything over 200 tokens.

```
def is_not_too_long(input_ids):
    input_length = len(input_ids)
    return input_length < 200


dataset = dataset.filter(is_not_too_long, input_columns=["input_ids"])
len(dataset)
```

Output:

```
8259
```

Next, create a basic train/test split:

```
dataset = dataset.train_test_split(test_size=0.1)
```

Data collator

In order to combine multiple examples into a batch, you need to define a custom data collator. This collator will pad shorter sequences with padding tokens, ensuring that all examples have the same length. For the spectrogram labels, the padded portions are replaced with the special value `-100`. This special value instructs the model to ignore that part of the spectrogram when calculating the spectrogram loss.

```
from dataclasses import dataclass
from typing import Any, Dict, List, Union


@dataclass
class TTSDataCollatorWithPadding:
    processor: Any

    def __call__(
        self, features: List[Dict[str, Union[List[int], torch.Tensor]]]
    ) -> Dict[str, torch.Tensor]:
        input_ids = [ for feature in features]
        label_features = [ for feature in features]
        speaker_features = [feature["speaker_embeddings"] for feature in features]

        # collate the inputs and targets into a batch
        batch = processor.pad(
            input_ids=input_ids, labels=label_features, return_tensors="pt"
        )
```

```
        # replace padding with -100 to ignore loss correctly
        batch["labels"] = batch["labels"].masked_fill(
            batch.decoder_attention_mask.unsqueeze(-1).ne(1), -100
        )

        # not used during fine-tuning
        del batch["decoder_attention_mask"]

        # round down target lengths to multiple of reduction factor
        if model.config.reduction_factor > 1:
            target_lengths = torch.tensor(
                [len(feature["input_values"]) for feature in label_features]
            )
            target_lengths = target_lengths.new(
                [
                    length - length % model.config.reduction_factor
                    for length in target_lengths
                ]
            )
            max_length = max(target_lengths)
            batch["labels"] = batch["labels"][:, :max_length]

        # also add in the speaker embeddings
        batch["speaker_embeddings"] = torch.tensor(speaker_features)

        return batch
```

In SpeechT5, the input to the decoder part of the model is reduced by a factor 2. In other words, it throws away every other timestep from the target sequence. The decoder then predicts a sequence that is twice as long. Since the original target sequence length may be odd, the data collator makes sure to round the maximum length of the batch down to be a multiple of 2.

```
data_collator = TTSDataCollatorWithPadding(processor=processor)
```

Train the model

Load the pre-trained model from the same checkpoint as you used for loading the processor:

```
from transformers import SpeechT5ForTextToSpeech

model = SpeechT5ForTextToSpeech.from_pretrained(checkpoint)
```

The `use_cache=True` option is incompatible with gradient checkpointing. Disable it for training, and re-enable cache for generation to speed-up inference time:

```
from functools import partial

# disable cache during training since it's incompatible with gradient checkpointing
model.config.use_cache = False

# set language and task for generation and re-enable cache
model.generate = partial(model.generate, use_cache=True)
```

Define the training arguments. Here we are not computing any evaluation metrics during the training process, we'll talk about evaluation later in this chapter. Instead, we'll only look at the loss:

```
from transformers import Seq2SeqTrainingArguments

training_args = Seq2SeqTrainingArguments(
    output_dir="speecht5_finetuned_voxpopuli_nl",  # change to a repo name of your
choice
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    learning_rate=1e-5,
    warmup_steps=500,
    max_steps=4000,
    gradient_checkpointing=True,
    fp16=True,
    eval_strategy="steps",
    per_device_eval_batch_size=2,
    save_steps=1000,
    eval_steps=1000,
    logging_steps=25,
    report_to=["tensorboard"],
    load_best_model_at_end=True,
    greater_is_better=False,
    label_names=["labels"],
    push_to_hub=True,
)
```

Instantiate the `Trainer` object and pass the model, dataset, and data collator to it.

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
    args=training_args,
    model=model,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    data_collator=data_collator,
    tokenizer=processor,
)
```

And with that, we're ready to start training! Training will take several hours. Depending on your GPU, it is possible that you will encounter a CUDA "out-of-memory" error when you start training. In this case, you can reduce the `per_device_train_batch_size` incrementally by factors of 2 and increase `gradient_accumulation_steps` by 2x to compensate.

```
trainer.train()
```

Push the final model to the 🤗 Hub:

```
trainer.push_to_hub()
```

Inference

Once you have fine-tuned a model, you can use it for inference! Load the model from the 🤗 Hub (make sure to use your account name in the following code snippet):

```
model = SpeechT5ForTextToSpeech.from_pretrained(
    "YOUR_ACCOUNT/speecht5_finetuned_voxpopuli_nl"
)
```

Pick an example, here we'll take one from the test dataset. Obtain a speaker embedding.

```
example = dataset["test"][304]
speaker_embeddings = torch.tensor(example["speaker_embeddings"]).unsqueeze(0)
```

Define some input text and tokenize it.

```
text = "hallo allemaal, ik praat nederlands. groetjes aan iedereen!"
```

Preprocess the input text:

```
inputs = processor(text=text, return_tensors="pt")
```

Instantiate a vocoder and generate speech:

```
from transformers import SpeechT5HifiGan

vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan")
```

```
speech = model.generate_speech(inputs["input_ids"], speaker_embeddings,
vocoder=vocoder)
```

Ready to listen to the result?

```
from IPython.display import Audio

Audio(speech.numpy(), rate=16000)
```

Obtaining satisfactory results from this model on a new language can be challenging. The quality of the speaker embeddings can be a significant factor. Since SpeechT5 was pre-trained with English x-vectors, it performs best when using English speaker embeddings. If the synthesized speech sounds poor, try using a different speaker embedding.

Increasing the training duration is also likely to enhance the quality of the results. Even so, the speech clearly is Dutch instead of English, and it does capture the voice characteristics of the speaker (compare to the original audio in the example). Another thing to experiment with is the model's configuration. For example, try using `config.reduction_factor = 1` to see if this improves the results.

In the next section, we'll talk about how we evaluate text-to-speech models.

Hands-on exercise

In this unit, we have explored text-to-speech audio task, talked about existing datasets, pretrained models and nuances of fine-tuning SpeechT5 for a new language.

As you've seen, fine-tuning models for text-to-speech task can be challenging in low-resource scenarios. At the same time, evaluating text-to-speech models isn't easy either.

For these reasons, this hands-on exercise will focus on practicing the skills rather than achieving a certain metric value.

Your objective for this task is to fine-tune SpeechT5 on a dataset of your choosing. You have the freedom to select another language from the same `voxpopuli` dataset, or you can pick any other dataset listed in this unit.

Be mindful of the training data size! For training on a free tier GPU from Google Colab, we recommend limiting the training data to about 10-15 hours.

Once you have completed the fine-tuning process, share your model by uploading it to the Hub. Make sure to tag your model as a `text-to-speech` model either with appropriate kwargs, or in the Hub UI.

Remember, the primary aim of this exercise is to provide you with ample practice, allowing you to refine your skills and gain a deeper understanding of text-to-speech audio tasks.

Unit 6. From text to speech

In the previous unit, you learned how to use Transformers to convert spoken speech into text. Now let's flip the script and see how you can transform a given input text into an audio output that sounds like human speech.

The task we will study in this unit is called "Text-to-speech" (TTS). Models capable of converting text into audible human speech have a wide range of potential applications:

- Assistive apps: think about tools that can leverage these models to enable visually-impaired people to access digital content through the medium of sound.
- Audiobook narration: converting written books into audio form makes literature more accessible to individuals who prefer to listen or have difficulty with reading.
- Virtual assistants: TTS models are a fundamental component of virtual assistants like Siri, Google Assistant, or Amazon Alexa. Once they have used a classification model to catch the wake word, and used ASR model to process your request, they can use a TTS model to respond to your inquiry.
- Entertainment, gaming and language learning: give voice to your NPC characters, narrate game events, or help language learners with examples of correct pronunciation and intonation of words and phrases.

These are just a few examples, and I am sure you can imagine many more! However, with so much power comes the responsibility, and it is important to highlight that TTS models have the potential to be used for malicious purposes. For example, with sufficient voice samples, malicious actors could potentially create convincing fake audio recordings, leading to the unauthorized use of someone's voice for fraudulent purposes or manipulation. If you plan to collect data for fine-tuning your own systems, carefully consider privacy and informed consent. Voice data should be obtained with explicit consent from individuals, ensuring they

understand the purpose, scope, and potential risks associated with their voice being used in a TTS system. Please use text-to-speech responsibly.

What you'll learn and what you'll build

In this unit we will talk about:

- Datasets suitable for text-to-speech training
- Pre-trained models for text-to-speech
- Fine-tuning SpeechT5 on a new language
- Evaluating TTS models

Pre-trained models for text-to-speech

Compared to ASR (automatic speech recognition) and audio classification tasks, there are significantly fewer pre-trained model checkpoints available. On the 🤗 Hub, you'll find close to 300 suitable checkpoints. Among these pre-trained models we'll focus on two architectures that are readily available for you in the 🤗 Transformers library - SpeechT5 and Massive Multilingual Speech (MMS). In this section, we'll explore how to use these pre-trained models in the Transformers library for TTS.

SpeechT5

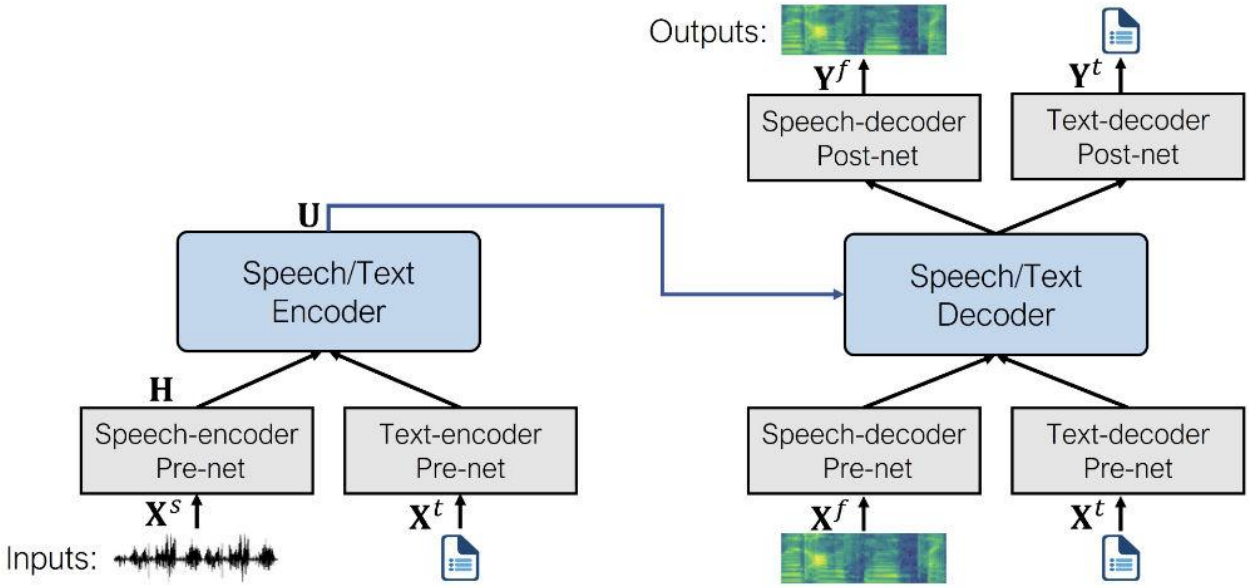
SpeechT5 is a model published by Junyi Ao et al. from Microsoft that is capable of handling a range of speech tasks. While in this unit, we focus on the text-to-speech aspect, this model can be tailored to speech-to-text tasks (automatic speech recognition or speaker identification), as well as speech-to-speech (e.g. speech enhancement or converting between different voices). This is due to how the model is designed and pre-trained.

At the heart of SpeechT5 is a regular Transformer encoder-decoder model. Just like any other Transformer, the encoder-decoder network models a sequence-to-sequence transformation using hidden representations. This Transformer backbone is the same for all tasks SpeechT5 supports.

This Transformer is complemented with six modal-specific (speech/text) *pre-nets* and *post-nets*. The input speech or text (depending on the task) is preprocessed through a corresponding pre-net to obtain the hidden representations that

Transformer can use. The Transformer's output is then passed to a post-net that will use it to generate the output in the target modality.

This is what the architecture looks like (image from the original paper):



SpeechT5 is first pre-trained using large-scale unlabeled speech and text data, to acquire a unified representation of different modalities. During the pre-training phase all pre-nets and post-nets are used simultaneously.

After pre-training, the entire encoder-decoder backbone is fine-tuned for each individual task. At this step, only the pre-nets and post-nets relevant to the specific task are employed. For example, to use SpeechT5 for text-to-speech, you'd need the text encoder pre-net for the text inputs and the speech decoder pre- and post-nets for the speech outputs.

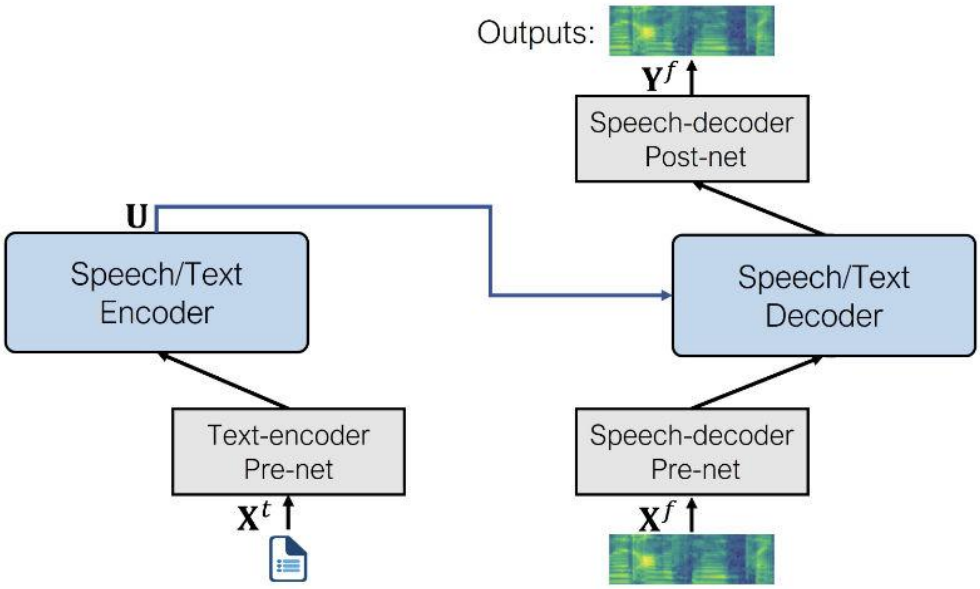
This approach allows to obtain several models fine-tuned for different speech tasks that all benefit from the initial pre-training on unlabeled data.

Even though the fine-tuned models start out using the same set of weights from the shared pre-trained model, the final versions are all quite different in the end. You can't take a fine-tuned ASR model and swap out the pre-nets and post-net to get a working TTS model, for example. SpeechT5 is flexible, but not that flexible ;)

Let's see what are the pre- and post-nets that SpeechT5 uses for the TTS task specifically:

- Text encoder pre-net: A text embedding layer that maps text tokens to the hidden representations that the encoder expects. This is similar to what happens in an NLP model such as BERT.
- Speech decoder pre-net: This takes a log mel spectrogram as input and uses a sequence of linear layers to compress the spectrogram into hidden representations.
- Speech decoder post-net: This predicts a residual to add to the output spectrogram and is used to refine the results.

When combined, this is what SpeechT5 architecture for text-to-speech looks like:



As you can see, the output is a log mel spectrogram and not a final waveform. If you recall, we briefly touched on this topic in Unit 3. It is common for models that generate audio to produce a log mel spectrogram, which needs to be converted to a waveform with an additional neural network known as a vocoder.

Let's see how you could do that.

First, let's load the fine-tuned TTS SpeechT5 model from the 🤗 Hub, along with the processor object used for tokenization and feature extraction:

```
from transformers import SpeechT5Processor, SpeechT5ForTextToSpeech

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")
model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts")
```

Next, tokenize the input text.

```
inputs = processor(text="Don't count the days, make the days count.",
return_tensors="pt")
```

The SpeechT5 TTS model is not limited to creating speech for a single speaker. Instead, it uses so-called speaker embeddings that capture a particular speaker's voice characteristics.

Speaker embeddings is a method of representing a speaker's identity in a compact way, as a vector of fixed size, regardless of the length of the utterance. These embeddings capture essential information about a speaker's voice, accent, intonation, and other unique characteristics that distinguish one speaker from another. Such embeddings can be used for speaker verification, speaker diarization, speaker identification, and more. The most common techniques for generating speaker embeddings include:

- I-Vectors (identity vectors): I-Vectors are based on a Gaussian mixture model (GMM). They represent speakers as low-dimensional fixed-length vectors derived from the statistics of a speaker-specific GMM, and are obtained in unsupervised manner.
- X-Vectors: X-Vectors are derived using deep neural networks (DNNs) and capture frame-level speaker information by incorporating temporal context.

X-Vectors are a state-of-the-art method that shows superior performance on evaluation datasets compared to I-Vectors. The deep neural network is used to obtain X-Vectors: it trains to discriminate between speakers, and maps variable-length utterances to fixed-dimensional embeddings. You can also load an X-Vector speaker embedding that has been computed ahead of time, which will encapsulate the speaking characteristics of a particular speaker.

Let's load such a speaker embedding from a dataset on the Hub. The embeddings were obtained from the CMU ARCTIC dataset using this script, but any X-Vector embedding should work.

```
from datasets import load_dataset

embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors",
split="validation")



speaker_embeddings = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)
```

The speaker embedding is a tensor of shape (1, 512). This particular speaker embedding describes a female voice.

At this point we already have enough inputs to generate a log mel spectrogram as an output, you can do it like this:

```
spectrogram = model.generate_speech(inputs["input_ids"], speaker_embeddings)
```

This outputs a tensor of shape (140, 80) containing a log mel spectrogram. The first dimension is the sequence length, and it may vary between runs as the speech decoder pre-net always applies dropout to the input sequence. This adds a bit of random variability to the generated speech.

However, if we are looking to generate speech waveform, we need to specify a vocoder to use for the spectrogram to waveform conversion. In theory, you can use any vocoder that works on 80-bin mel spectrograms. Conveniently, 🤗 Transformers offers a vocoder based on HiFi-GAN. Its weights were kindly provided by the original authors of SpeechT5.

HiFi-GAN is a state-of-the-art generative adversarial network (GAN) designed for high-fidelity speech synthesis. It is capable of generating high-quality and realistic audio waveforms from spectrogram inputs.

On a high level, HiFi-GAN consists of one generator and two discriminators. The generator is a fully convolutional neural network that takes a mel-spectrogram as input and learns to produce raw audio waveforms. The discriminators' role is to distinguish between real and generated audio. The two discriminators focus on different aspects of the audio.

HiFi-GAN is trained on a large dataset of high-quality audio recordings. It uses a so-called *adversarial training*, where the generator and discriminator networks compete against each other. Initially, the generator produces low-quality audio, and the discriminator can easily differentiate it from real audio. As training progresses, the generator improves its output, aiming to fool the discriminator. The discriminator, in turn, becomes more accurate in distinguishing real and generated audio. This adversarial feedback loop helps both networks improve over time. Ultimately, HiFi-GAN learns to generate high-fidelity audio that closely resembles the characteristics of the training data.

Loading the vocoder is as easy as any other 🤗 Transformers model.

```
from transformers import SpeechT5HifiGan

vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan")
```

Now all you need to do is pass it as an argument when generating speech, and the outputs will be automatically converted to the speech waveform.

```
speech = model.generate_speech(inputs["input_ids"], speaker_embeddings,
vocoder=vocoder)
```

Let's listen to the result. The sample rate used by SpeechT5 is always 16 kHz.

```
from IPython.display import Audio

Audio(speech, rate=16000)
```

Neat!

Feel free to play with the SpeechT5 text-to-speech demo, explore other voices, experiment with inputs. Note that this pre-trained checkpoint only supports English language:

Bark

Bark is a transformer-based text-to-speech model proposed by Suno AI in suno-ai/bark.

Unlike SpeechT5, Bark generates raw speech waveforms directly, eliminating the need for a separate vocoder during inference – it's already integrated. This efficiency is achieved through the utilization of `Encodec`, which serves as both a codec and a compression tool.

With `Encodec`, you can compress audio into a lightweight format to reduce memory usage and subsequently decompress it to restore the original audio. This compression process is facilitated by 8 codebooks, each consisting of integer vectors. Think of these codebooks as representations or embeddings of the audio in integer form. It's important to note that each successive codebook improves the quality of the audio reconstruction from the previous codebooks. As codebooks are integer vectors, they can be learned by transformer models, which are very efficient in this task. This is what Bark was specifically trained to do.

To be more specific, Bark is made of 4 main models:

- `BarkSemanticModel` (also referred to as the 'text' model): a causal auto-regressive transformer model that takes as input tokenized text, and predicts semantic text tokens that capture the meaning of the text.
- `BarkCoarseModel` (also referred to as the 'coarse acoustics' model): a causal autoregressive transformer, that takes as input the results of the `BarkSemanticModel` model. It aims at predicting the first two audio codebooks necessary for EnCodec.
- `BarkFineModel` (the 'fine acoustics' model), this time a non-causal autoencoder transformer, which iteratively predicts the last codebooks based on the sum of the previous codebooks embeddings.
- having predicted all the codebook channels from the `EncodecModel`, Bark uses it to decode the output audio array.

It should be noted that each of the first three modules can support conditional speaker embeddings to condition the output sound according to specific predefined voice.

Bark is an highly-controllable text-to-speech model, meaning you can use with various settings, as we are going to see.

Before everything, load the model and its processor.

The processor role here is two-sides: 1. It is used to tokenize the input text, i.e. to cut it into small pieces that the model can understand. 2. It stores speaker embeddings, i.e voice presets that can condition the generation.

```
from transformers import BarkModel, BarkProcessor

model = BarkModel.from_pretrained("suno/bark-small")
processor = BarkProcessor.from_pretrained("suno/bark-small")
```

Bark is very versatile and can generate audio conditioned by a speaker embeddings library which can be loaded via the processor.

```
# add a speaker embedding
inputs = processor("This is a test!", voice_preset="v2/en_speaker_3")

speech_output = model.generate(**inputs).cpu().numpy()
```

Your browser does not support the audio element.

It can also generate ready-to-use multilingual speeches, such as French and Chinese. You can find a list of supported languages here. Unlike MMS, discussed below, it is not necessary to specify the language used, but simply adapt the input text to the corresponding language.

```
# try it in French, let's also add a French speaker embedding
inputs = processor("C'est un test!", voice_preset="v2/fr_speaker_1")

speech_output = model.generate(**inputs).cpu().numpy()
```

Your browser does not support the audio element.

The model can also generate **non-verbal communications** such as laughing, sighing and crying. You just have to modify the input text with corresponding cues such as `[clears throat]`, `[laughter]`, or `...`.

```
inputs = processor(
    "[clears throat] This is a test ... and I just took a long pause.",
    voice_preset="v2/fr_speaker_1",
)

speech_output = model.generate(**inputs).cpu().numpy()
```

Your browser does not support the audio element.

Bark can even generate music. You can help by adding ♪ musical notes ♪ around your words.

```
inputs = processor(
    "♪ In the mighty jungle, I'm trying to generate barks.",
)

speech_output = model.generate(**inputs).cpu().numpy()
```

Your browser does not support the audio element.

In addition to all these features, Bark supports batch processing, which means you can process several text entries at the same time, at the expense of more intensive computation. On some hardware, such as GPUs, batching enables faster overall generation, which means it can be faster to generate samples all at once than to generate them one by one.

Let's try generating a few examples:

```
input_list = [
    "[clears throat] Hello uh ..., my dog is cute [laughter]",
```

```
    "Let's try generating speech, with Bark, a text-to-speech model",
    "♪ In the jungle, the mighty jungle, the lion barks tonight ♪",
]

# also add a speaker embedding
inputs = processor(input_list, voice_preset="v2/en_speaker_3")

speech_output = model.generate(**inputs).cpu().numpy()
```

Let's listen to the outputs one by one.

First one:

```
from IPython.display import Audio

sampling_rate = model.generation_config.sample_rate
Audio(speech_output[0], rate=sampling_rate)
```

Your browser does not support the audio element.

Second one:

```
Audio(speech_output[1], rate=sampling_rate)
```

Your browser does not support the audio element.

Third one:

```
Audio(speech_output[2], rate=sampling_rate)
```

Your browser does not support the audio element.

Bark, like other 🤗 Transformers models, can be optimized in just a few lines of code regarding speed and memory impact. To find out how, click on this colab demonstration notebook.

Massive Multilingual Speech (MMS)

What if you are looking for a pre-trained model in a language other than English? Massive Multilingual Speech (MMS) is another model that covers an array of speech tasks, however, it supports a large number of languages. For instance, it can synthesize speech in over 1,100 languages.

MMS for text-to-speech is based on VITS Kim et al., 2021, which is one of the state-of-the-art TTS approaches.

VITS is a speech generation network that converts text into raw speech waveforms. It works like a conditional variational auto-encoder, estimating audio features from the input text. First, acoustic features, represented as spectrograms, are generated. The waveform is then decoded using transposed convolutional layers adapted from HiFi-GAN. During inference, the text encodings are upsampled and transformed into waveforms using the flow module and HiFi-GAN decoder. Like Bark, there's no need for a vocoder, as waveforms are generated directly.

MMS model has been added to 🤗 Transformers very recently, so you will have to install the library from source:

```
pip install git+https://github.com/huggingface/transformers.git
```

Let's give MMS a go, and see how we can synthesize speech in a language other than English, e.g. German. First, we'll load the model checkpoint and the tokenizer for the correct language:

```
from transformers import VitsModel, VitsTokenizer

model = VitsModel.from_pretrained("facebook/mms-tts-deu")
tokenizer = VitsTokenizer.from_pretrained("facebook/mms-tts-deu")
```

You may notice that to load the MMS model you need to use `VitsModel` and `VitsTokenizer`. This is because MMS for text-to-speech is based on the VITS model as mentioned earlier.

Let's pick an example text in German, like these first two lines from a children's song:

```
text_example = (
    "Ich bin Schnappi das kleine Krokodil, komm aus Ägypten das liegt direkt am
Nil."
)
```

To generate a waveform output, preprocess the text with the tokenizer, and pass it to the model:

```
inputs = tokenizer(text_example, return_tensors="pt")
input_ids = inputs["input_ids"]


with torch.no_grad():
    outputs = model(input_ids)
```

```
speech = outputs["waveform"]
```

Let's listen to it:

```
from IPython.display import Audio

Audio(speech, rate=16000)
```

Wunderbar! If you'd like to try MMS with another language, find other suitable `vits` checkpoints on 🤗 Hub.

Now let's see how you can fine-tune a TTS model yourself!

Supplemental reading and resources

This unit introduced the text-to-speech task, and covered a lot of ground. Want to learn more? Here you will find additional resources that will help you deepen your understanding of the topics and enhance your learning experience.

- HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis: a paper introducing HiFi-GAN for speech synthesis.
- X-Vectors: Robust DNN Embeddings For Speaker Recognition: a paper introducing X-Vector method for speaker embeddings.
- FastSpeech 2: Fast and High-Quality End-to-End Text to Speech: a paper introducing FastSpeech 2, another popular text-to-speech model that uses a non-autoregressive TTS method.
- A Vector Quantized Approach for Text to Speech Synthesis on Real-World Spontaneous Speech: a paper introducing MQTTS, an autoregressive TTS system that replaces mel-spectrograms with quantized discrete representation.

Text-to-speech datasets

Text-to-speech task (also called *speech synthesis*) comes with a range of challenges.

First, just like in the previously discussed automatic speech recognition, the alignment between text and speech can be tricky.
However, unlike ASR, TTS is a **one-to-many** mapping problem, i.e. the same text can be synthesised in many different ways. Think about the diversity of voices and

speaking styles in the speech you hear on a daily basis - each person has a different way of speaking the same sentence, but they are all valid and correct! Even different outputs (spectrograms or audio waveforms) can correspond to the same ground truth. The model has to learn to generate the correct duration and timing for each phoneme, word, or sentence which can be challenging, especially for long and complex sentences.

Next, there's the long-distance dependency problem: language has a temporal aspect, and understanding the meaning of a sentence often requires considering the context of surrounding words. Ensuring that the TTS model captures and retains contextual information over long sequences is crucial for generating coherent and natural-sounding speech.

Finally, training TTS models typically requires pairs of text and corresponding speech recordings. On top of that, to ensure the model can generate speech that sounds natural for various speakers and speaking styles, data should contain diverse and representative speech samples from multiple speakers. Collecting such data is expensive, time-consuming and for some languages is not feasible. You may think, why not just take a dataset designed for ASR (automatic speech recognition) and use it for training a TTS model? Unfortunately, automated speech recognition (ASR) datasets are not the best option. The features that make it beneficial for ASR, such as excessive background noise, are typically undesirable in TTS. It's great to be able to pick out speach from a noisy street recording, but not so much if your voice assistant replies to you with cars honking and construction going full-swing in the background. Still, some ASR datasets can sometimes be useful for fine-tuning, as finding top-quality, multilingual, and multi-speaker TTS datasets can be quite challenging.

Let's explore a few datasets suitable for TTS that you can find on the 🤗 Hub.

LJSpeech

LJSpeech is a dataset that consists of 13,100 English-language audio clips paired with their corresponding transcriptions. The dataset contains recording of a single speaker reading sentences from 7 non-fiction books in English. LJSpeech is often used as a benchmark for evaluating TTS models due to its high audio quality and diverse linguistic content.

Multilingual LibriSpeech

Multilingual LibriSpeech is a multilingual extension of the LibriSpeech dataset, which is a large-scale collection of read English-language audiobooks. Multilingual LibriSpeech expands on this by including additional languages, such as German, Dutch, Spanish, French, Italian, Portuguese, and Polish. It offers audio recordings along with aligned transcriptions for each language. The dataset provides a valuable resource for developing multilingual TTS systems and exploring cross-lingual speech synthesis techniques.

VCTK (Voice Cloning Toolkit)

VCTK is a dataset specifically designed for text-to-speech research and development. It contains audio recordings of 110 English speakers with various accents. Each speaker reads out about 400 sentences, which were selected from a newspaper, the rainbow passage and an elicitation paragraph used for the speech accent archive. VCTK offers a valuable resource for training TTS models with varied voices and accents, enabling more natural and diverse speech synthesis.

Libri-TTS/ LibriTTS-R

Libri-TTS/ LibriTTS-R is a multi-speaker English corpus of approximately 585 hours of read English speech at 24kHz sampling rate, prepared by Heiga Zen with the assistance of Google Speech and Google Brain team members. The LibriTTS corpus is designed for TTS research. It is derived from the original materials (mp3 audio files from LibriVox and text files from Project Gutenberg) of the LibriSpeech corpus. The main differences from the LibriSpeech corpus are listed below:

- The audio files are at 24kHz sampling rate.
- The speech is split at sentence breaks.
- Both original and normalized texts are included.
- Contextual information (e.g., neighbouring sentences) can be extracted.
- Utterances with significant background noise are excluded.

Assembling a good dataset for TTS is no easy task as such dataset would have to possess several key characteristics:

- High-quality and diverse recordings that cover a wide range of speech patterns, accents, languages, and emotions. The recordings should be clear, free from background noise, and exhibit natural speech characteristics.

- Transcriptions: Each audio recording should be accompanied by its corresponding text transcription.
- Variety of linguistic content: The dataset should contain a diverse range of linguistic content, including different types of sentences, phrases, and words. It should cover various topics, genres, and domains to ensure the model's ability to handle different linguistic contexts.

Good news is, it is unlikely that you would have to train a TTS model from scratch. In the next section we'll look into pre-trained models available on the 🤗 Hub.

Hands-on exercise

In this Unit, we consolidated the material covered in the previous six units of the course to build three integrated audio applications. As you've experienced, building more involved audio tools is fully within reach by using the foundational skills you've acquired in this course.

The hands-on exercise takes one of the applications covered in this Unit, and extends it with a few multilingual tweaks 🌍 Your objective is to take the cascaded speech-to-speech translation Gradio demo from the first section in this Unit, and update it to translate to any **non-English** language. That is to say, the demo should take speech in language X, and translate it to speech in language Y, where the target language Y is not English. You should start by duplicating the template under your Hugging Face namespace. There's no requirement to use a GPU accelerator device - the free CPU tier works just fine 🤗 However, you should ensure that the visibility of your demo is set to **public**. This is required such that your demo is accessible to us and can thus be checked for correctness.

Tips for updating the speech translation function to perform multilingual speech translation are provided in the section on speech-to-speech translation. By following these instructions, you should be able to update the demo to translate from speech in language X to text in language Y, which is half of the task!

To synthesise from text in language Y to speech in language Y, where Y is a multilingual language, you will need to use a multilingual TTS checkpoint. For this, you can either use the SpeechT5 TTS checkpoint that you fine-tuned in the previous hands-on exercise, or a pre-trained multilingual TTS checkpoint. There are two options for pre-trained checkpoints, either the checkpoint sanchit-gandhi/speecht5_tts_vox_nl, which is a SpeechT5 checkpoint fine-tuned on the Dutch split of

the VoxPopuli dataset, or an MMS TTS checkpoint (see section on pretrained models for TTS).

In our experience experimenting with the Dutch language, using an MMS TTS checkpoint results in better performance than a fine-tuned SpeechT5 one, but you might find that your fine-tuned TTS checkpoint is preferable in your language. If you decide to use an MMS TTS checkpoint, you will need to update the requirements.txt file of your demo to install `transformers` from the PR branch:

```
git+https://github.com/hollance/transformers.git@6900e8ba6532162a8613d2270ec2286c3f58f57b
```

Your demo should take as input an audio file, and return as output another audio file, matching the signature of the `speech_to_speech_translation` function in the template demo. Therefore, we recommend that you leave the main function `speech_to_speech_translation` as is, and only update the `translate` and `synthesise` functions as required.

Once you have built your demo as a Gradio demo on the Hugging Face Hub, you can submit it for assessment. Head to the Space audio-course-u7-assessment and provide the repository id of your demo when prompted. This Space will check that your demo has been built correctly by sending a sample audio file to your demo and checking that the returned audio file is indeed non-English. If your demo works correctly, you'll get a green tick next to your name on the overall progress space ✅

Unit 7. Putting it all together 🪢

Well done on making it to Unit 7 🥳 You're just a few steps away from completing the course and acquiring the final few skills you need to navigate the field of Audio ML. In terms of understanding, you already know everything there is to know! Together, we've comprehensively covered the main topics that constitute the audio domain and their accompanying theory (audio data, audio classification, speech recognition and text-to-speech). What this Unit aims to deliver is a framework for **putting it all together**: now that you know how each of these tasks work in isolation, we're going to explore how you can combine them together to build some real-world applications.

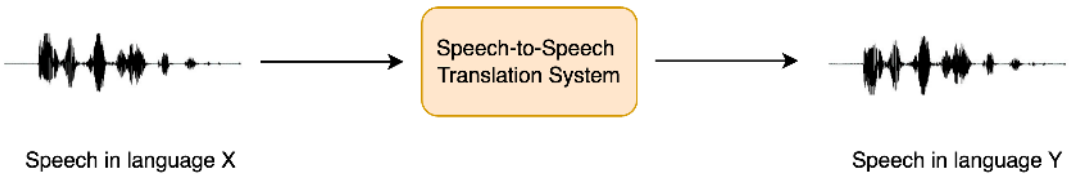
What you'll learn and what you'll build

In this Unit, we'll cover the following three topics:

- Speech-to-speech translation: translate speech from one language into speech in a different language
- Creating a voice assistant: build your own voice assistant that works in a similar way to Alexa or Siri
- Transcribing meetings: transcribe a meeting and label the transcript with who spoke when

Speech-to-speech translation

Speech-to-speech translation (STST or S2ST) is a relatively new spoken language processing task. It involves translating speech from one langauge into speech in a **different** language:



STST can be viewed as an extension of the traditional machine translation (MT) task: instead of translating **text** from one language into another, we translate **speech** from one language into another. STST holds applications in the field of multilingual communication, enabling speakers in different languages to communicate with one another through the medium of speech.

Suppose you want to communicate with another individual across a langauge barrier. Rather than writing the information that you want to convey and then translating it to text in the target language, you can speak it directly and have a STST system convert your spoken speech into the target langauge. The recipient can then respond by speaking back at the STST system, and you can listen to their response. This is a more natural way of communicating compared to text-based machine translation.

In this chapter, we'll explore a *cascaded* approach to STST, piecing together the knowledge you've acquired in Units 5 and 6 of the course. We'll use a *speech*

translation (ST) system to transcribe the source speech into text in the target language, then *text-to-speech (TTS)* to generate speech in the target language from the translated text:

We could also have used a three stage approach, where first we use an automatic speech recognition (ASR) system to transcribe the source speech into text in the same language, then machine translation to translate the transcribed text into the target language, and finally text-to-speech to generate speech in the target language. However, adding more components to the pipeline lends itself to *error propagation*, where the errors introduced in one system are compounded as they flow through the remaining systems, and also increases latency, since inference has to be conducted for more models.

While this cascaded approach to STST is pretty straightforward, it results in very effective STST systems. The three-stage cascaded system of ASR + MT + TTS was previously used to power many commercial STST products, including Google Translate. It's also a very data and compute efficient way of developing a STST system, since existing speech recognition and text-to-speech systems can be coupled together to yield a new STST model without any additional training.

In the remainder of this Unit, we'll focus on creating a STST system that translates speech from any language X to speech in English. The methods covered can be extended to STST systems that translate from any language X to any langauge Y, but we leave this as an extension to the reader and provide pointers where applicable. We further divide up the task of STST into its two constituent components: ST and TTS. We'll finish by piecing them together to build a Gradio demo to showcase our system.

Speech translation

We'll use the Whisper model for our speech translation system, since it's capable of translating from over 96 languages to English. Specifically, we'll load the Whisper Base checkpoint, which clocks in at 74M parameters. It's by no means the most performant Whisper model, with the largest Whisper checkpoint being over 20x larger, but since we're concatenating two auto-regressive systems together (ST +

TTS), we want to ensure each model can generate relatively quickly so that we get reasonable inference speed:

```
from transformers import pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
pipe = pipeline(
    "automatic-speech-recognition", model="openai/whisper-base", device=device
)
```

Great! To test our STST system, we'll load an audio sample in a non-English language. Let's load the first example of the Italian (`it`) split of the VoxPopuli dataset:

```
from datasets import load_dataset

dataset = load_dataset("qmeeus/voxpopuli", "it", split="validation",
streaming=True)
sample = next(iter(dataset))
```

To listen to this sample, we can either play it using the dataset viewer on the Hub: qmeeus/voxpopuli/viewer

Or playback using the ipynb audio feature:

```
from IPython.display import Audio

Audio(sample["audio"]["array"], rate=sample["audio"]["sampling_rate"])
```

Now let's define a function that takes this audio input and returns the translated text. You'll remember that we have to pass the generation key-word argument for the `"task"`, setting it to `"translate"` to ensure that Whisper performs speech translation and not speech recognition:

```
def translate(audio):
    outputs = pipe(audio, max_new_tokens=256, generate_kwargs=)
    return outputs["text"]
```

```
Whisper can also be 'tricked' into translating from speech in any language X to any
language Y. Simply set the task to
`"transcribe"` and the `"language"` to your target language in the generation key-
word arguments,
e.g. for Spanish, one would set:
```

```
`generate_kwargs={"task": "transcribe", "language": "es"&rcub;`
```

Great! Let's quickly check that we get a sensible result from the model:

```
translate(sample["audio"].copy())
```

```
' psychological and social. I think that it is a very important step in the
construction of a juridical space of freedom, circulation and protection of
rights.'
```

Alright! If we compare this to the source text:

```
sample["text"]
```

```
'Penso che questo sia un passo in avanti importante nella costruzione di uno spazio
giuridico di libertà di circolazione e di protezione dei diritti per le persone in
Europa.'
```

We see that the translation more or less lines up (you can double check this using Google Translate), barring a small extra few words at the start of the transcription where the speaker was finishing off their previous sentence.

With that, we've completed the first half of our cascaded STST pipeline, putting into practice the skills we gained in Unit 5 when we learnt how to use the Whisper model for speech recognition and translation. If you want a refresher on any of the steps we covered, have a read through the section on Pre-trained models for ASR from Unit 5.

Text-to-speech

The second half of our cascaded STST system involves mapping from English text to English speech. For this, we'll use the pre-trained SpeechT5 TTS model for English TTS. 🤗 Transformers currently doesn't have a TTS `pipeline`, so we'll have to use the model directly ourselves. This is no biggie, you're all experts on using the model for inference following Unit 6!

First, let's load the SpeechT5 processor, model and vocoder from the pre-trained checkpoint:

```
from transformers import SpeechT5Processor, SpeechT5ForTextToSpeech,
SpeechT5HifiGan
```

```
processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")

model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts")
vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan")
```

Here we're using SpeechT5 checkpoint trained specifically for English TTS. Should you wish to translate into a language other than English, either swap the checkpoint for a SpeechT5 TTS model fine-tuned on your language of choice, or use an MMS TTS checkpoint pre-trained in your target langauge.

As with the Whisper model, we'll place the SpeechT5 model and vocoder on our GPU accelerator device if we have one:

```
model.to(device)
vocoder.to(device)
```

Great! Let's load up the speaker embeddings:

```
embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors",
split="validation")
speaker_embeddings = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)
```

We can now write a function that takes a text prompt as input, and generates the corresponding speech. We'll first pre-process the text input using the SpeechT5 processor, tokenizing the text to get our input ids. We'll then pass the input ids and speaker embeddings to the SpeechT5 model, placing each on the accelerator device if available. Finally, we'll return the generated speech, bringing it back to the CPU so that we can play it back in our ipynb notebook:

```
def synthesise(text):
    inputs = processor(text=text, return_tensors="pt")
    speech = model.generate_speech(
        inputs["input_ids"].to(device), speaker_embeddings.to(device),
vocoder=vocoder
    )
    return speech.cpu()
```

Let's check it works with a dummy text input:

```
speech = synthesise("Hey there! This is a test!")

Audio(speech, rate=16000)
```

Sounds good! Now for the exciting part - piecing it all together.

Creating a STST demo

Before we create a Gradio demo to showcase our STST system, let's first do a quick sanity check to make sure we can concatenate the two models, putting an audio sample in and getting an audio sample out. We'll do this by concatenating the two functions we defined in the previous two sub-sections, such that we input the source audio and retrieve the translated text, then synthesise the translated text to get the translated speech. Finally, we'll convert the synthesised speech to an `int16` array, which is the output audio file format expected by Gradio. To do this, we first have to normalise the audio array by the dynamic range of the target dtype (`int16`), and then convert from the default NumPy dtype (`float64`) to the target dtype (`int16`):

```
target_dtype = np.int16
max_range = np.iinfo(target_dtype).max


def speech_to_speech_translation(audio):
    translated_text = translate(audio)
    synthesised_speech = synthesise(translated_text)
    synthesised_speech = (synthesised_speech.numpy() * max_range).astype(np.int16)
    return 16000, synthesised_speech
```

Let's check this concatenated function gives the expected result:

```
sampling_rate, synthesised_speech = speech_to_speech_translation(sample["audio"])

Audio(synthesised_speech, rate=sampling_rate)
```

Perfect! Now we'll wrap this up into a nice Gradio demo so that we can record our source speech using a microphone input or file input and playback the system's prediction:

```
demo = gr.Blocks()

mic_translate = gr.Interface(
    fn=speech_to_speech_translation,
    inputs=gr.Audio(source="microphone", type="filepath"),
    outputs=gr.Audio(label="Generated Speech", type="numpy"),
)

file_translate = gr.Interface(
    fn=speech_to_speech_translation,
    inputs=gr.Audio(source="upload", type="filepath"),
    outputs=gr.Audio(label="Generated Speech", type="numpy"),
)

with demo:
```

```
    gr.TabbedInterface([mic_translate, file_translate], ["Microphone", "Audio
File"])

demo.launch(debug=True)
```

This will launch a Gradio demo similar to the one running on the Hugging Face Space:

You can duplicate this demo and adapt it to use a different Whisper checkpoint, a different TTS checkpoint, or relax the constraint of outputting English speech and follow the tips provide for translating into a langauge of your choice!

Going forwards

While the cascaded system is a compute and data efficient way of building a STST system, it suffers from the issues of error propagation and additive latency described above. Recent works have explored a *direct* approach to STST, one that does not predict an intermediate text output and instead maps directly from source speech to target speech. These systems are also capable of retaining the speaking characteristics of the source speaker in the target speech (such a prosody, pitch and intonation). If you're interested in finding out more about these systems, check-out the resources listed in the section on supplemental reading.

Supplemental reading and resources

This Unit pieced together many components from previous units, introducing the tasks of speech-to-speech translation, voice assistants and speaker diarization. The supplemental reading material is thus split into these three new tasks for your convenience:

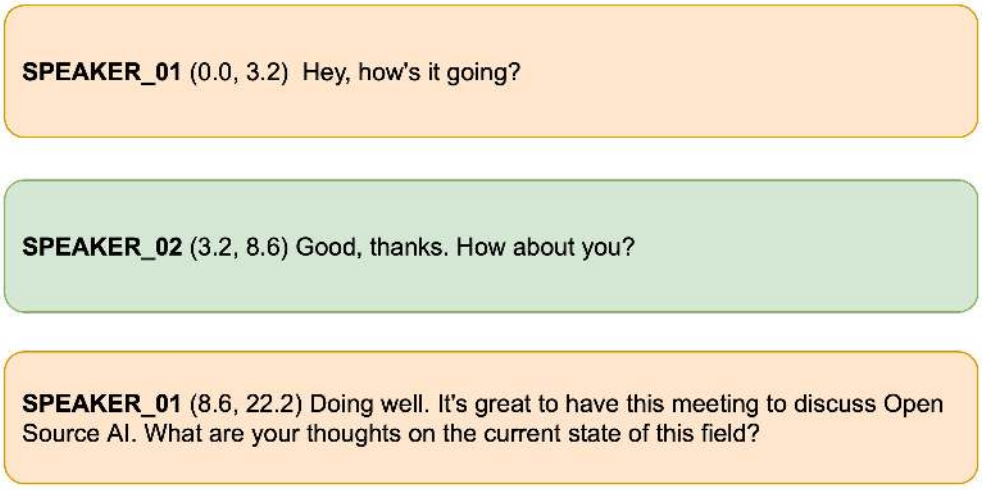
Speech-to-speech translation: * STST with discrete units by Meta AI: a direct approach to STST through encoder-decoder models * Hokkien direct speech-to-speech translation by Meta AI: a direct approach to STST using encoder-decoder models with a two-stage decoder * Leveraging unsupervised and weakly-supervised data to improve direct STST by Google: proposes new approaches for leveraging unsupervised and weakly supervised data for training direct STST models and a small change to the Transformer architecture * Translatotron-2 by Google: a system that is able to retain speaker characteristics in translated speech

Voice Assistant: * Accurate wakeword detection by Amazon: a low latency approach for wakeword detection for on-device applications * RNN-Transducer Architecture by Google: a modification to the CTC architecture for streaming on-device ASR

Meeting Transcriptions: * pyannote.audio Technical Report by Hervé Bredin: this report describes the main principles behind the `pyannote.audio` speaker diarization pipeline * Whisper X by Max Bain et al.: a superior approach to computing word-level timestamps using the Whisper model

Transcribe a meeting

In this final section, we'll use the Whisper model to generate a transcription for a conversation or meeting between two or more speakers. We'll then pair it with a *speaker diarization* model to predict "who spoke when". By matching the timestamps from the Whisper transcriptions with the timestamps from the speaker diarization model, we can predict an end-to-end meeting transcription with fully formatted start / end times for each speaker. This is a basic version of the meeting transcription services you might have seen online from the likes of Otter.ai and co:



Speaker Diarization

Speaker diarization (or diarisation) is the task of taking an unlabelled audio input and predicting "who spoke when". In doing so, we can predict start / end timestamps for each speaker turn, corresponding to when each speaker starts speaking and when they finish.

🤗 Transformers currently does not have a model for speaker diarization included in the library, but there are checkpoints on the Hub that can be used with relative ease. In this example, we'll use the pre-trained speaker diarization model from pyannote.audio. Let's get started and pip install the package:

```
pip install --upgrade pyannote.audio
```

Great! The weights for this model are hosted on the Hugging Face Hub. To access them, we first have to agree to the speaker diarization model's terms of use: pyannote/speaker-diarization. And subsequently the segmentation model's terms of use: pyannote/segmentation.

Once complete, we can load the pre-trained speaker diarization pipeline locally on our device:

```
from pyannote.audio import Pipeline

diarization_pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization@2.1", use_auth_token=True
)
```

Let's try it out on a sample audio file! For this, we'll load a sample of the LibriSpeech ASR dataset that consists of two different speakers that have been concatenated together to give a single audio file:

```
from datasets import load_dataset

concatenated_librispeech = load_dataset(
    "sanchit-gandhi/concatenated_librispeech", split="train", streaming=True
)
sample = next(iter(concatenated_librispeech))
```

We can listen to the audio to see what it sounds like:

```
from IPython.display import Audio

Audio(sample["audio"]["array"], rate=sample["audio"]["sampling_rate"])
```

Cool! We can clearly hear two different speakers, with a transition roughly 15s of the way through. Let's pass this audio file to the diarization model to get the speaker start / end times. Note that pyannote.audio expects the audio input to be a PyTorch tensor of shape `(channels, seq_len)`, so we need to perform this conversion prior to running the model:

```
input_tensor = torch.from_numpy(sample["audio"]["array"][None, :]).float()
outputs = diarization_pipeline(

)

outputs.for_json()["content"]
```

```
[,
  'track': 'B',
  'label': 'SPEAKER_01'},
 ,
  'track': 'A',
  'label': 'SPEAKER_00'}]
```

This looks pretty good! We can see that the first speaker is predicted as speaking up until the 14.5 second mark, and the second speaker from 15.4s onwards. Now we need to get our transcription!

Speech transcription

For the third time in this Unit, we'll use the Whisper model for our speech transcription system. Specifically, we'll load the Whisper Base checkpoint, since it's small enough to give good inference speed with reasonable transcription accuracy. As before, feel free to use any speech recognition checkpoint on the Hub, including Wav2Vec2, MMS ASR or other Whisper checkpoints:

```
from transformers import pipeline

asr_pipeline = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-base",
)
```

Let's get the transcription for our sample audio, returning the segment level timestamps as well so that we know the start / end times for each segment. You'll remember from Unit 5 that we need to pass the argument `return_timestamps=True` to activate the timestamp prediction task for Whisper:

```
asr_pipeline(
    sample["audio"].copy(),
    generate_kwargs=,
    return_timestamps=True,
)
```

```
,
        ,
        ,
        ,
        ,
        ,
    ],
}
```

Alright! We see that each segment of the transcript has a start and end time, with the speakers changing at the 15.48 second mark. We can now pair this transcription with the speaker timestamps that we got from our diarization model to get our final transcription.

Speechbox

To get the final transcription, we'll align the timestamps from the diarization model with those from the Whisper model. The diarization model predicted the first speaker to end at 14.5 seconds, and the second speaker to start at 15.4s, whereas Whisper predicted segment boundaries at 13.88, 15.48 and 19.44 seconds respectively. Since the timestamps from Whisper don't match perfectly with those from the diarization model, we need to find which of these boundaries are closest to 14.5 and 15.4 seconds, and segment the transcription by speakers accordingly. Specifically, we'll find the closest alignment between diarization and transcription timestamps by minimising the absolute distance between both.

Luckily for us, we can use the 🤗 Speechbox package to perform this alignment. First, let's pip install `speechbox` from main:

```
pip install git+https://github.com/huggingface/speechbox
```

We can now instantiate our combined diarization plus transcription pipeline, by passing the diarization model and ASR model to the `ASRDiarizationPipeline` class:

```
from speechbox import ASRDiarizationPipeline

pipeline = ASRDiarizationPipeline(
    asr_pipeline=asr_pipeline, diarization_pipeline=diarization_pipeline
)
```

You can also instantiate the `ASRDiarizationPipeline` directly from pre-trained by specifying the model id of an ASR model on the Hub:

`pipeline = ASRDiarizationPipeline.from_pretrained("openai/whisper-base")`

Let's pass the audio file to the composite pipeline and see what we get out:

```
pipeline(sample["audio"].copy())
```

```
[,
 ]
```

Excellent! The first speaker is segmented as speaking from 0 to 15.48 seconds, and the second speaker from 15.48 to 21.28 seconds, with the corresponding transcriptions for each.

We can format the timestamps a little more nicely by defining two helper functions. The first converts a tuple of timestamps to a string, rounded to a set number of decimal places. The second combines the speaker id, timestamp and text information onto one line, and splits each speaker onto their own line for ease of reading:

```
def tuple_to_string(start_end_tuple, ndigits=1):
    return str((round(start_end_tuple[0], ndigits), round(start_end_tuple[1],
ndigits)))


def format_as_transcription(raw_segments):
    return "\n\n".join(
        [
            chunk["speaker"] + " " + tuple_to_string(chunk["timestamp"]) +
chunk["text"]
            for chunk in raw_segments
        ]
    )
```

Let's re-run the pipeline, this time formatting the transcription according to the function we've just defined:

```
outputs = pipeline(sample["audio"].copy())

format_as_transcription(outputs)
```

```
SPEAKER_01 (0.0, 15.5) The second and importance is as follows. Sovereignty may be
defined to be the right of making laws.
In France, the king really exercises a portion of the sovereign power, since the
laws have no weight.

SPEAKER_00 (15.5, 21.3) He was in a favored state of mind, owing to the blight his
```

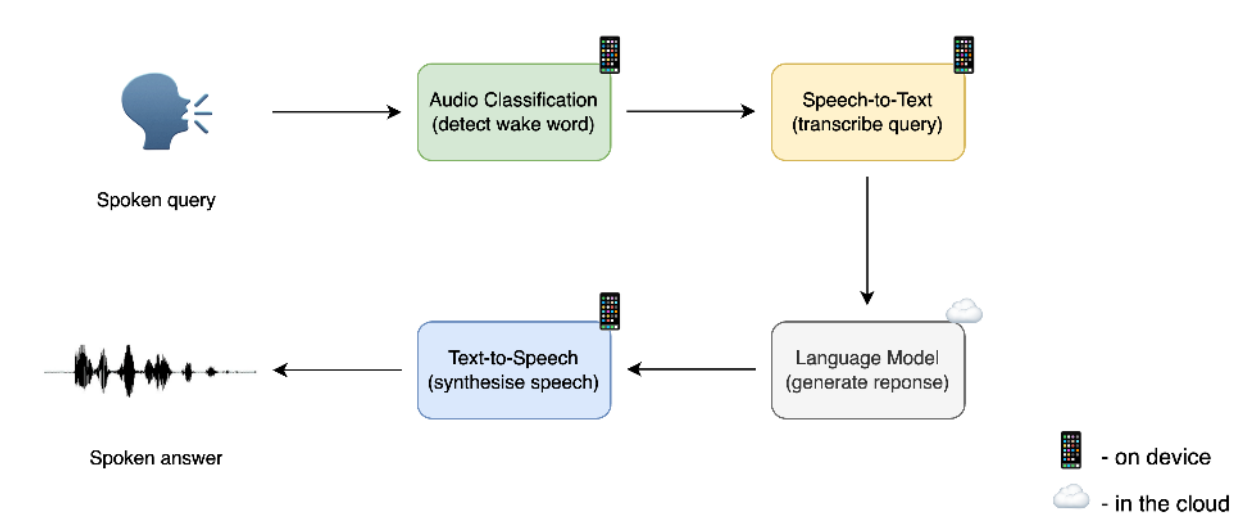
```
wife's action threatened to cast upon
his entire future.
```

There we go! With that, we've both diarized and transcribe our input audio and returned speaker-segmented transcriptions. While the minimum distance algoirthm to align the diarized timestamps and transcribed timestamps is simple, it works well in practice. If you want to explore more advanced methods for combining the timestamps, the source code for the `ASRDiarizationPipeline` is a good place to start: speechbox/diarize.py

Creating a voice assistant

In this section, we'll piece together three models that we've already had hands-on experience with to build an end-to-end voice assistant called **Marvin** 🤖. Like Amazon's Alexa or Apple's Siri, Marvin is a virtual voice assistant who responds to a particular 'wake word', then listens out for a spoken query, and finally responds with a spoken answer.

We can break down the voice assistant pipeline into four stages, each of which requires a standalone model:



1. Wake word detection

Voice assistants are constantly listening to the audio inputs coming through your device's microphone, however they only boot into action when a particular 'wake word' or 'trigger word' is spoken.

The wake word detection task is handled by a small on-device audio classification model, which is much smaller and lighter than the speech recognition model, often only several millions of parameters compared to several hundred millions for speech recognition. Thus, it can be run continuously on your device without draining your battery. Only when the wake word is detected is the larger speech recognition model launched, and afterwards it is shut down again.

2. Speech transcription

The next stage in the pipeline is transcribing the spoken query to text. In practice, transferring audio files from your local device to the Cloud is slow due to the large nature of audio files, so it's more efficient to transcribe them directly using an automatic speech recognition (ASR) model on-device rather than using a model in the Cloud. The on-device model might be smaller and thus less accurate than one hosted in the Cloud, but the faster inference speed makes it worthwhile since we can run speech recognition in near real-time, our spoken audio utterance being transcribed as we say it.

We're very familiar with the speech recognition process now, so this should be a piece of cake!

3. Language model query

Now that we know what the user asked, we need to generate a response! The best candidate models for this task are *large language models (LLMs)*, since they are effectively able to understand the semantics of the text query and generate a suitable response.

Since our text query is small (just a few text tokens), and language models large (many billions of parameters), the most efficient way of running LLM inference is to send our text query from our device to an LLM running in the Cloud, generate a text response, and return the response back to the device.

4. Synthesise speech

Finally, we'll use a text-to-speech (TTS) model to synthesise the text response as spoken speech. This is done on-device, but you could feasibly run a TTS model in the Cloud, generating the audio output and transferring it back to the device.

Again, we've done this several times now, so the process will be very familiar!

The following section requires the use of a microphone to record a voice input. Since Google Colab machines do not have microphone compatibility, it is recommended to

run this section locally, either on your CPU, or on a GPU if you have local access. The checkpoint sizes have been selected as those small enough to run adequately fast on CPU, so you will still get good performance without a GPU.

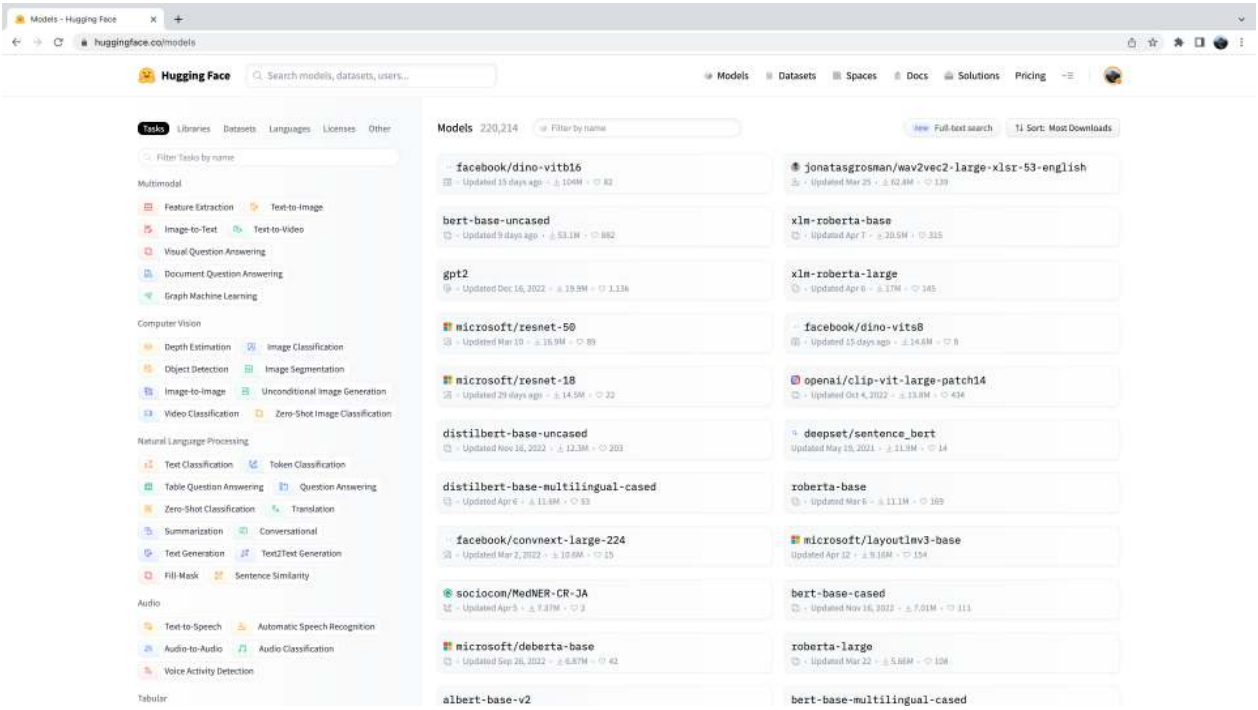
Wake word detection

The first stage in the voice assistant pipeline is detecting whether the wake word was spoken, and we need to find ourselves an appropriate pre-trained model for this task! You'll remember from the section on pre-trained models for audio classification that Speech Commands is a dataset of spoken words designed to evaluate audio classification models on 15+ simple command words like `"up"`, `"down"`, `"yes"` and `"no"`, as well as a `"silence"` label to classify no speech. Take a minute to listen through the samples on the datasets viewer on the Hub and re-acquaint yourself with the Speech Commands dataset: datasets viewer.

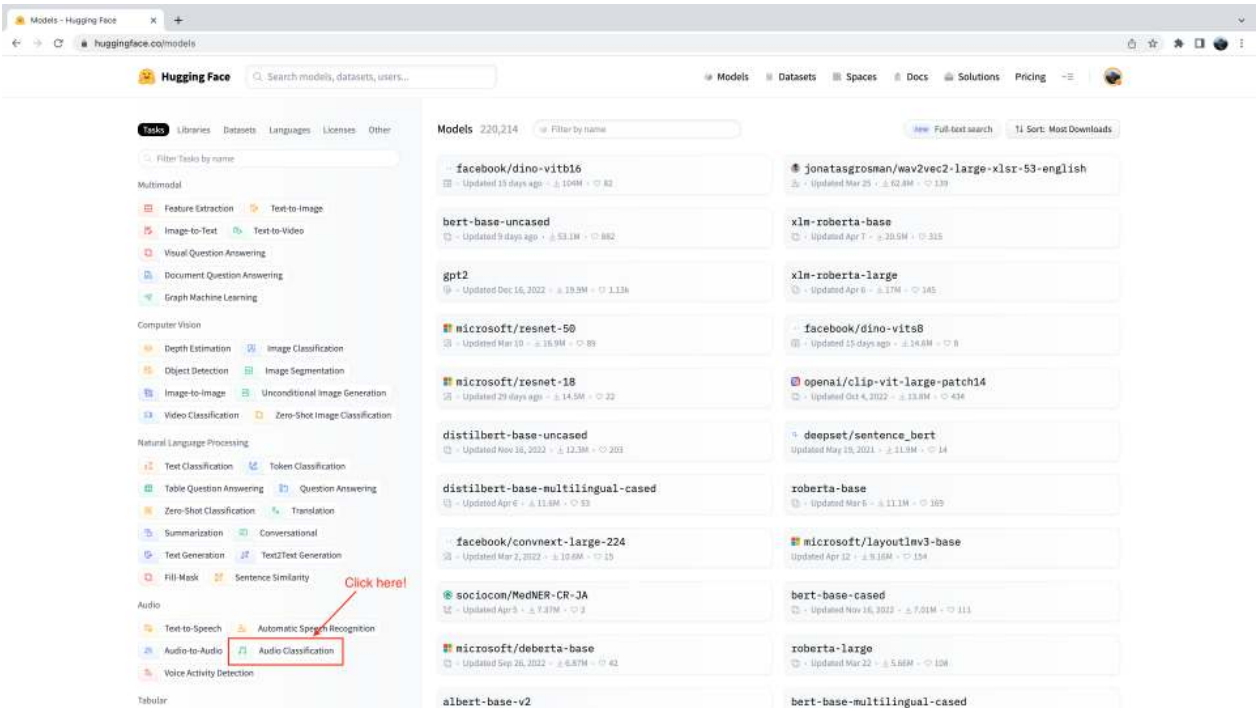
We can take an audio classification model pre-trained on the Speech Commands dataset and pick one of these simple command words to be our chosen wake word. Out of the 15+ possible command words, if the model predicts our chosen wake word with the highest probability, we can be fairly certain that the wake word has been said.

Let's head to the Hugging Face Hub and click on the "Models" tab: https://huggingface.co/models

This is going to bring up all the models on the Hugging Face Hub, sorted by downloads in the past 30 days:

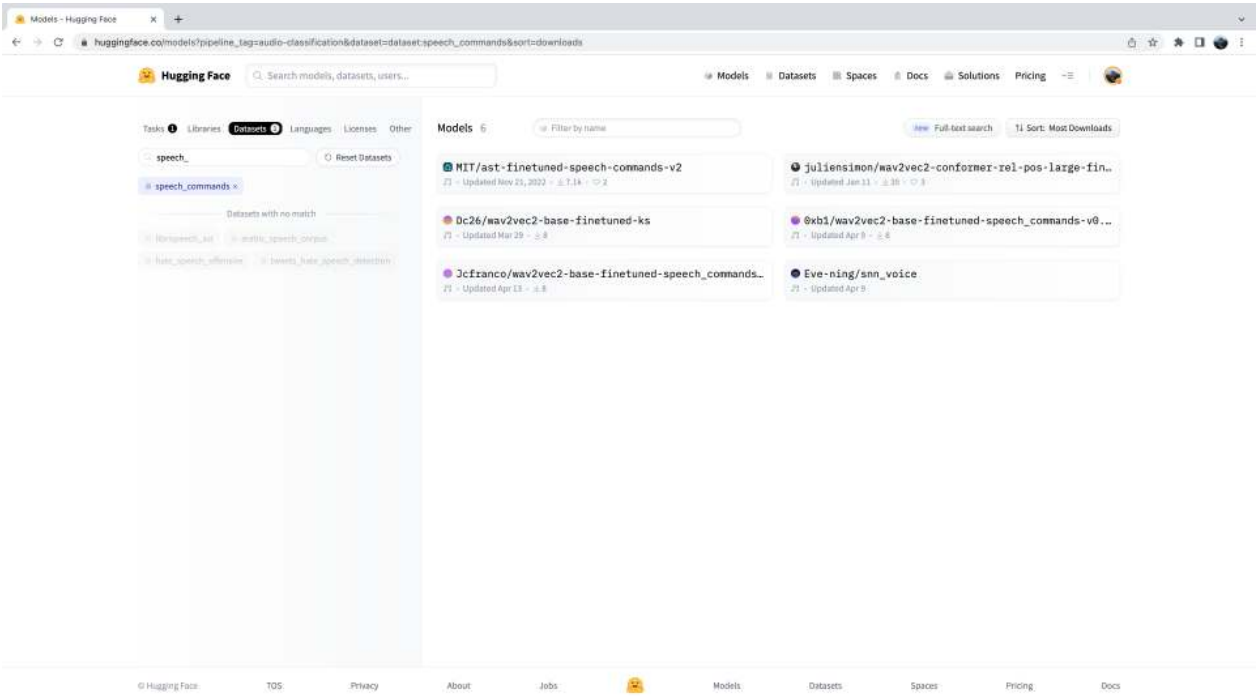


You'll notice on the left-hand side that we have a selection of tabs that we can select to filter models by task, library, dataset, etc. Scroll down and select the task "Audio Classification" from the list of audio tasks:



We're now presented with the sub-set of 500+ audio classification models on the Hub. To further refine this selection, we can filter models by dataset. Click on the tab "Datasets", and in the search box type "speech_commands". As you begin typing, you'll see the selection for `speech_commands` appear underneath the search tab. You

can click this button to filter all audio classification models to those fine-tuned on the Speech Commands dataset:



Great! We see that we have six pre-trained models available to us for this specific dataset and task (although there may be new models added if you're reading at a later date!). You'll recognise the first of these models as the Audio Spectrogram Transformer checkpoint that we used in Unit 4 example. We'll use this checkpoint again for our wake word detection task.

Let's go ahead and load the checkpoint using the `pipeline` class:

```
from transformers import pipeline


device = "cuda:0" if torch.cuda.is_available() else "cpu"

classifier = pipeline(
    "audio-classification", model="MIT/ast-finetuned-speech-commands-v2",
device=device
)
```

We can check what labels the model was trained on by checking the `id2label` attribute in the model config:

```
classifier.model.config.id2label
```

Alright! We see that the model was trained on 35 class labels, including some simple command words that we described above, as well as some particular objects like `"bed"`, `"house"` and `"cat"`. We see that there is one name in these class labels: id 27 corresponds to the label **"marvin"**:

```
classifier.model.config.id2label[27]
```

```
'marvin'
```

Perfect! We can use this name as our wake word for our voice assistant, similar to how "Alexa" is used for Amazon's Alexa, or "Hey Siri" is used for Apple's Siri. Of all the possible labels, if the model predicts `"marvin"` with the highest class probability, we can be fairly sure that our chosen wake word has been said.

Now we need to define a function that is constantly listening to our device's microphone input, and continuously passes the audio to the classification model for inference. To do this, we'll use a handy helper function that comes with 🤗 Transformers called `ffmpeg_microphone_live`.

This function forwards small chunks of audio of specified length `chunk_length_s` to the model to be classified. To ensure that we get smooth boundaries across chunks of audio, we run a sliding window across our audio with stride `chunk_length_s / 6`. So that we don't have to wait for the entire first chunk to be recorded before we start inferring, we also define a minimal temporary audio input length `stream_chunk_s` that is forwarded to the model before `chunk_length_s` time is reached.

The function `ffmpeg_microphone_live` returns a *generator* object, yielding a sequence of audio chunks that can each be passed to the classification model to make a prediction. We can pass this generator directly to the `pipeline`, which in turn returns a sequence of output predictions, one for each chunk of audio input. We can inspect the class label probabilities for each audio chunk, and stop our wake word detection loop when we detect that the wake word has been spoken.

We'll use a very simple criteria for classifying whether our wake word was spoken: if the class label with the highest probability was our wake word, and this probability exceeds a threshold `prob_threshold`, we declare that the wake word as having been spoken. Using a probability threshold to gate our classifier this way ensures that the wake word is not erroneously predicted if the audio input is noise, which is typically when the model is very uncertain and all the class label probabilities low. You might want to tune this probability threshold, or explore more sophisticated means for the wake word decision through an *entropy* (or uncertainty) based metric.

```
from transformers.pipelines.audio_utils import ffmpeg_microphone_live


def launch_fn(
    wake_word="marvin",
    prob_threshold=0.5,
    chunk_length_s=2.0,
    stream_chunk_s=0.25,
    debug=False,
):
    if wake_word not in classifier.model.config.label2id.keys():
        raise ValueError(
            f"Wake word  not in set of valid class labels, pick a wake word in the
set ."
        )

    sampling_rate = classifier.feature_extractor.sampling_rate

    mic = ffmpeg_microphone_live(
        sampling_rate=sampling_rate,
        chunk_length_s=chunk_length_s,
        stream_chunk_s=stream_chunk_s,
    )

    print("Listening for wake word...")
    for prediction in classifier(mic):
        prediction = prediction[0]
        if debug:
            print(prediction)
        if prediction["label"] == wake_word:
            if prediction["score"] > prob_threshold:
                return True
```

Let's give this function a try to see how it works! We'll set the flag `debug=True` to print out the prediction for each chunk of audio. Let the model run for a few seconds to see the kinds of predictions that it makes when there is no speech input, then clearly say the wake word `"marvin"` and watch the class label prediction for `"marvin"` spike to near 1:

```
launch_fn(debug=True)
```

```
Listening for wake word...
```

Awesome! As we expect, the model generates garbage predictions for the first few seconds. There is no speech input, so the model makes close to random predictions, but with very low probability. As soon as we say the wake word, the model predicts `"marvin"` with probability close to 1 and terminates the loop, signalling that the wake word has been detected and that the ASR system should be activated!

Speech transcription

Once again, we'll use the Whisper model for our speech transcription system. Specifically, we'll load the Whisper Base English checkpoint, since it's small enough to give good inference speed with reasonable transcription accuracy. We'll use a trick to get near real-time transcription by being clever with how we forward our audio inputs to the model. As before, feel free to use any speech recognition checkpoint on the Hub, including Wav2Vec2, MMS ASR or other Whisper checkpoints:

```
transcriber = pipeline(
    "automatic-speech-recognition", model="openai/whisper-base.en", device=device
)
```

If you're using a GPU, you can increase the checkpoint size to use the Whisper Small English checkpoint, which will return better transcription accuracy and still be within the required latency threshold. Simply swap the model id to: `"openai/whisper-small.en"`.

We can now define a function to record our microphone input and transcribe the corresponding text. With the `ffmpeg_microphone_live` helper function, we can control how 'real-time' our speech recognition model is. Using a smaller `stream_chunk_s` lends itself to more real-time speech recognition, since we divide our input audio into smaller chunks and transcribe them on the fly. However, this comes at the expense of poorer accuracy, since there's less context for the model to infer from.

As we're transcribing the speech, we also need to have an idea of when the user **stops** speaking, so that we can terminate the recording. For simplicity, we'll terminate our microphone recording after the first `chunk_length_s` (which is set to 5 seconds by default), but you can experiment with using a voice activity detection (VAD) model to predict when the user has stopped speaking.

```
def transcribe(chunk_length_s=5.0, stream_chunk_s=1.0):
    sampling_rate = transcriber.feature_extractor.sampling_rate

    mic = ffmpeg_microphone_live(
        sampling_rate=sampling_rate,
        chunk_length_s=chunk_length_s,
        stream_chunk_s=stream_chunk_s,
    )

    print("Start speaking...")
    for item in transcriber(mic, generate_kwargs=):
        sys.stdout.write("\033[K")
        print(item["text"], end="\r")
```

```
        if not item["partial"][0]:
            break

        return item["text"]
```

Let's give this a go and see how we get on! Once the microphone is live, start speaking and watch your transcription appear in semi real-time:

```
transcribe()
```

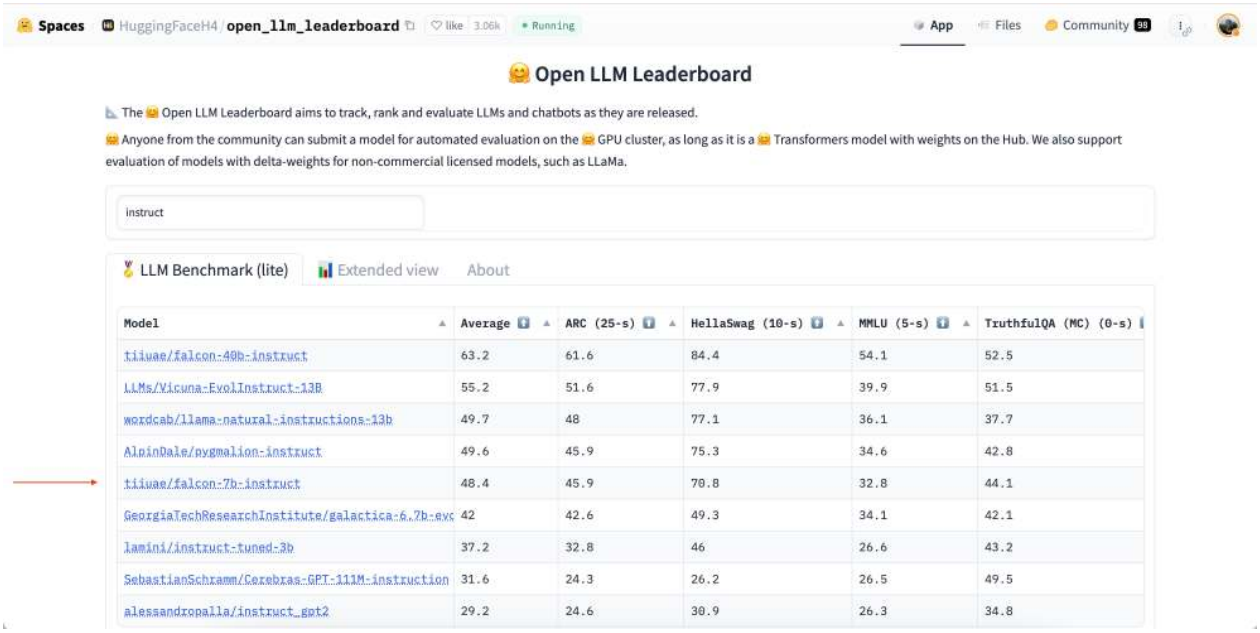
```
Start speaking...
 Hey, this is a test with the whisper model.
```

Nice! You can adjust the maximum audio length `chunk_length_s` based on how fast or slow you speak (increase it if you felt like you didn't have enough time to speak, decrease it if you were left waiting at the end), and the `stream_chunk_s` for the real-time factor. Just pass these as arguments to the `transcribe` function.

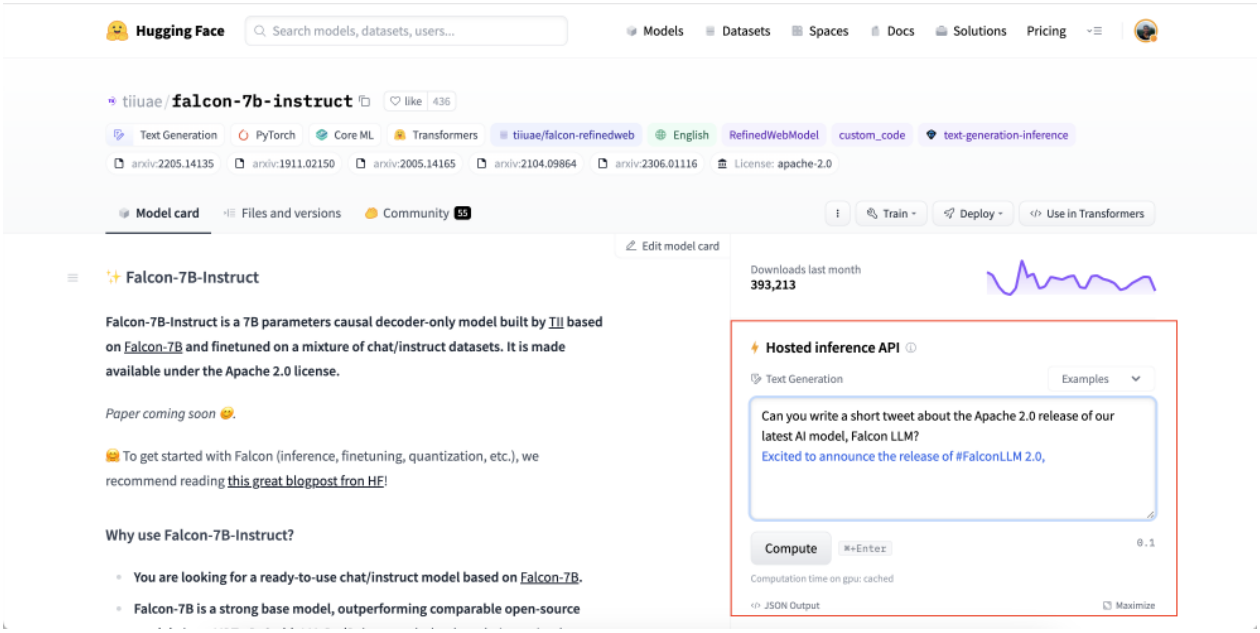
Language model query

Now that we have our spoken query transcribed, we want to generate a meaningful response. To do this, we'll use an LLM hosted on the Cloud. Specifically, we'll pick an LLM on the Hugging Face Hub and use the Inference API to easily query the model.

First, let's head over to the Hugging Face Hub. To find our LLM, we'll use the 🤗 Open LLM Leaderboard, a Space that ranks LLM models by performance over four generation tasks. We'll search by "instruct" to filter out models that have been instruction fine-tuned, since these should work better for our querying task:



We'll use the tiiuae/falcon-7b-instruct checkpoint by TII, a 7B parameter decoder-only LM fine-tuned on a mixture of chat and instruction datasets. You can use any LLM on the Hugging Face Hub that has the "Hosted inference API" enabled, just look out for the widget on the right-side of the model card:



The Inference API allows us to send a HTTP request from our local machine to the LLM hosted on the Hub, and returns the response as a `json` file. All we need to provide is our Hugging Face Hub token (which we retrieve directly from our Hugging Face Hub folder) and the model id of the LLM we wish to query:

```
from huggingface_hub import HfFolder
```

```
def query(text, model_id="tiiuae/falcon-7b-instruct"):
    api_url = f"https://api-inference.huggingface.co/models/"
    headers = "}
    payload =

    print(f"Querying...: ")
    response = requests.post(api_url, headers=headers, json=payload)
    return response.json()[0]["generated_text"][len(text) + 1 :]
```

Let's give it a try with a test input!

```
query("What does Hugging Face do?")
```

```
'Hugging Face is a company that provides natural language processing and machine
learning tools for developers. They'
```

You'll notice just how fast inference is using the Inference API - we only have to send a small number of text tokens from our local machine to the hosted model, so the communication cost is very low. The LLM is hosted on GPU accelerators, so inference runs very quickly. Finally, the generated response is transferred back from the model to our local machine, again with low communication overhead.

Synthesise speech

And now we're ready to get the final spoken output! Once again, we'll use the Microsoft SpeechT5 TTS model for English TTS, but you can use any TTS model of your choice. Let's go ahead and load the processor and model:

```
from transformers import SpeechT5Processor, SpeechT5ForTextToSpeech,
SpeechT5HifiGan

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")

model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/
speecht5_tts").to(device)
vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan").to(device)
```

And also the speaker embeddings:

```
from datasets import load_dataset

embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors",
```

```
split="validation")
speaker_embeddings = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)
```

We'll re-use the `synthesise` function that we defined in the previous chapter on Speech-to-speech translation:

```
def synthesise(text):
    inputs = processor(text=text, return_tensors="pt")
    speech = model.generate_speech(
        inputs["input_ids"].to(device), speaker_embeddings.to(device),
vocoder=vocoder
    )
    return speech.cpu()
```

Let's quickly verify this works as expected:

```
from IPython.display import Audio

audio = synthesise(
    "Hugging Face is a company that provides natural language processing and
machine learning tools for developers."
)

Audio(audio, rate=16000)
```

Nice job 👍

Marvin 🤖

Now that we've defined a function for each of the four stages of the voice assistant pipeline, all that's left to do is piece them together to get our end-to-end voice assistant. We'll simply concatenate the four stages, starting with wake word detection (`launch_fn`), speech transcription, querying the LLM, and finally speech synthesis.

```
launch_fn()
transcription = transcribe()
response = query(transcription)
audio = synthesise(response)

Audio(audio, rate=16000, autoplay=True)
```

Try it out with a few prompts! Here are some examples to get you started: * *What is the hottest country in the world? * How do Transformer models work? * Do you know Spanish?*

And with that, we have our end-to-end voice assistant complete, made using the 🤗 audio tools you've learnt throughout this course, with a sprinkling of LLM magic at the end. There are several extensions that we could make to improve the voice assistant. Firstly, the audio classification model classifies 35 different labels. We could use a smaller, more lightweight binary classification model that only predicts whether the wake word was spoken or not. Secondly, we pre-load all the models ahead and keep them running on our device. If we wanted to save power, we would only load each model at the time it was required, and subsequently un-load them afterwards. Thirdly, we're missing a voice activity detection model in our transcription function, transcribing for a fixed amount of time, which in some cases is too long, and in others too short.

Generalise to anything 🪄

So far, we've seen how we can generate speech outputs with our voice assistant Marvin. To finish, we'll demonstrate how we can generalise these speech outputs to text, audio and image.

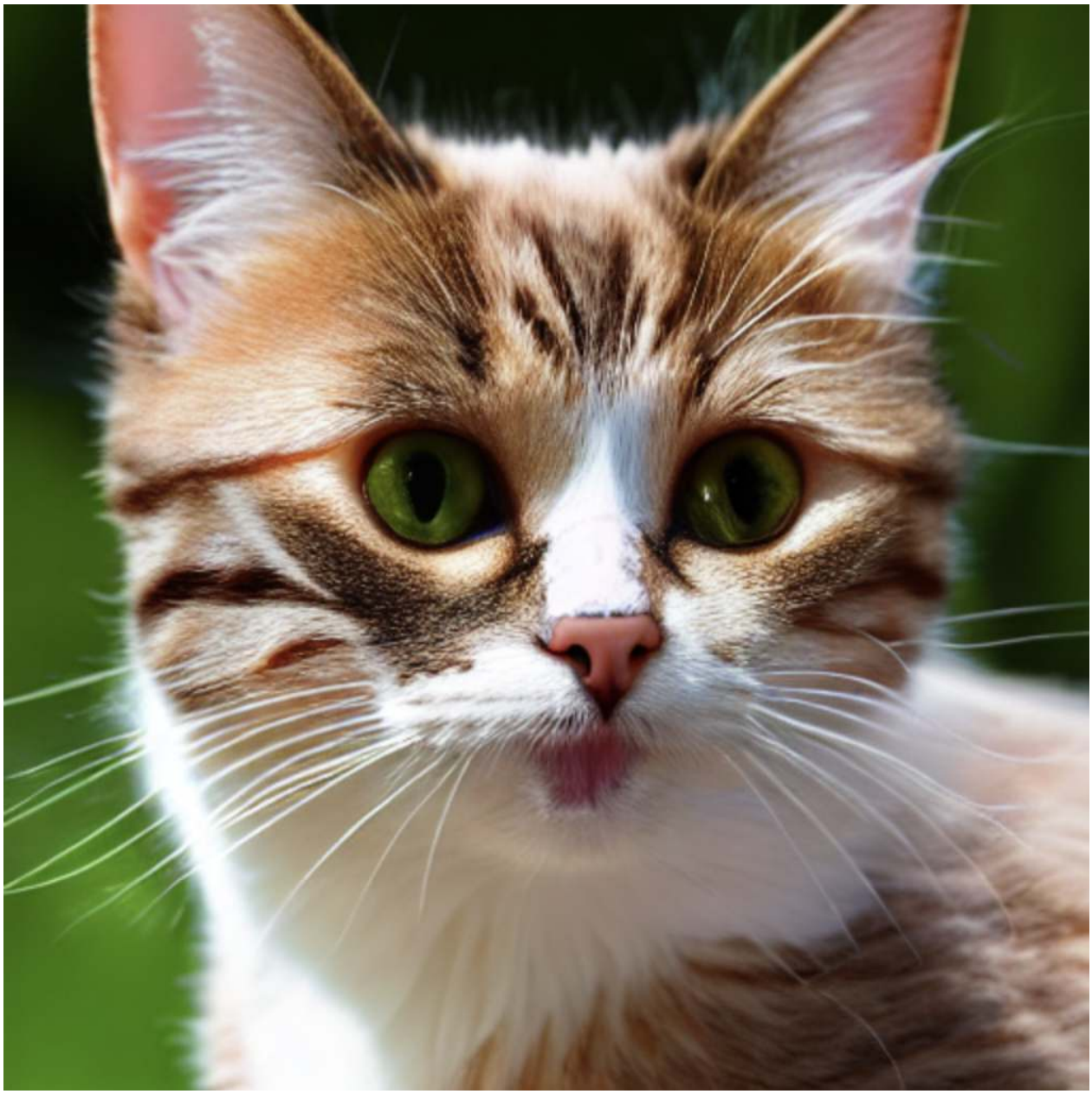
We'll use Transformers Agents to build our assistant. Transformers Agents provides a natural language API on top of the 🤗 Transformers and Diffusers libraries, interpreting a natural language input using an LLM with carefully crafted prompts, and using a set of curated tools to provide multimodal outputs.

Let's go ahead and instantiate an agent. There are three LLMs available for Transformers Agents, two of which are open-source and free on the Hugging Face Hub. The third is a model from OpenAI that requires an OpenAI API key. We'll use the free Bigcode Starcoder model in this example, but you can also try either of the other LLMs available:

```
from transformers import HfAgent

agent = HfAgent(
    url_endpoint="https://api-inference.huggingface.co/models/bigcode/starcoder"
)
```

To use the agent, we simply have to call `agent.run` with our text prompt. As an example, we'll get it to generate an image of a cat 🐈 (that hopefully looks a bit better than this emoji):

```
agent.run("Generate an image of a cat")
```



Note that the first time calling this will trigger the model weights to be downloaded, which might take some time depending on your Hub download speed.

Easy as that! The Agent interpreted our prompt, and used Stable Diffusion under the hood to generate the image, without us having to worry about loading the model, writing the function or executing the code.

We can now replace our LLM query function and text synthesis step with our Transformers Agent in our voice assistant, since the Agent is going to take care of both of these steps for us:

```
launch_fn()
transcription = transcribe()
agent.run(transcription)
```

Try speaking the same prompt "Generate an image of a cat" and see how the system gets on. If you ask the Agent a simple question / answer query, the Agent will respond with a text answer. You can encourage it to generate multimodal outputs by asking it to return an image or speech. For example, you can ask it to: "Generate an image of a cat, caption it, and speak the caption".

While the Agent is more flexible than our first iteration Marvin 🤖 assistant, generalising the voice assistant task in this way may lead to inferior performance on standard voice assistant queries. To recover performance, you can try using a more performant LLM checkpoint, such as the one from OpenAI, or define a set of custom tools that are specific to the voice assistant task.

Introduction to audio data

By nature, a sound wave is a continuous signal, meaning it contains an infinite number of signal values in a given time. This poses problems for digital devices which expect finite arrays. To be processed, stored, and transmitted by digital devices, the continuous sound wave needs to be converted into a series of discrete values, known as a digital representation.

If you look at any audio dataset, you'll find digital files with sound excerpts, such as text narration or music. You may encounter different file formats such as `.wav` (Waveform Audio File), `.flac` (Free Lossless Audio Codec) and `.mp3` (MPEG-1 Audio Layer 3). These formats mainly differ in how they compress the digital representation of the audio signal.

Let's take a look at how we arrive from a continuous signal to this representation. The analog signal is first captured by a microphone, which converts the sound waves into an electrical signal. The electrical signal is then digitized by an Analog-to-Digital Converter to get the digital representation through sampling.

Sampling and sampling rate

Sampling is the process of measuring the value of a continuous signal at fixed time steps. The sampled waveform is *discrete*, since it contains a finite number of signal values at uniform intervals.

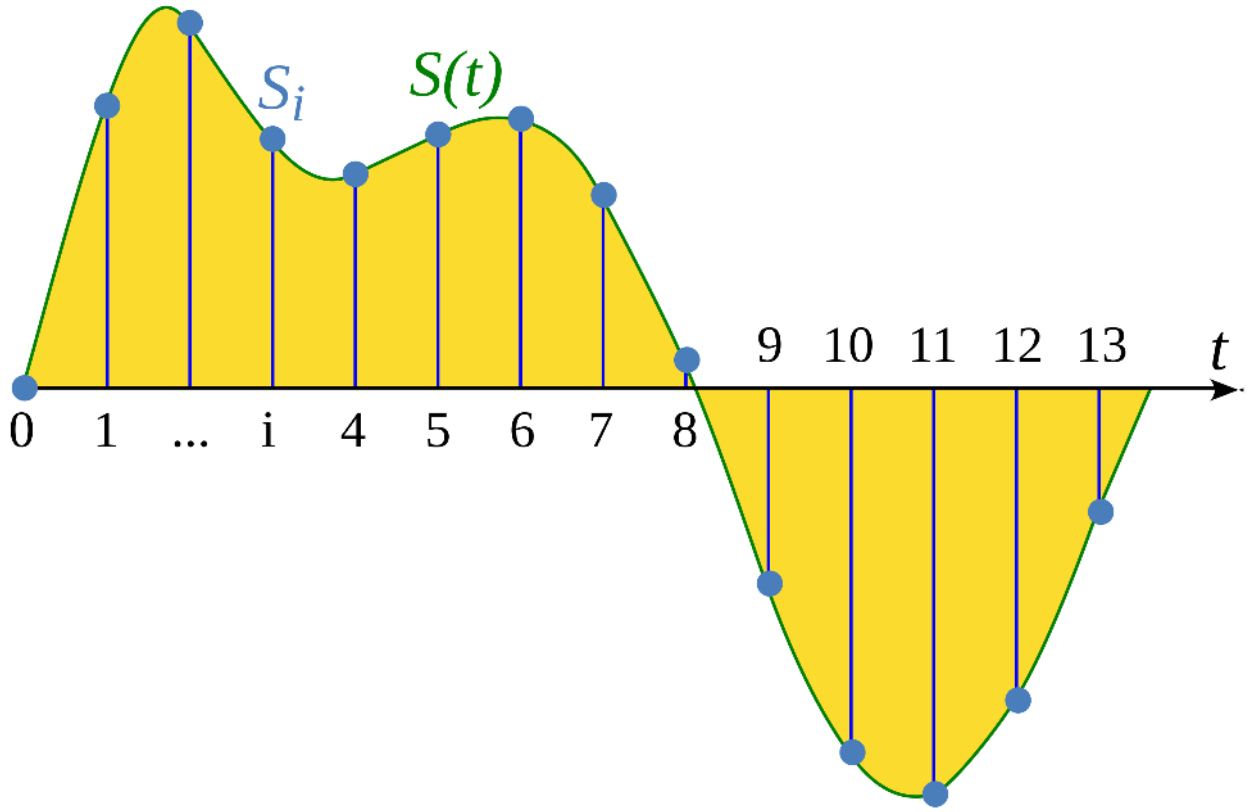


Illustration from Wikipedia article: Sampling (signal processing)

The **sampling rate** (also called sampling frequency) is the number of samples taken in one second and is measured in hertz (Hz). To give you a point of reference, CD-quality audio has a sampling rate of 44,100 Hz, meaning samples are taken 44,100 times per second. For comparison, high-resolution audio has a sampling rate of 192,000 Hz or 192 kHz. A common sampling rate used in training speech models is 16,000 Hz or 16 kHz.

The choice of sampling rate primarily determines the highest frequency that can be captured from the signal. This is also known as the Nyquist limit and is exactly half the sampling rate. The audible frequencies in human speech are below 8 kHz and therefore sampling speech at 16 kHz is sufficient. Using a higher sampling rate will not capture more information and merely leads to an increase in the computational cost of processing such files. On the other hand, sampling audio at too low a sampling rate will result in information loss. Speech sampled at 8 kHz will sound muffled, as the higher frequencies cannot be captured at this rate.

It's important to ensure that all audio examples in your dataset have the same sampling rate when working on any audio task. If you plan to use custom audio data to fine-tune a pre-trained model, the sampling rate of your data should match the sampling rate of the data the model was pre-trained on. The sampling rate

determines the time interval between successive audio samples, which impacts the temporal resolution of the audio data. Consider an example: a 5-second sound at a sampling rate of 16,000 Hz will be represented as a series of 80,000 values, while the same 5-second sound at a sampling rate of 8,000 Hz will be represented as a series of 40,000 values. Transformer models that solve audio tasks treat examples as sequences and rely on attention mechanisms to learn audio or multimodal representation. Since sequences are different for audio examples at different sampling rates, it will be challenging for models to generalize between sampling rates. **Resampling** is the process of making the sampling rates match, and is part of preprocessing the audio data.

Amplitude and bit depth

While the sampling rate tells you how often the samples are taken, what exactly are the values in each sample?

Sound is made by changes in air pressure at frequencies that are audible to humans. The **amplitude** of a sound describes the sound pressure level at any given instant and is measured in decibels (dB). We perceive the amplitude as loudness. To give you an example, a normal speaking voice is under 60 dB, and a rock concert can be at around 125 dB, pushing the limits of human hearing.

In digital audio, each audio sample records the amplitude of the audio wave at a point in time. The **bit depth** of the sample determines with how much precision this amplitude value can be described. The higher the bit depth, the more faithfully the digital representation approximates the original continuous sound wave.

The most common audio bit depths are 16-bit and 24-bit. Each is a binary term, representing the number of possible steps to which the amplitude value can be quantized when it's converted from continuous to discrete: 65,536 steps for 16-bit audio, a whopping 16,777,216 steps for 24-bit audio. Because quantizing involves rounding off the continuous value to a discrete value, the sampling process introduces noise. The higher the bit depth, the smaller this quantization noise. In practice, the quantization noise of 16-bit audio is already small enough to be inaudible, and using higher bit depths is generally not necessary.

You may also come across 32-bit audio. This stores the samples as floating-point values, whereas 16-bit and 24-bit audio use integer samples. The precision of a 32-bit floating-point value is 24 bits, giving it the same bit depth as 24-bit audio. Floating-point audio samples are expected to lie within the [-1.0, 1.0] range. Since machine learning models naturally work on floating-point data, the audio must first

be converted into floating-point format before it can be used to train the model. We'll see how to do this in the next section on Preprocessing.

Just as with continuous audio signals, the amplitude of digital audio is typically expressed in decibels (dB). Since human hearing is logarithmic in nature — our ears are more sensitive to small fluctuations in quiet sounds than in loud sounds — the loudness of a sound is easier to interpret if the amplitudes are in decibels, which are also logarithmic. The decibel scale for real-world audio starts at 0 dB, which represents the quietest possible sound humans can hear, and louder sounds have larger values. However, for digital audio signals, 0 dB is the loudest possible amplitude, while all other amplitudes are negative. As a quick rule of thumb: every -6 dB is a halving of the amplitude, and anything below -60 dB is generally inaudible unless you really crank up the volume.

Audio as a waveform

You may have seen sounds visualized as a **waveform**, which plots the sample values over time and illustrates the changes in the sound's amplitude. This is also known as the *time domain* representation of sound.

This type of visualization is useful for identifying specific features of the audio signal such as the timing of individual sound events, the overall loudness of the signal, and any irregularities or noise present in the audio.

To plot the waveform for an audio signal, we can use a Python library called `librosa`:

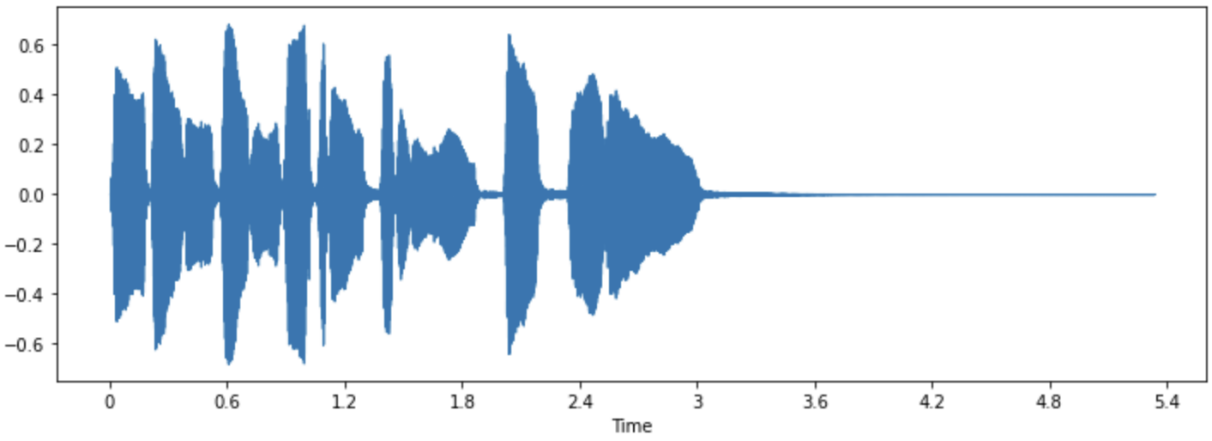
```
pip install librosa
```

Let's take an example sound called "trumpet" that comes with the library:

```
array, sampling_rate = librosa.load(librosa.ex("trumpet"))
```

The example is loaded as a tuple of audio time series (here we call it `array`), and sampling rate (`sampling_rate`). Let's take a look at this sound's waveform by using librosa's `waveshow()` function:

```
plt.figure().set_figwidth(12)
librosa.display.waveshow(array, sr=sampling_rate)
```



This plots the amplitude of the signal on the y-axis and time along the x-axis. In other words, each point corresponds to a single sample value that was taken when this sound was sampled. Also note that librosa returns the audio as floating-point values already, and that the amplitude values are indeed within the [-1.0, 1.0] range.

Visualizing the audio along with listening to it can be a useful tool for understanding the data you are working with. You can see the shape of the signal, observe patterns, learn to spot noise or distortion. If you preprocess data in some ways, such as normalization, resampling, or filtering, you can visually confirm that preprocessing steps have been applied as expected. After training a model, you can also visualize samples where errors occur (e.g. in audio classification task) to debug the issue.

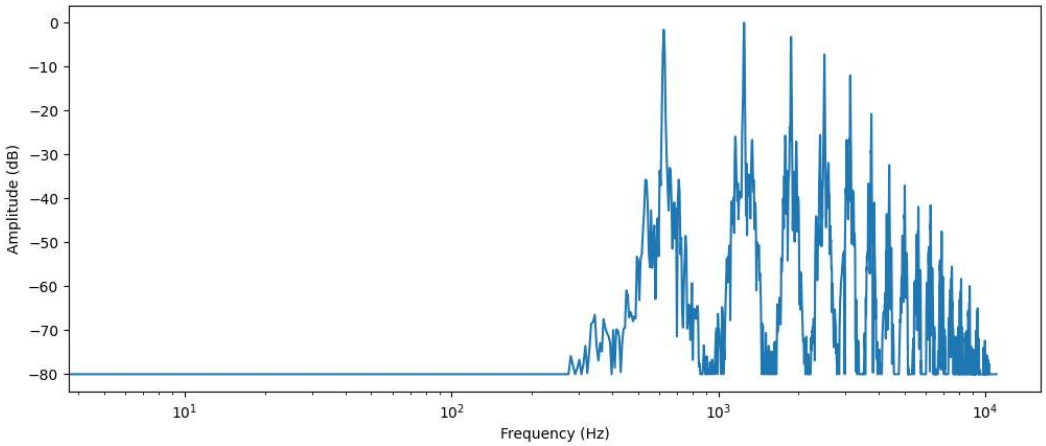
The frequency spectrum

Another way to visualize audio data is to plot the **frequency spectrum** of an audio signal, also known as the *frequency domain* representation. The spectrum is computed using the discrete Fourier transform or DFT. It describes the individual frequencies that make up the signal and how strong they are.

Let's plot the frequency spectrum for the same trumpet sound by taking the DFT using numpy's `rfft()` function. While it is possible to plot the spectrum of the entire sound, it's more useful to look at a small region instead. Here we'll take the DFT over the first 4096 samples, which is roughly the length of the first note being played:

```
dft_input = array[:4096]

# calculate the DFT
window = np.hanning(len(dft_input))
windowed_input = dft_input * window
```

```
dft = np.fft.rfft(windowed_input)

# get the amplitude spectrum in decibels
amplitude = np.abs(dft)
amplitude_db = librosa.amplitude_to_db(amplitude, ref=np.max)

# get the frequency bins
frequency = librosa.fft_frequencies(sr=sampling_rate, n_fft=len(dft_input))

plt.figure().set_figwidth(12)
plt.plot(frequency, amplitude_db)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude (dB)")
plt.xscale("log")
```



This plots the strength of the various frequency components that are present in this audio segment. The frequency values are on the x-axis, usually plotted on a logarithmic scale, while their amplitudes are on the y-axis.

The frequency spectrum that we plotted shows several peaks. These peaks correspond to the harmonics of the note that's being played, with the higher harmonics being quieter. Since the first peak is at around 620 Hz, this is the frequency spectrum of an E♭ note.

The output of the DFT is an array of complex numbers, made up of real and imaginary components. Taking the magnitude with `np.abs(dft)` extracts the amplitude information from the spectrogram. The angle between the real and imaginary components provides the so-called phase spectrum, but this is often discarded in machine learning applications.

You used `librosa.amplitude_to_db()` to convert the amplitude values to the decibel scale, making it easier to see the finer details in the spectrum. Sometimes people

use the **power spectrum**, which measures energy rather than amplitude; this is simply a spectrum with the amplitude values squared.

💡 In practice, people use the term FFT interchangeably with DFT, as the FFT or Fast Fourier Transform is the only efficient way to calculate the DFT on a computer.

The frequency spectrum of an audio signal contains the exact same information as its waveform — they are simply two different ways of looking at the same data (here, the first 4096 samples from the trumpet sound). Where the waveform plots the amplitude of the audio signal over time, the spectrum visualizes the amplitudes of the individual frequencies at a fixed point in time.

Spectrogram

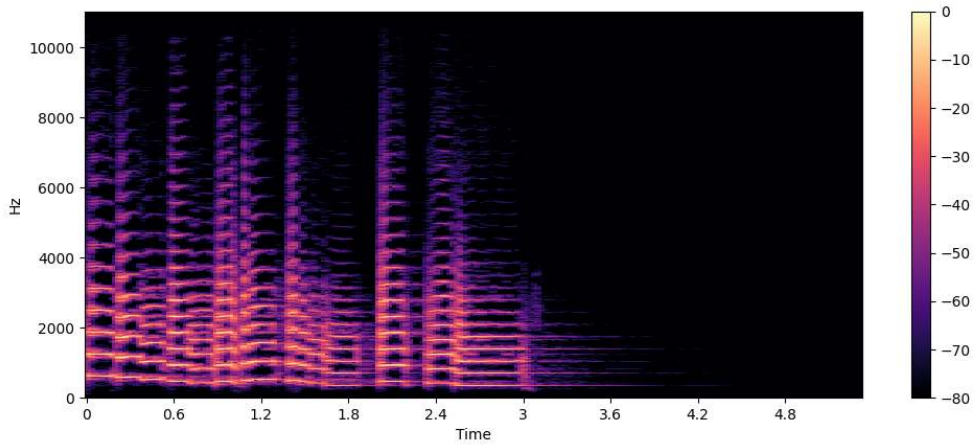
What if we want to see how the frequencies in an audio signal change? The trumpet plays several notes and they all have different frequencies. The problem is that the spectrum only shows a frozen snapshot of the frequencies at a given instant. The solution is to take multiple DFTs, each covering only a small slice of time, and stack the resulting spectra together into a **spectrogram**.

A spectrogram plots the frequency content of an audio signal as it changes over time. It allows you to see time, frequency, and amplitude all on one graph. The algorithm that performs this computation is the STFT or Short Time Fourier Transform.

The spectrogram is one of the most informative audio tools available to you. For example, when working with a music recording, you can see the various instruments and vocal tracks and how they contribute to the overall sound. In speech, you can identify different vowel sounds as each vowel is characterized by particular frequencies.

Let's plot a spectrogram for the same trumpet sound, using librosa's `stft()` and `specshow()` functions:

```
D = librosa.stft(array)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

plt.figure().set_figwidth(12)
librosa.display.specshow(S_db, x_axis="time", y_axis="hz")
plt.colorbar()
```



In this plot, the x-axis represents time as in the waveform visualization but now the y-axis represents frequency in Hz. The intensity of the color gives the amplitude or power of the frequency component at each point in time, measured in decibels (dB).

The spectrogram is created by taking short segments of the audio signal, typically lasting a few milliseconds, and calculating the discrete Fourier transform of each segment to obtain its frequency spectrum. The resulting spectra are then stacked together on the time axis to create the spectrogram. Each vertical slice in this image corresponds to a single frequency spectrum, seen from the top. By default, `librosa.stft()` splits the audio signal into segments of 2048 samples, which gives a good trade-off between frequency resolution and time resolution.

Since the spectrogram and the waveform are different views of the same data, it's possible to turn the spectrogram back into the original waveform using the inverse STFT. However, this requires the phase information in addition to the amplitude information. If the spectrogram was generated by a machine learning model, it typically only outputs the amplitudes. In that case, we can use a phase reconstruction algorithm such as the classic Griffin-Lim algorithm, or using a neural network called a vocoder, to reconstruct a waveform from the spectrogram.

Spectrograms aren't just used for visualization. Many machine learning models will take spectrograms as input — as opposed to waveforms — and produce spectrograms as output.

Now that we know what a spectrogram is and how it's made, let's take a look at a variant of it widely used for speech processing: the mel spectrogram.

Mel spectrogram

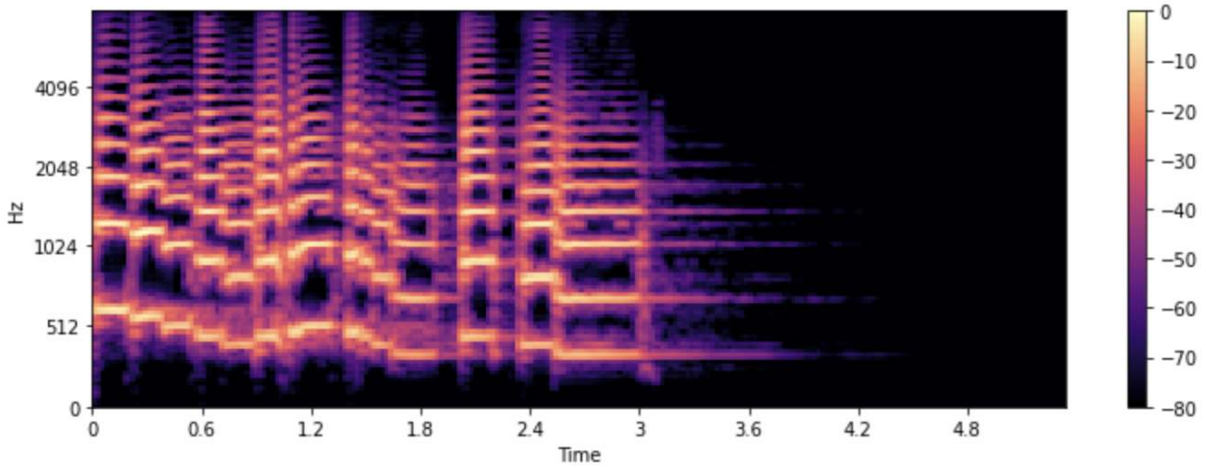
A mel spectrogram is a variation of the spectrogram that is commonly used in speech processing and machine learning tasks. It is similar to a spectrogram in that it shows the frequency content of an audio signal over time, but on a different frequency axis.

In a standard spectrogram, the frequency axis is linear and is measured in hertz (Hz). However, the human auditory system is more sensitive to changes in lower frequencies than higher frequencies, and this sensitivity decreases logarithmically as frequency increases. The mel scale is a perceptual scale that approximates the non-linear frequency response of the human ear.

To create a mel spectrogram, the STFT is used just like before, splitting the audio into short segments to obtain a sequence of frequency spectra. Additionally, each spectrum is sent through a set of filters, the so-called mel filterbank, to transform the frequencies to the mel scale.

Let's see how we can plot a mel spectrogram using librosa's `melspectrogram()` function, which performs all of those steps for us:

```
S = librosa.feature.melspectrogram(y=array, sr=sampling_rate, n_mels=128,
fmax=8000)
S_dB = librosa.power_to_db(S, ref=np.max)

plt.figure().set_figwidth(12)
librosa.display.specshow(S_dB, x_axis="time", y_axis="mel", sr=sampling_rate,
fmax=8000)
plt.colorbar()
```



In the example above, `n_mels` stands for the number of mel bands to generate. The mel bands define a set of frequency ranges that divide the spectrum into

perceptually meaningful components, using a set of filters whose shape and spacing are chosen to mimic the way the human ear responds to different frequencies. Common values for `n_mels` are 40 or 80. `fmax` indicates the highest frequency (in Hz) we care about.

Just as with a regular spectrogram, it's common practice to express the strength of the mel frequency components in decibels. This is commonly referred to as a **log-mel spectrogram**, because the conversion to decibels involves a logarithmic operation. The above example used `librosa.power_to_db()` as `librosa.feature.melspectrogram()` creates a power spectrogram.

💡 Not all mel spectrograms are the same! There are two different mel scales in common use ("htk" and "slaney"), and instead of the power spectrogram the amplitude spectrogram may be used. The conversion to a log-mel spectrogram doesn't always compute true decibels but may simply take the `log`. Therefore, if a machine learning model expects a mel spectrogram as input, double check to make sure you're computing it the same way.

Creating a mel spectrogram is a lossy operation as it involves filtering the signal. Converting a mel spectrogram back into a waveform is more difficult than doing this for a regular spectrogram, as it requires estimating the frequencies that were thrown away. This is why machine learning models such as HiFiGAN vocoder are needed to produce a waveform from a mel spectrogram.

Compared to a standard spectrogram, a mel spectrogram can capture more meaningful features of the audio signal for human perception, making it a popular choice in tasks such as speech recognition, speaker identification, and music genre classification.

Now that you know how to visualize audio data examples, go ahead and try to see what your favorite sounds look like. :)

Unit 1. Working with audio data

What you'll learn in this unit

Every audio or speech task starts with an audio file. Before we can dive into solving these tasks, it's important to understand what these files actually contain, and how to work with them.

In this unit, you will gain an understanding of the fundamental terminology related to audio data, including waveform, sampling rate, and spectrogram. You will also learn how to work with audio datasets, including loading and preprocessing audio data, and how to stream large datasets efficiently.

By the end of this unit, you will have a strong grasp of the essential audio data terminology and will be equipped with the skills necessary to work with audio datasets for various applications. The knowledge you'll gain in this unit is going to lay a foundation to understanding the remainder of the course.

Load and explore an audio dataset

In this course we will use the 🤗 Datasets library to work with audio datasets. 🤗 Datasets is an open-source library for downloading and preparing datasets from all modalities including audio. The library offers easy access to an unparalleled selection of machine learning datasets publicly available on Hugging Face Hub. Moreover, 🤗 Datasets includes multiple features tailored to audio datasets that simplify working with such datasets for both researchers and practitioners.

To begin working with audio datasets, make sure you have the 🤗 Datasets library installed along with the audio dependencies:

```
pip install datasets[audio] soundfile librosa torchcodec
```

One of the key defining features of 🤗 Datasets is the ability to download and prepare a dataset in just one line of Python code using the `load_dataset()` function.

Let's load and explore and audio dataset called MINDS-14, which contains recordings of people asking an e-banking system questions in several languages and dialects.

To load the MINDS-14 dataset, we need to copy the dataset's identifier on the Hub (`PolyAI/minds14`) and pass it to the `load_dataset` function. We'll also specify that we're only interested in the Australian subset (`en-AU`) of the data, and limit it to the training split:

```
from datasets import load_dataset

minds = load_dataset("PolyAI/minds14", name="en-AU", split="train")
minds
```

Output:

```
Dataset(

)
```

The dataset contains 654 audio files, each of which is accompanied by a transcription, an English translation, and a label indicating the intent behind the person's query. The audio column contains the raw audio data. Let's take a closer look at one of the examples:

```
example = minds[0]
example
```

Output:

```
,
    "transcription": "I would like to pay my electricity bill using my card can you
please assist",
    "english_transcription": "I would like to pay my electricity bill using my card
can you please assist",
    "intent_class": 13,
    "lang_id": 2,
}
```

You may notice that the audio column contains several features. Here's what they are: * `path`: the path to the audio file (`*.wav` in this case). * `array`: The decoded audio data, represented as a 1-dimensional NumPy array. * `sampling_rate`. The sampling rate of the audio file (8,000 Hz in this example).

The `intent_class` is a classification category of the audio recording. To convert this number into a meaningful string, we can use the `int2str()` method:

```
id2label = minds.features["intent_class"].int2str
id2label(example["intent_class"])
```

Output:

```
"pay_bill"
```

If you look at the transcription feature, you can see that the audio file indeed has recorded a person asking a question about paying a bill.

If you plan to train an audio classifier on this subset of data, you may not necessarily need all of the features. For example, the `lang_id` is going to have the same value

for all examples, and won't be useful. The `english_transcription` will likely duplicate the `transcription` in this subset, so we can safely remove them.

You can easily remove irrelevant features using 🤗 Datasets' `remove_columns` method:

```
columns_to_remove = ["lang_id", "english_transcription"]
minds = minds.remove_columns(columns_to_remove)
minds
```

Output:

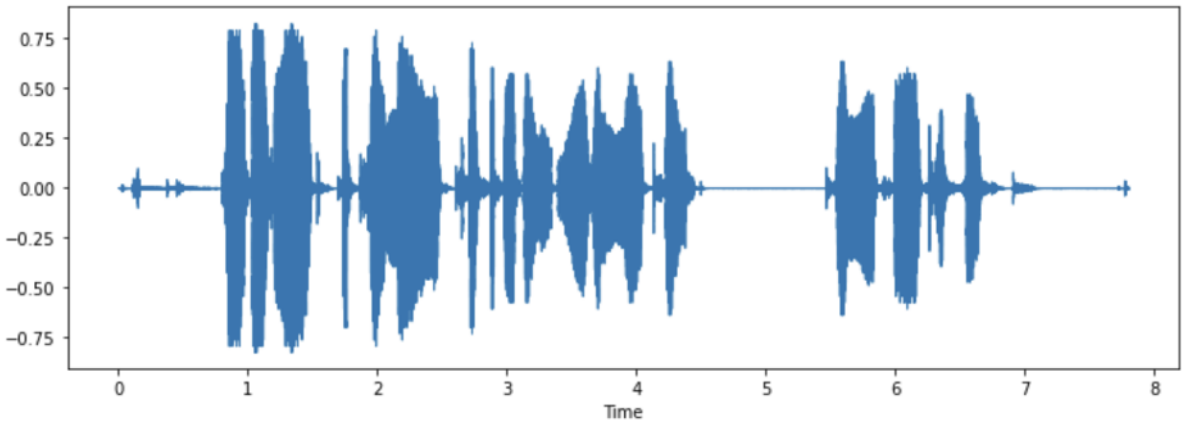
```
Dataset()
```

Now that we've loaded and inspected the raw contents of the dataset, let's listen to a few examples! We'll use the `Blocks` and `Audio` features from `Gradio` to decode a few random samples from the dataset:

```
def generate_audio():
    example = minds.shuffle()[0]
    audio = example["audio"]
    return (
        audio["sampling_rate"],
        audio["array"],
    ), id2label(example["intent_class"])


with gr.Blocks() as demo:
    with gr.Column():
        for _ in range(4):
            audio, label = generate_audio()
            output = gr.Audio(audio, label=label)

demo.launch(debug=True)
```

If you'd like to, you can also visualize some of the examples. Let's plot the waveform for the first example.

```
array = example["audio"]["array"]
sampling_rate = example["audio"]["sampling_rate"]

plt.figure().set_figwidth(12)
librosa.display.waveshow(array, sr=sampling_rate)
```



Try it out! Download another dialect or language of the MINDS-14 dataset, listen and visualize some examples to get a sense of the variation in the whole dataset. You can find the full list of available languages here.

Preprocessing an audio dataset

Loading a dataset with 🤗 Datasets is just half of the fun. If you plan to use it either for training a model, or for running inference, you will need to pre-process the data first. In general, this will involve the following steps:

- Resampling the audio data
- Filtering the dataset
- Converting audio data to model's expected input

Resampling the audio data

The `load_dataset` function downloads audio examples with the sampling rate that they were published with. This is not always the sampling rate expected by a model you plan to train, or use for inference. If there's a discrepancy between the sampling rates, you can resample the audio to the model's expected sampling rate.

Most of the available pretrained models have been pretrained on audio datasets at a sampling rate of 16 kHz. When we explored MINDS-14 dataset, you may have noticed that it is sampled at 8 kHz, which means we will likely need to upsample it.

To do so, use 🤗 Datasets' `cast_column` method. This operation does not change the audio in-place, but rather signals to datasets to resample the audio examples on the fly when they are loaded. The following code will set the sampling rate to 16kHz:

```
from datasets import Audio

minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
```

Re-load the first audio example in the MINDS-14 dataset, and check that it has been resampled to the desired `sampling rate`:

```
minds[0]
```

Output:

```
,
    "transcription": "I would like to pay my electricity bill using my card can you
please assist",
    "intent_class": 13,
}
```

You may notice that the array values are now also different. This is because we've now got twice the number of amplitude values for every one that we had before.

💡 Some background on resampling: If an audio signal has been sampled at 8 kHz, so that it has 8000 sample readings per second, we know that the audio does not contain any frequencies over 4 kHz. This is guaranteed by the Nyquist sampling theorem. Because of this, we can be certain that in between the sampling points the original continuous signal always makes a smooth curve. Upsampling to a higher sampling rate is then a matter of calculating additional sample values that go in between the existing ones, by approximating this curve. Downsampling, however, requires that we first filter out any frequencies that would be higher than the new Nyquist limit, before estimating the new sample points. In other words, you can't downsample by a factor 2x by simply throwing away every other sample — this will create distortions in the signal called aliases. Doing resampling correctly is tricky and best left to well-tested libraries such as librosa or 🤗 Datasets.

Filtering the dataset

You may need to filter the data based on some criteria. One of the common cases involves limiting the audio examples to a certain duration. For instance, we might want to filter out any examples longer than 20s to prevent out-of-memory errors when training a model.

We can do this by using the 🤗 Datasets' `filter` method and passing a function with filtering logic to it. Let's start by writing a function that indicates which examples to

keep and which to discard. This function, `is_audio_length_in_range`, returns `True` if a sample is shorter than 20s, and `False` if it is longer than 20s.

```
MAX_DURATION_IN_SECONDS = 20.0


def is_audio_length_in_range(input_length):
    return input_length < MAX_DURATION_IN_SECONDS
```

The filtering function can be applied to a dataset's column but we do not have a column with audio track duration in this dataset. However, we can create one, filter based on the values in that column, and then remove it.

```
# use librosa to get example's duration from the audio file
new_column = [
    librosa.get_duration(y=x["array"], sr=x["sampling_rate"]) for x in
minds["audio"]
]
minds = minds.add_column("duration", new_column)

# use 🤗 Datasets' `filter` method to apply the filtering function
minds = minds.filter(is_audio_length_in_range, input_columns=["duration"])

# remove the temporary helper column
minds = minds.remove_columns(["duration"])
minds
```

Output:

```
Dataset()
```

We can verify that dataset has been filtered down from 654 examples to 624.

Pre-processing audio data

One of the most challenging aspects of working with audio datasets is preparing the data in the right format for model training. As you saw, the raw audio data comes as an array of sample values. However, pre-trained models, whether you use them for inference, or want to fine-tune them for your task, expect the raw data to be converted into input features. The requirements for the input features may vary from one model to another — they depend on the model's architecture, and the data it was pre-trained with. The good news is, for every supported audio model, 🤗 Transformers offer a feature extractor class that can convert raw audio data into the input features the model expects.

So what does a feature extractor do with the raw audio data? Let's take a look at Whisper's feature extractor to understand some common feature extraction transformations. Whisper is a pre-trained model for automatic speech recognition (ASR) published in September 2022 by Alec Radford et al. from OpenAI.

First, the Whisper feature extractor pads/truncates a batch of audio examples such that all examples have an input length of 30s. Examples shorter than this are padded to 30s by appending zeros to the end of the sequence (zeros in an audio signal correspond to no signal or silence). Examples longer than 30s are truncated to 30s. Since all elements in the batch are padded/truncated to a maximum length in the input space, there is no need for an attention mask. Whisper is unique in this regard, most other audio models require an attention mask that details where sequences have been padded, and thus where they should be ignored in the self-attention mechanism. Whisper is trained to operate without an attention mask and infer directly from the speech signals where to ignore the inputs.

The second operation that the Whisper feature extractor performs is converting the padded audio arrays to log-mel spectrograms. As you recall, these spectrograms describe how the frequencies of a signal change over time, expressed on the mel scale and measured in decibels (the log part) to make the frequencies and amplitudes more representative of human hearing.

All these transformations can be applied to your raw audio data with a couple of lines of code. Let's go ahead and load the feature extractor from the pre-trained Whisper checkpoint to have ready for our audio data:

```
from transformers import WhisperFeatureExtractor

feature_extractor = WhisperFeatureExtractor.from_pretrained("openai/whisper-small")
```

Next, you can write a function to pre-process a single audio example by passing it through the `feature_extractor`.

```
def prepare_dataset(example):
    audio = example["audio"]

    if audio["sampling_rate"] != 16000:
        audio_array = librosa.resample(
            audio["array"], orig_sr=audio["sampling_rate"], target_sr=16000
        )
        audio =

    features = feature_extractor(
        audio["array"], sampling_rate=audio["sampling_rate"], padding=True
```

```
    )
    return features
```

We can apply the data preparation function to all of our training examples using 🤗 Datasets' map method:

```
minds = minds.map(prepare_dataset)
minds
```

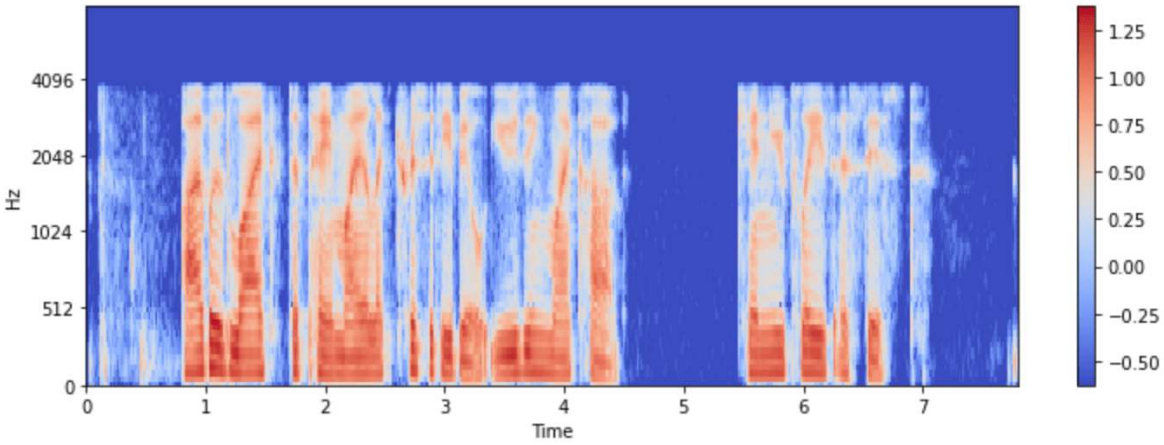
Output:

```
Dataset(

)
```

As easy as that, we now have log-mel spectrograms as `input_features` in the dataset.

Let's visualize it for one of the examples in the `minds` dataset:

```
example = minds[0]
input_features = example["input_features"]

plt.figure().set_figwidth(12)
librosa.display.specshow(
    np.asarray(input_features[0]),
    x_axis="time",
    y_axis="mel",
    sr=feature_extractor.sampling_rate,
    hop_length=feature_extractor.hop_length,
)
plt.colorbar()
```



Now you can see what the audio input to the Whisper model looks like after preprocessing.

The model's feature extractor class takes care of transforming raw audio data to the format that the model expects. However, many tasks involving audio are multimodal, e.g. speech recognition. In such cases 🤗 Transformers also offer model-specific tokenizers to process the text inputs. For a deep dive into tokenizers, please refer to our NLP course.

You can load the feature extractor and tokenizer for Whisper and other multimodal models separately, or you can load both via a so-called processor. To make things even simpler, use `AutoProcessor` to load a model's feature extractor and processor from a checkpoint, like this:

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained("openai/whisper-small")
```

Here we have illustrated the fundamental data preparation steps. Of course, custom data may require more complex preprocessing. In this case, you can extend the function `prepare_dataset` to perform any sort of custom data transformations. With 🤗 Datasets, if you can write it as a Python function, you can apply it to your dataset!

Check your understanding of the course material

1. What units is the sampling rate measured in?

<Question choices=, ,

```
]}
```

/>

2. When streaming a large audio dataset, how soon can you start using it?

<Question choices=, ,

```
]}
```

/>

3. What is a spectrogram?

<Question choices=, ,

```
]}
```

/>

4. What is the easiest way to convert raw audio data into log-mel spectrogram expected by Whisper?

A.

```
librosa.feature.melspectrogram(audio["array"])
```

B.

```
feature_extractor = WhisperFeatureExtractor.from_pretrained("openai/whisper-small")
feature_extractor(audio["array"])
```

C.

```
dataset.feature(audio["array"], model="whisper")
```

<Question choices=, ,

```
]}
```

/>

5. How do you load a dataset from 🤗 Hub?

A.

```
from datasets import load_dataset

dataset = load_dataset(DATASET_NAME_ON_HUB)
```

B.

```
dataset = librosa.load(PATH_TO_DATASET)
```

C.

```
from transformers import load_dataset

dataset = load_dataset(DATASET_NAME_ON_HUB)
```

<Question choices=, ,

```
]}
```

/>

6. Your custom dataset contains high-quality audio with 32 kHz sampling rate. You want to train a speech recognition model that expects the audio examples to have a 16 kHz sampling rate. What should you do?

<Question choices=, ,

```
]}
```

/>

7. How can you convert a spectrogram generated by a machine learning model into a waveform?

<Question choices=, ,

```
]}
```

/>

Streaming audio data

One of the biggest challenges faced with audio datasets is their sheer size. A single minute of uncompressed CD-quality audio (44.1kHz, 16-bit) takes up a bit more than 5 MB of storage. Typically, an audio dataset would contains hours of recordings.

In the previous sections we used a very small subset of MINDS-14 audio dataset, however, typical audio datasets are much larger. For example, the `xs` (smallest) configuration of GigaSpeech from SpeechColab contains only 10 hours of training data, but takes over 13GB of storage space for download and preparation. So what happens when we want to train on a larger split? The full `xl` configuration of the same dataset contains 10,000 hours of training data, requiring over 1TB of storage space. For most of us, this well exceeds the specifications of a typical hard drive disk. Do we need to fork out and buy additional storage? Or is there a way we can train on these datasets with no disk space constraints?

🤗 Datasets comes to the rescue by offering the streaming mode. Streaming allows us to load the data progressively as we iterate over the dataset. Rather than downloading the whole dataset at once, we load the dataset one example at a time. We iterate over the dataset, loading and preparing examples on the fly when they are needed. This way, we only ever load the examples that we're using, and not the ones that we're not! Once we're done with an example sample, we continue iterating over the dataset and load the next one.

Streaming mode has three primary advantages over downloading the entire dataset at once:

- Disk space: examples are loaded to memory one-by-one as we iterate over the dataset. Since the data is not downloaded locally, there are no disk space requirements, so you can use datasets of arbitrary size.
- Download and processing time: audio datasets are large and need a significant amount of time to download and process. With streaming, loading and processing is done on the fly, meaning you can start using the dataset as soon as the first example is ready.
- Easy experimentation: you can experiment on a handful of examples to check that your script works without having to download the entire dataset.

There is one caveat to streaming mode. When downloading a full dataset without streaming, both the raw data and processed data are saved locally to disk. If we want to re-use this dataset, we can directly load the processed data from disk, skipping the download and processing steps. Consequently, we only have to perform

the downloading and processing operations once, after which we can re-use the prepared data.

With streaming mode, the data is not downloaded to disk. Thus, neither the downloaded nor pre-processed data are cached. If we want to re-use the dataset, the streaming steps must be repeated, with the audio files loaded and processed on the fly again. For this reason, it is advised to download datasets that you are likely to use multiple times.

How can you enable streaming mode? Easy! Just set `streaming=True` when you load your dataset. The rest will be taken care for you:

```
gigaspeech = load_dataset("speechcolab/gigaspeech", "xs", streaming=True)
```

Just like we applied preprocessing steps to a downloaded subset of MINDS-14, you can do the same preprocessing with a streaming dataset in the exactly the same manner.

The only difference is that you can no longer access individual samples using Python indexing (i.e. `gigaspeech["train"][sample_idx]`). Instead, you have to iterate over the dataset. Here's how you can access an example when streaming a dataset:

```
next(iter(gigaspeech["train"]))
```

Output:

```
,
    "begin_time": 2941.89,
    "end_time": 2945.07,
    "audio_id": "YOU0000000315",
    "title": "Return to Vasselheim | Critical Role: VOX MACHINA | Episode 43",
    "url": "https://www.youtube.com/watch?v=zr2n1fLVasU",
    "source": 2,
    "category": 24,
    "original_full_path": "audio/youtube/P0004/YOU0000000315.opus",
}
```

If you'd like to preview several examples from a large dataset, use the `take()` to get the first n elements. Let's grab the first two examples in the gigaspeech dataset:

```
gigaspeech_head = gigaspeech["train"].take(2)
list(gigaspeech_head)
```

Output:

```
[
    ,
        "begin_time": 2941.89,
        "end_time": 2945.07,
        "audio_id": "YOU0000000315",
        "title": "Return to Vasselheim | Critical Role: VOX MACHINA | Episode 43",
        "url": "https://www.youtube.com/watch?v=zr2n1fLVasU",
        "source": 2,
        "category": 24,
        "original_full_path": "audio/youtube/P0004/YOU0000000315.opus",
    },
    ,
        "begin_time": 3673.96,
        "end_time": 3675.26,
        "audio_id": "AUD0000001043",
        "title": "Asteroid of Fear",
        "url": "http//www.archive.org/download/asteroid_of_fear_1012_librivox/
asteroid_of_fear_1012_librivox_64kb_mp3.zip",
        "source": 0,
        "category": 28,
        "original_full_path": "audio/audiobook/P0011/AUD0000001043.opus",
    },
]
```


Streaming mode can take your research to the next level: not only are the biggest datasets accessible to you, but you can easily evaluate systems over multiple datasets in one go without worrying about your disk space. Compared to evaluating on a single dataset, multi-dataset evaluation gives a better metric for the generalisation abilities of a speech recognition system (c.f. End-to-end Speech Benchmark (ESB)).

Learn more

This unit covered many fundamental concepts relevant to understanding of audio data and working with it. Want to learn more? Here you will find additional resources that will help you deepen your understanding of the topics and enhance your learning experience.

In the following video, Monty Montgomery from xiph.org presents a real-time demonstrations of sampling, quantization, bit-depth, and dither on real audio equipment using both modern digital analysis and vintage analog bench equipment, check it out:

If you'd like to dive deeper into digital signal processing, check out the free "Digital Signals Theory" book authored by Brian McFee, an Assistant Professor of Music

Technology and Data Science at New York University and the principal maintainer of the `librosa` package.